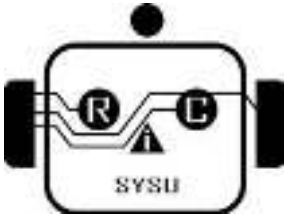


中山大學
SUN YAT-SEN UNIVERSITY



计算机组成原理

第四章：*MIPS* CPU逻辑设计

中山大学计算机学院
陈刚

2024年秋季

流水线竞争

在流水线各个阶段,指令之间可能存在关联

✓– 下一条指令可能需要某些无法获得的资源
(访问权?新的数值?到底该怎么办的指示等等)

✓– If all instructions are dependent

- No advantage of a shorter cycle (since all must wait)
- This will limit clock cycle time

这些限制就是

——**流水线的竞争Pipeline Hazards**

流水线竞争

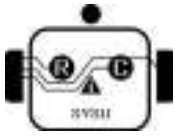
- **Structural Hazard (Resource Conflict)**
  - 两条指令需要使用同样的硬件块
- **Data Hazard**
  - 指令依赖于流水线中运行的其他指令结果
- **Control Hazard**
  - 取指令依赖于流水线中指令的结果

Control Hazards: Deal with instructions fetched before a branch or jump determines the next PC.

流水线冒险的处理

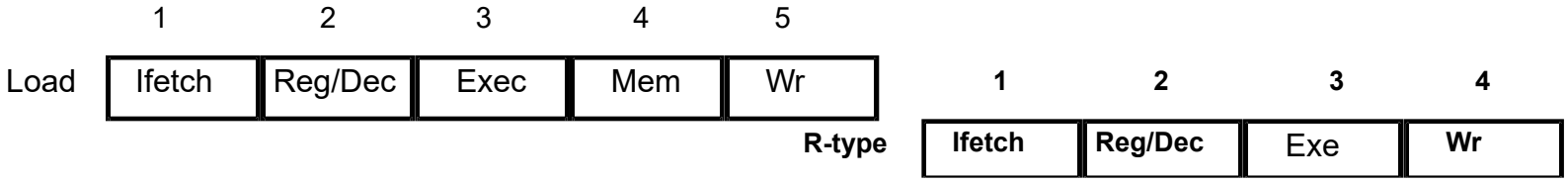
- 流水线冒险的几种类型
- 数据冒险的现象和对策
  - 数据冒险的种类
    - 相关的数据是ALU结果：可以通过转发解决
    - 相关的数据是DM读出的内容：随后的指令需被阻塞一个时钟
  - 数据冒险和转发
    - 转发检测 / 转发控制
  - 数据冒险和阻塞
    - 阻塞检测 / 阻塞控制
- 控制冒险的现象和对策
  - 静态分支预测技术
  - 动态分支预测技术
  - 缩短分支延迟技术
- 流水线中对异常和中断的处理
- 访问缺失对流水线的影响



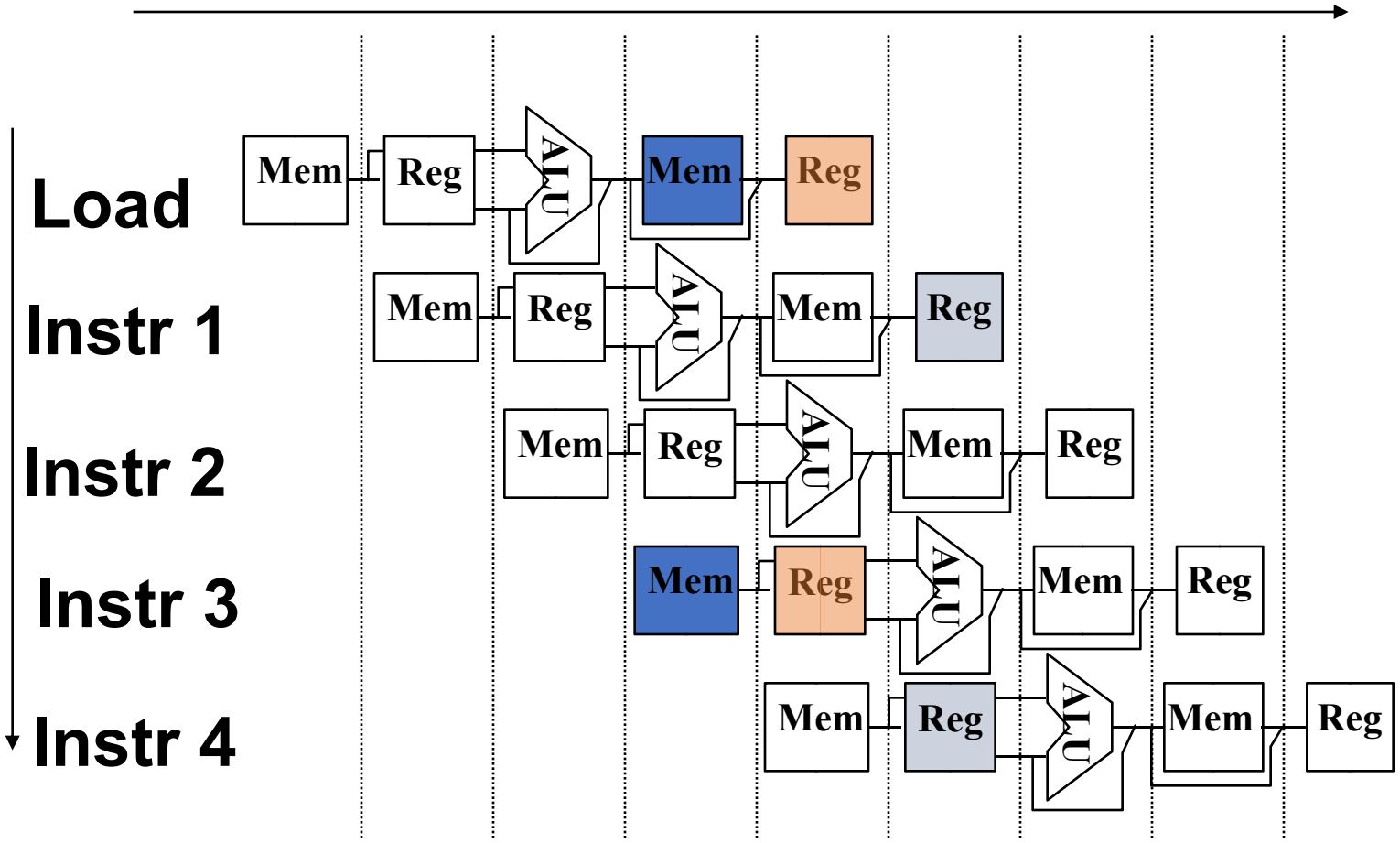


结构流水线竞争

- **资源使用冲突**
  - ✓定义为: 当两条指令需要同时访问相同硬件资源时
  - ✓经常当某个单元未完全按照流水线方式实现时发生
- **为避免结构竞争**
  - ✓限制资源使用方式,每个指令中只对某资源使用一次
  - ✓对相同资源的访问对于不同类型的指令都在同一周期进行
- **引起结构竞争的例子:**
  - ✓对load指令采用5级流水第5级回写寄存器
  - ✓对R类指令采用4级流水第4级回写寄存器



Structural Hazard（结构冒险）现象

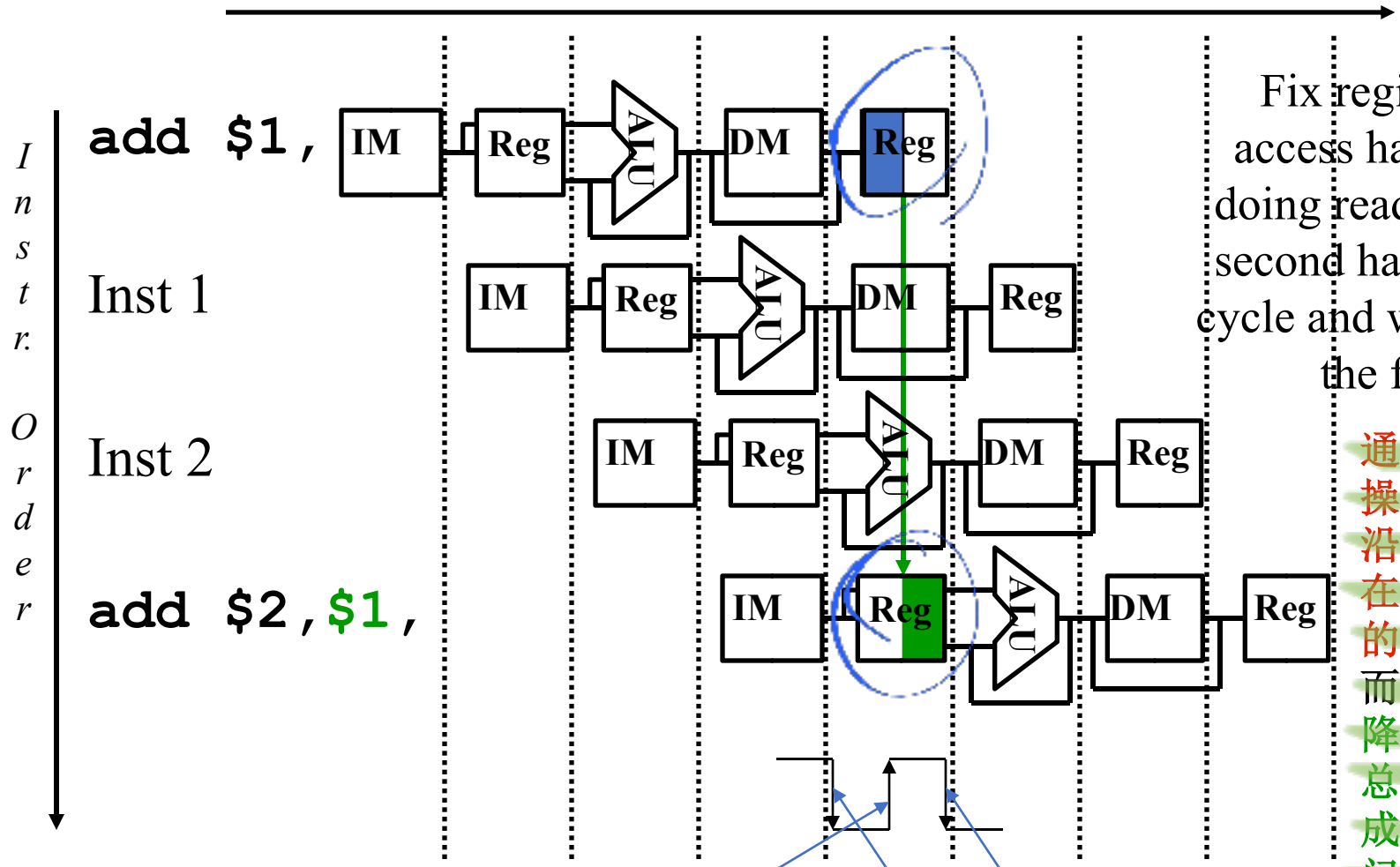


如果只有一个存储器，则在Load指令取数据同时又取指令的话，则发生冲突！

如果不对寄存器堆的写口和读口独立设置的话，则发生冲突！

结构冒险也称为硬件资源冲突：同一个执行部件被多条指令使用。

How About Register File Access?



Fix register file access hazard by doing reads in the second half of the cycle and writes in the first half

通过使写入操作（上升沿触发）总在一个周期的半段完成而读出（下降沿触发）总在半段完成来避免此问题.

clock edge that controls register writing

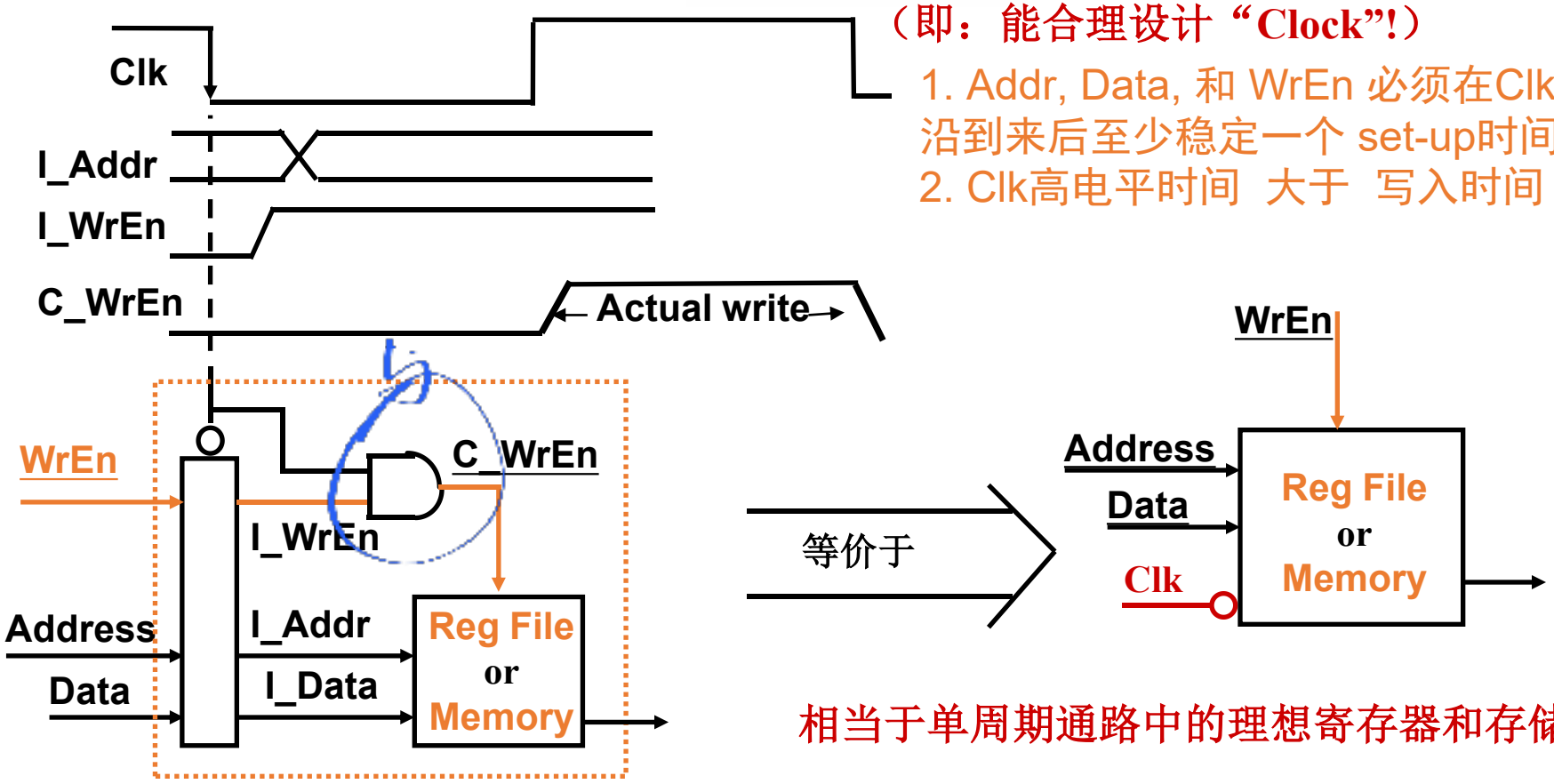
clock edge that controls loading of pipeline state registers

寄存器组的同步和存储器的同步

- 解决方案： 将Write Enable和时钟信号“与”

须由电路专家确保不会发生“定时错误”
（即：能合理设计“Clock”!）

1. Addr, Data, 和 WrEn 必须在Clk边沿到来后至少稳定一个 set-up时间
2. Clk高电平时间 大于 写入时间



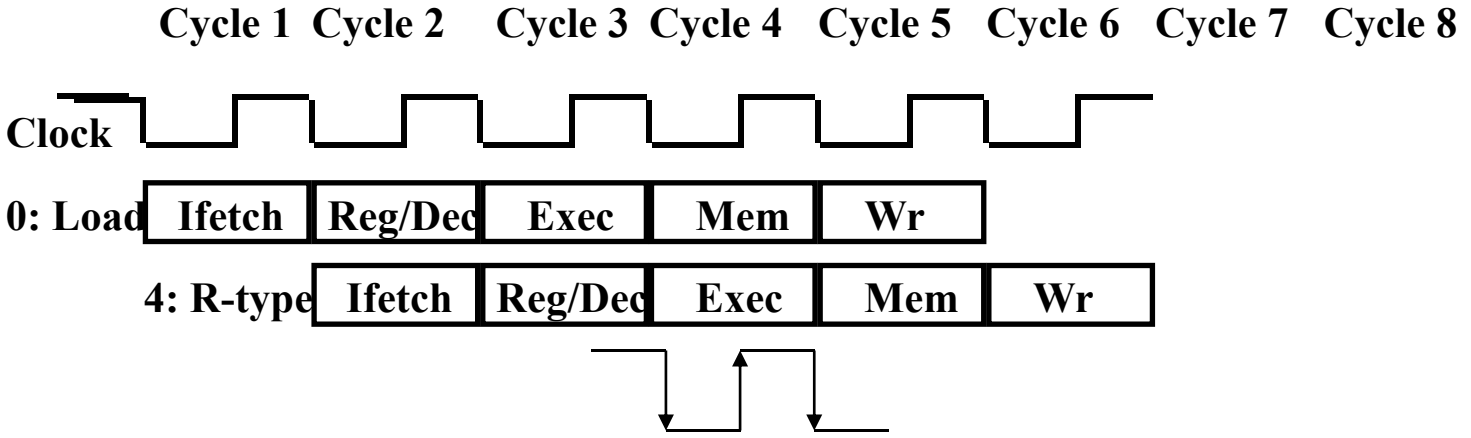
相当于单周期通路中的理想寄存器和存储器

Load 指令采用第五周期上升沿写入的状态：

Clk4上升沿后
指令流：结果存入MEM/WB

Clk5 开始的下降沿从MEM/WB读取到的addr和data有关数据送到寄存器，
Clk5 上升沿 结果存入寄存器

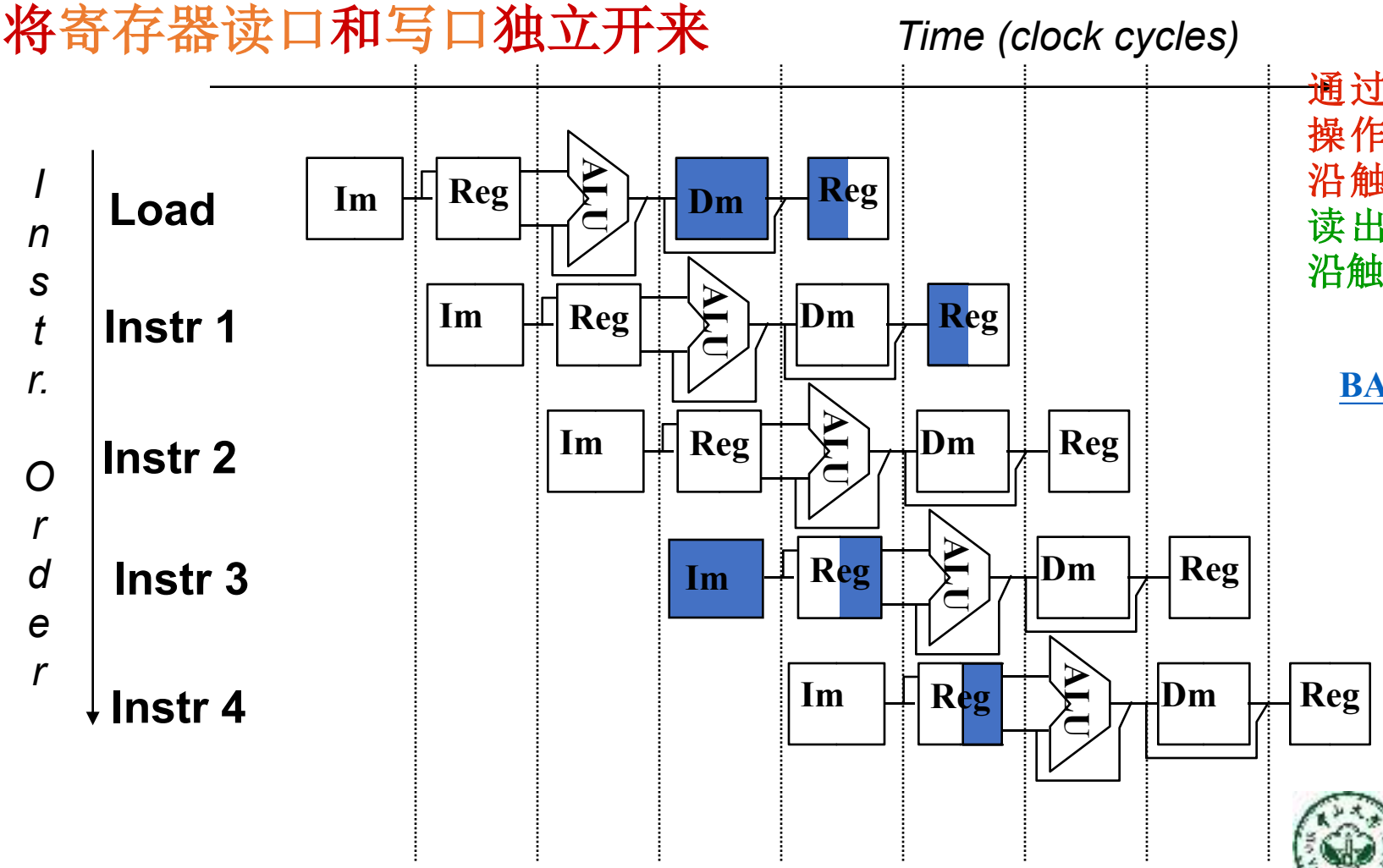
如果采用转发技术，在Clk5 开始的下降沿即可从MEM/WB读取到有关数据



仍需延迟一个时钟周期

Structural Hazard的解决方法

将Instruction Memory (Im) 和
Data Memory (Dm)分开
将寄存器读口和写口独立开来



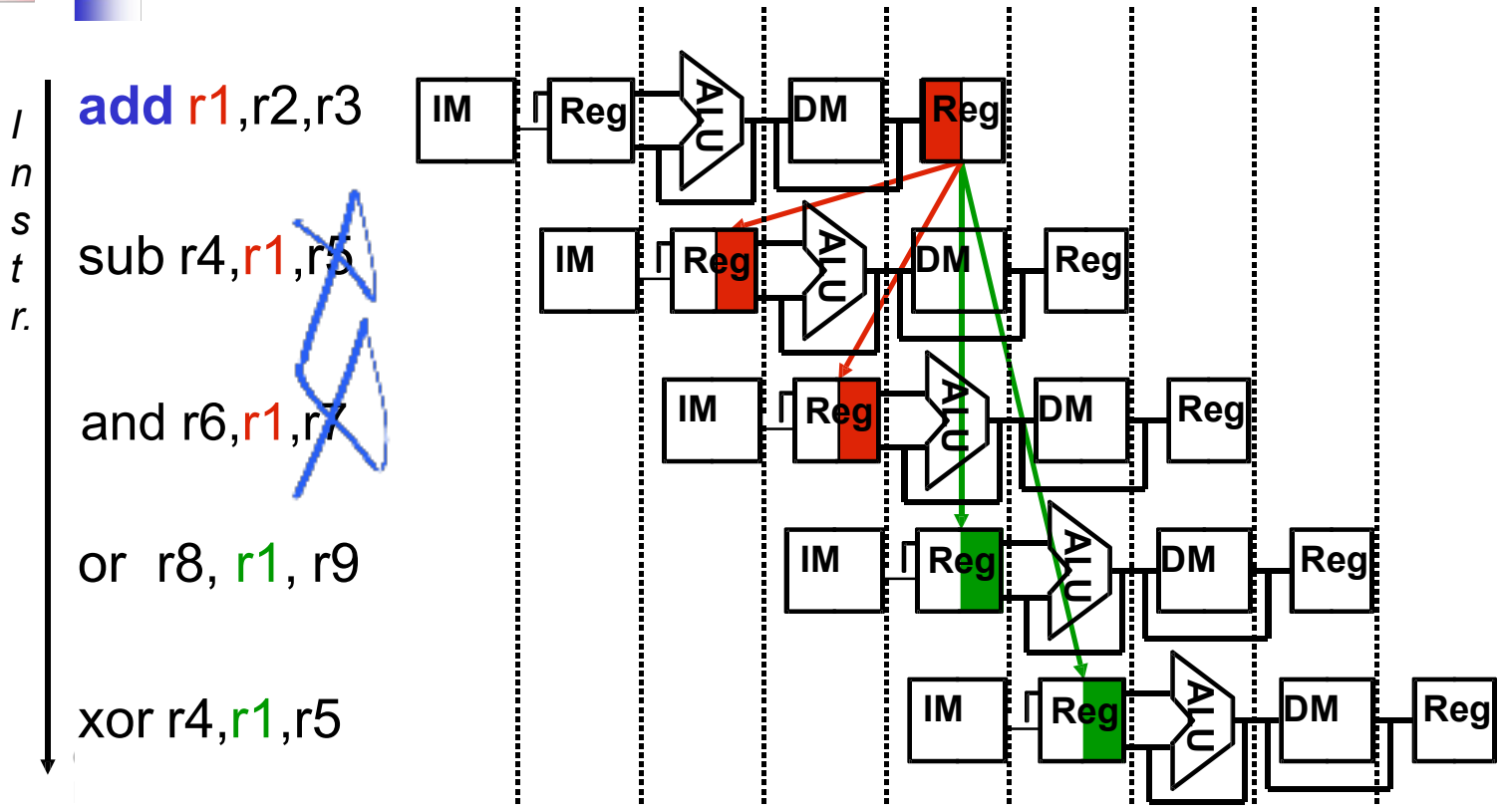
通过使写入操作（上升沿触发）而读出（下降沿触发）

BACK

Data Hazard现象

寄存器的使用引起竞争

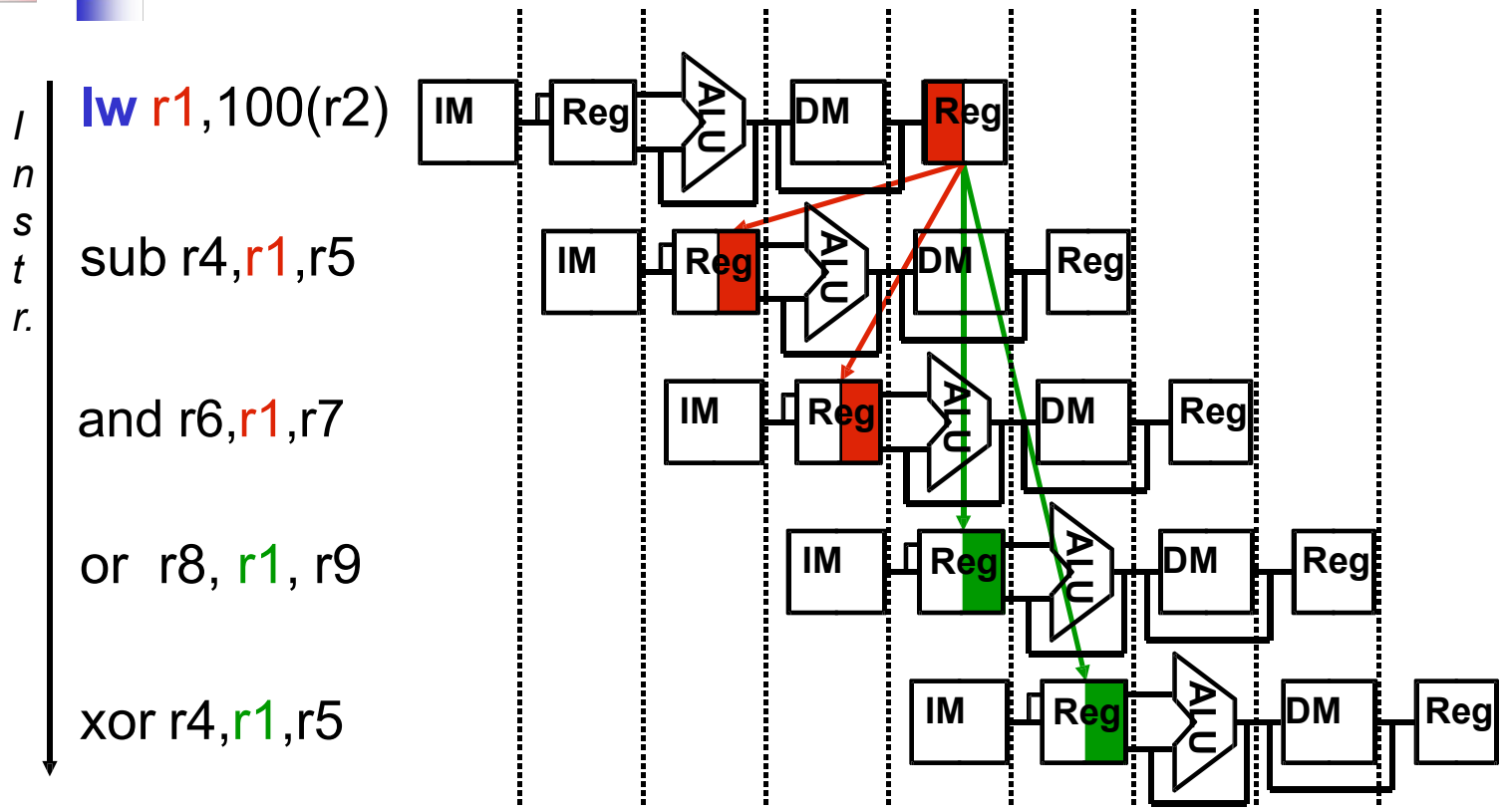
指令在时间顺序关于寄存器使用上的关联引起了数据竞争



哪些是 read before write data hazards?

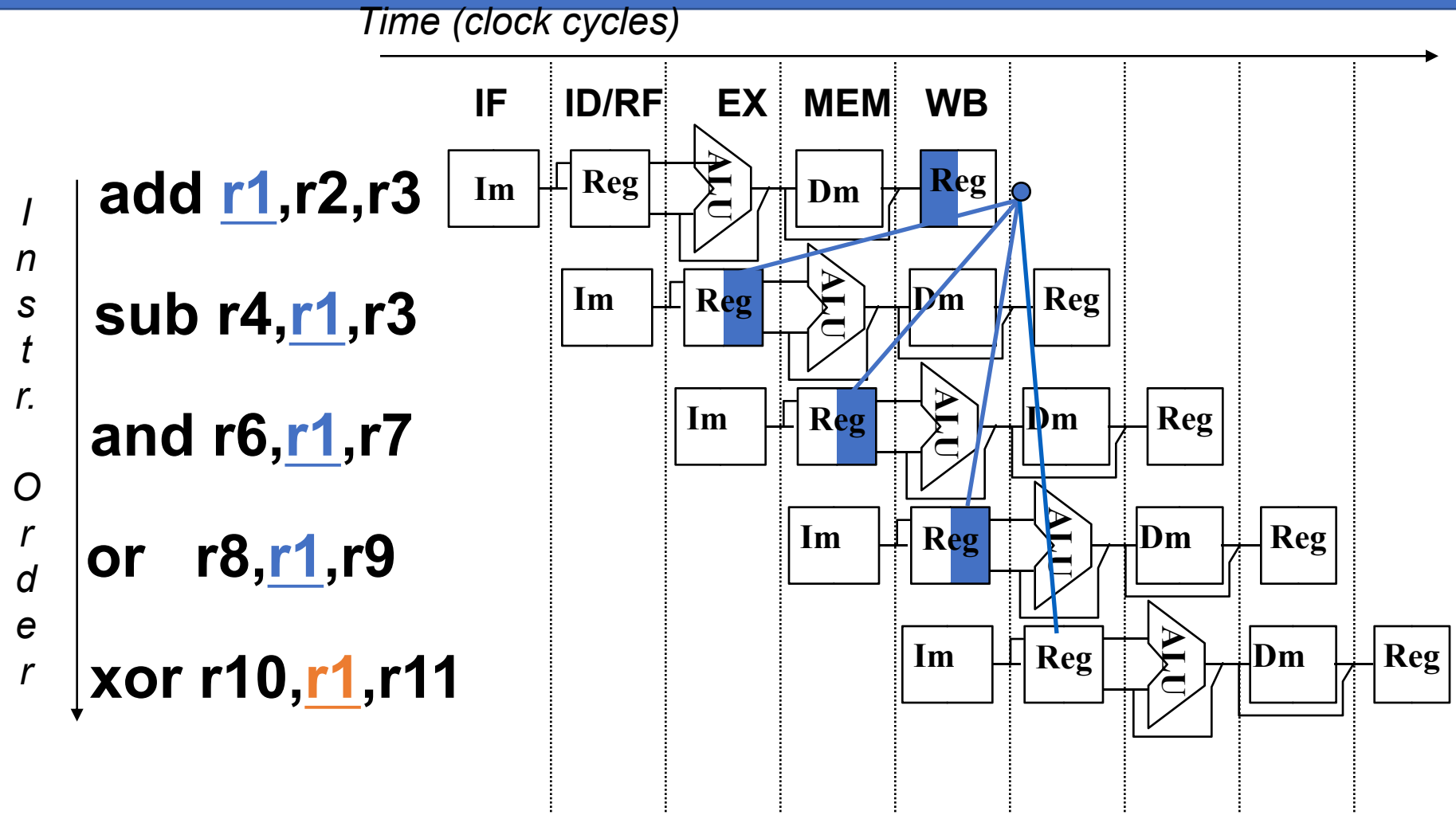
LOAD指令引起的竞争

指令在时间顺序关于寄存器使用上的关联引起了数据竞争



Load-use data hazard

Data Hazard on r1



最后一条指令的r1才是新的值！

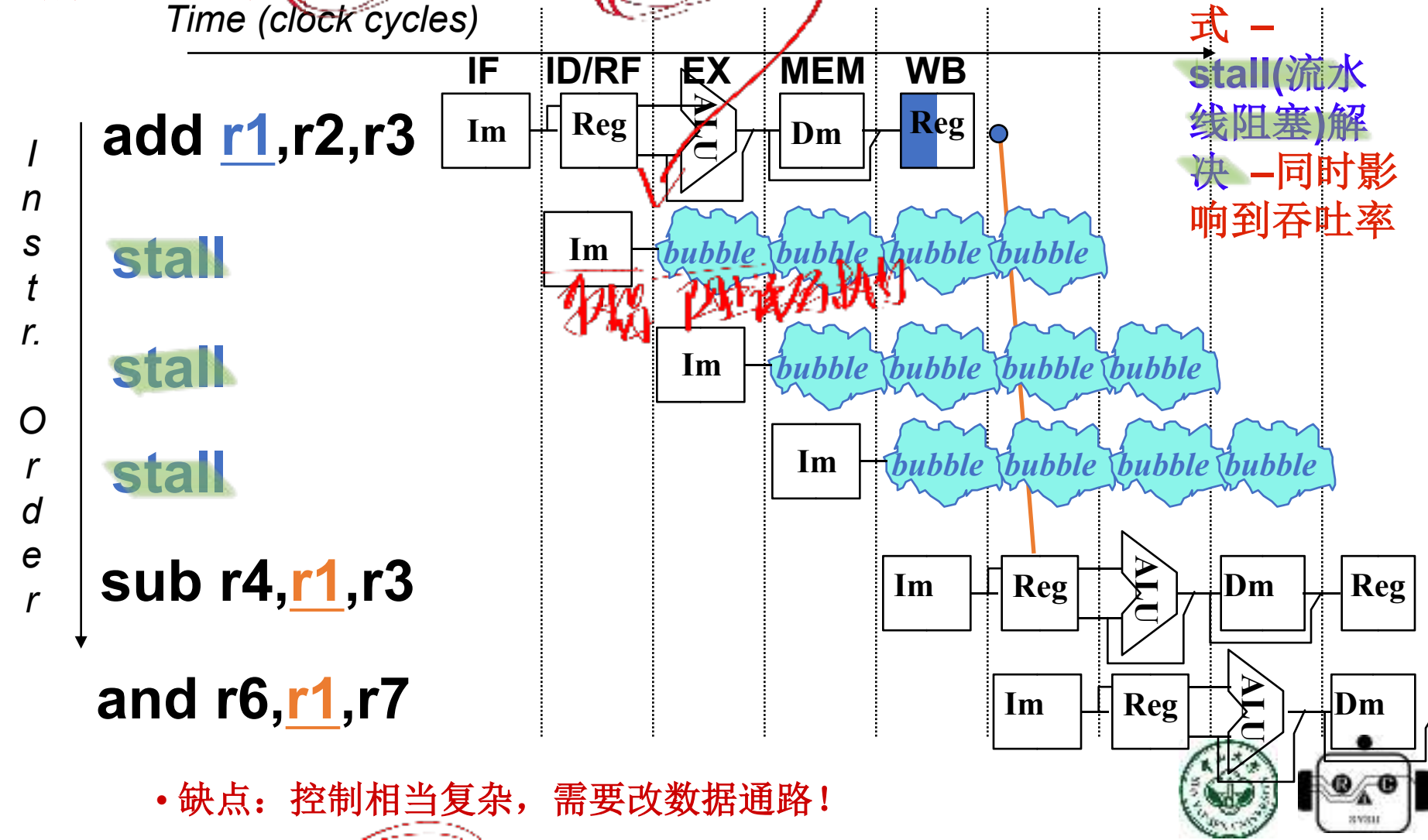
如何解决这个问题？

方案1：在硬件上采取措施，使相关指令延迟执行

- 硬件上通过阻塞(stall)方式阻止后续指令执行，延迟到有新值以后！

这种做法称为流水线阻塞，也称为“气泡Bubble”

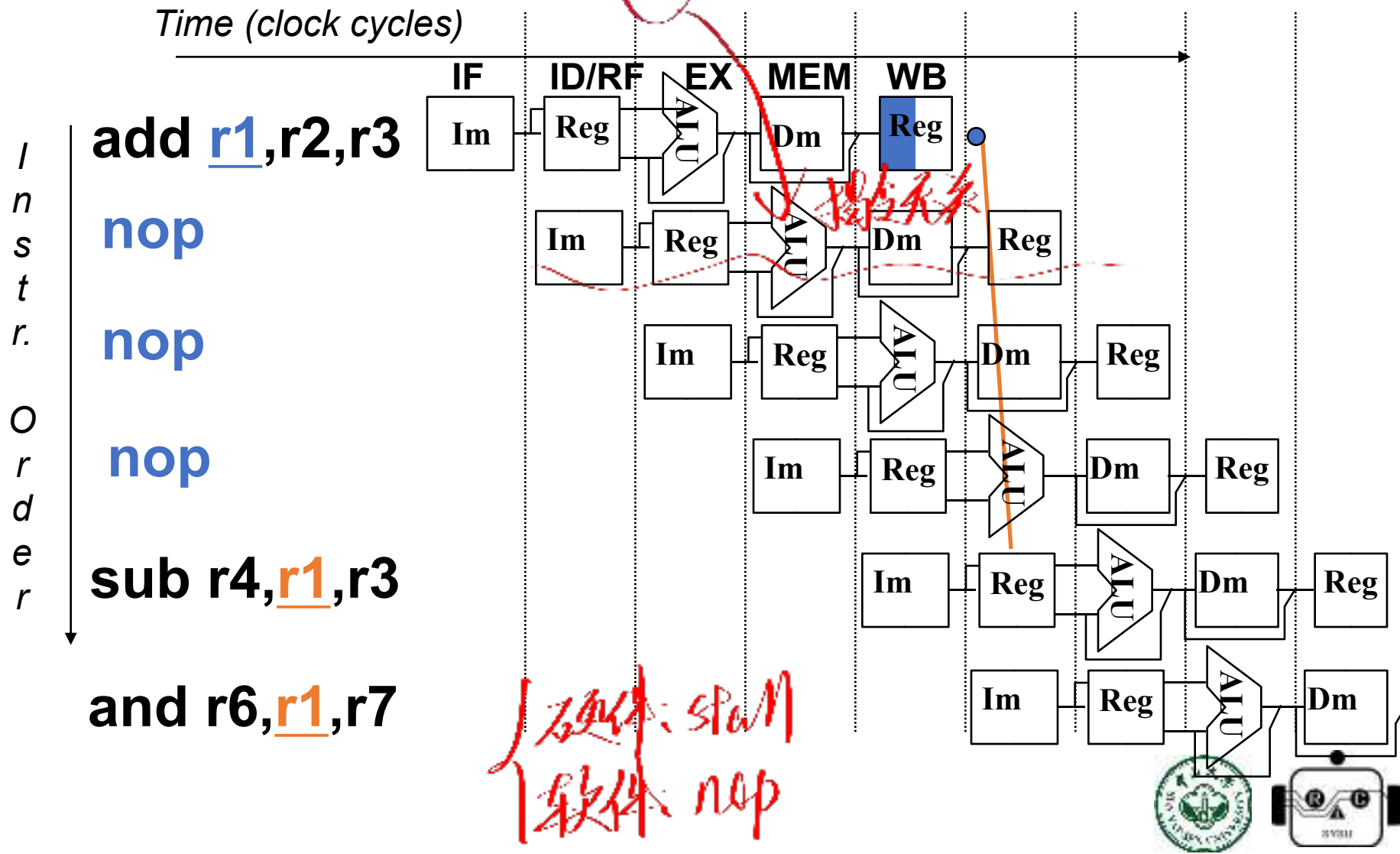
数据竞争可用等待的方式 – stall(流水线阻塞)解决 –同时影响到吞吐率



- 缺点：控制相当复杂，需要改数据通路！

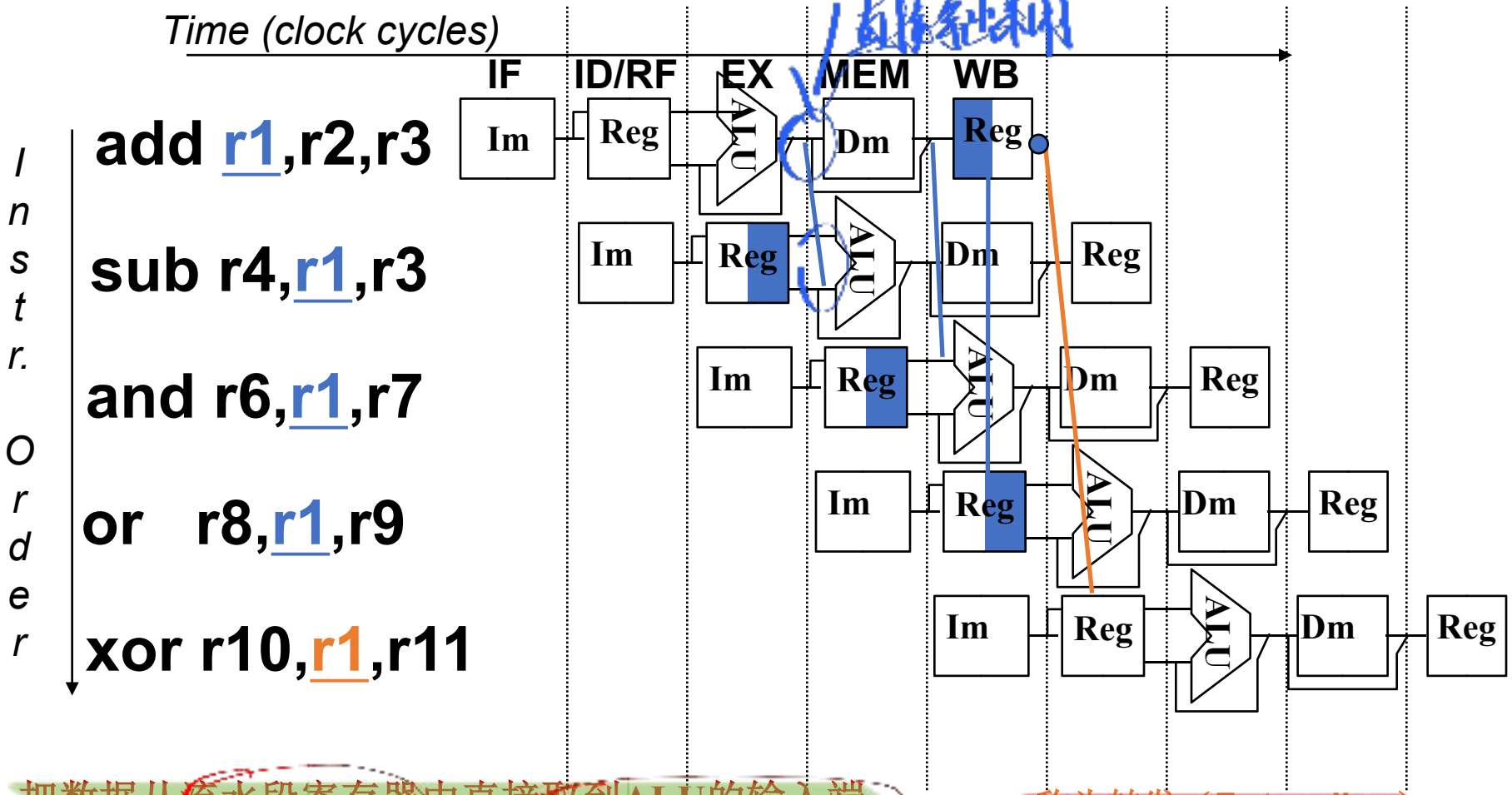
方案 2：软件上插入无关指令

- 最差的做法：由编译器插入三条NOP指令，浪费三条指令的空间和时间



方案3： 利用DataPath中的中间数据

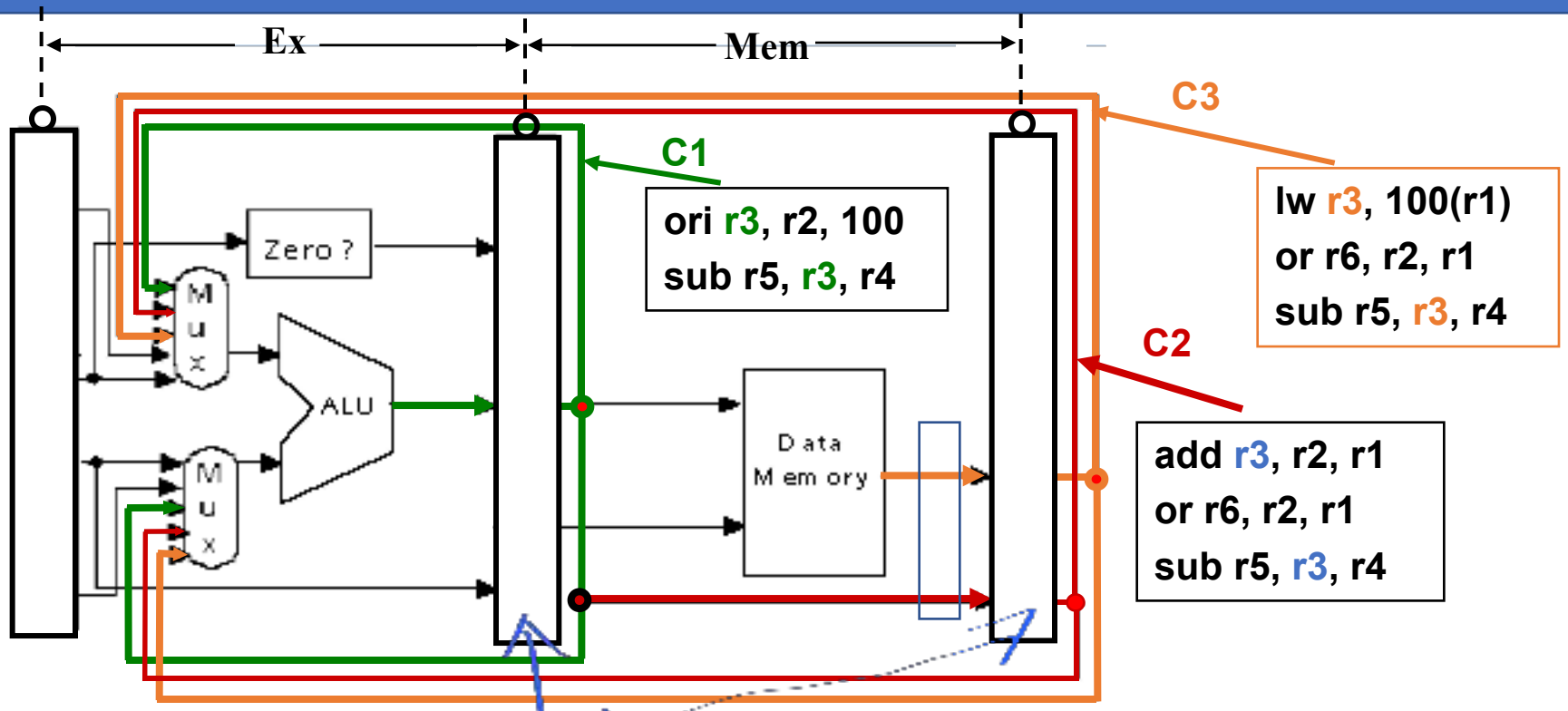
- 仔细观察后发现：流水段寄存器中已有需要的值r1！ 在哪个流水段R中？



1. 把数据从流水段寄存器中直接取到ALU的输入端
2. 寄存器写/读口分别在前/后半周期，使写入被直接读出

称为转发（Forwading）或旁路（Bypassing）

RAW（写后读）数据冒险的“转发”条件



后面指令需用ALU输出结果

C1: 目寄是后一条指令的源寄

C2: 目寄是后第二条指令的源寄

(例如：R-Type后跟R- / lw / sw / beq等)

后面指令需用从DM读出的结果

C3: 目寄是后第二指令的源寄

(例如：load指令后跟R-Type / beq等)

用流水段寄存器来表示转发条件（C3以后考虑）

C1(a): EX/MEM. RegisterRd=ID/EX. RegisterRs

C1(b): EX/MEM. RegisterRd=ID/EX. RegisterRt

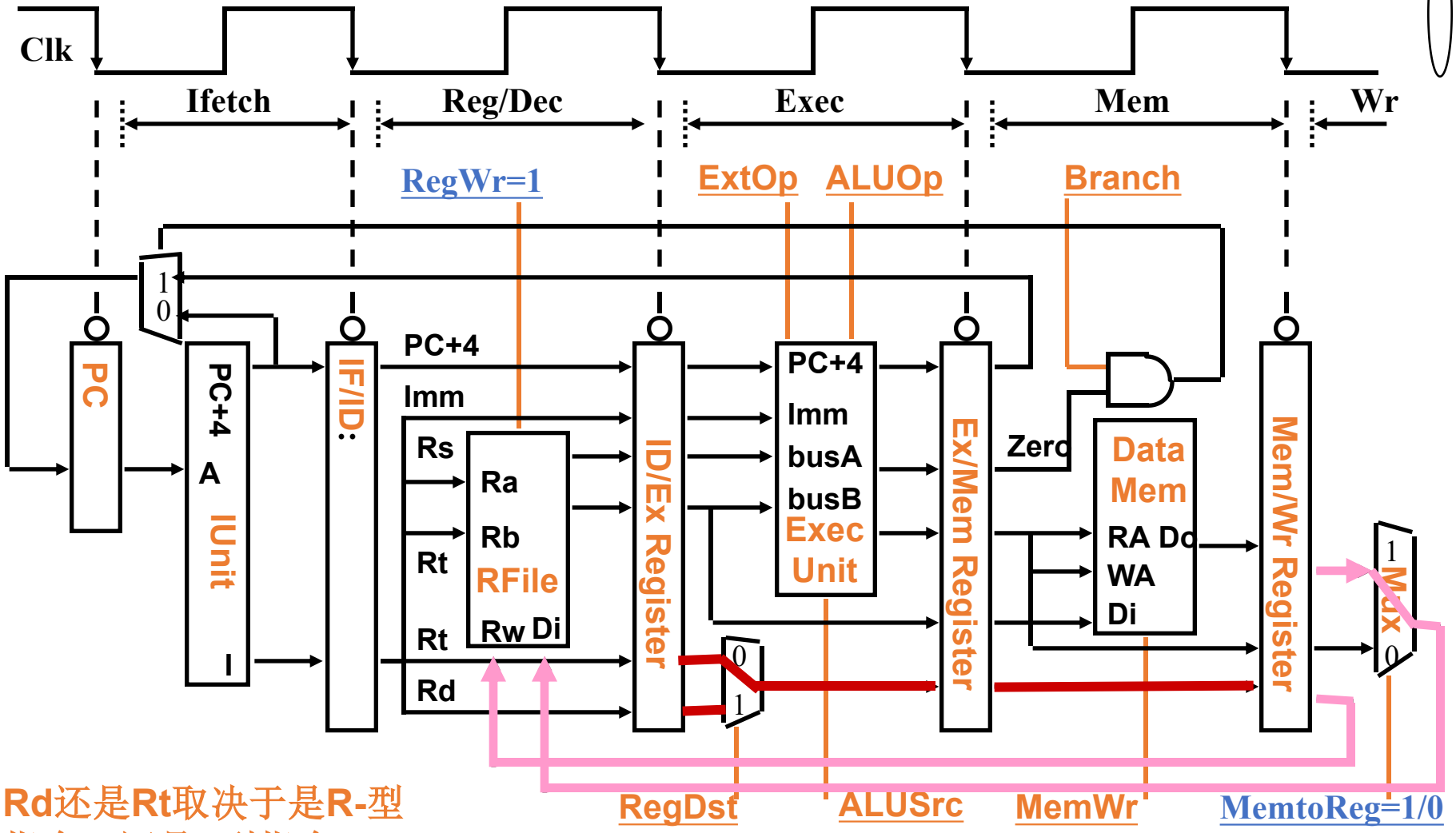
C2(a): MEM/WB. RegisterRd=ID/EX. RegisterRs

C2(b): MEM/WB. RegisterRd=ID/EX. RegisterRt

这里的RegisterRd是指目的寄存器

实际上是R-type的Rd 或 I-Type的rt

指令的回写（Write Back）阶段



Rd还是Rt取决于是R-型指令，还是I-型指令！

若是beq指令会怎样？

```
beq r3, r2, 100
sub r5, r3, r2
```

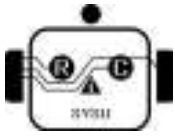
因为beq指令没有写结果，不能进行转发！

转发条件的进一步完善

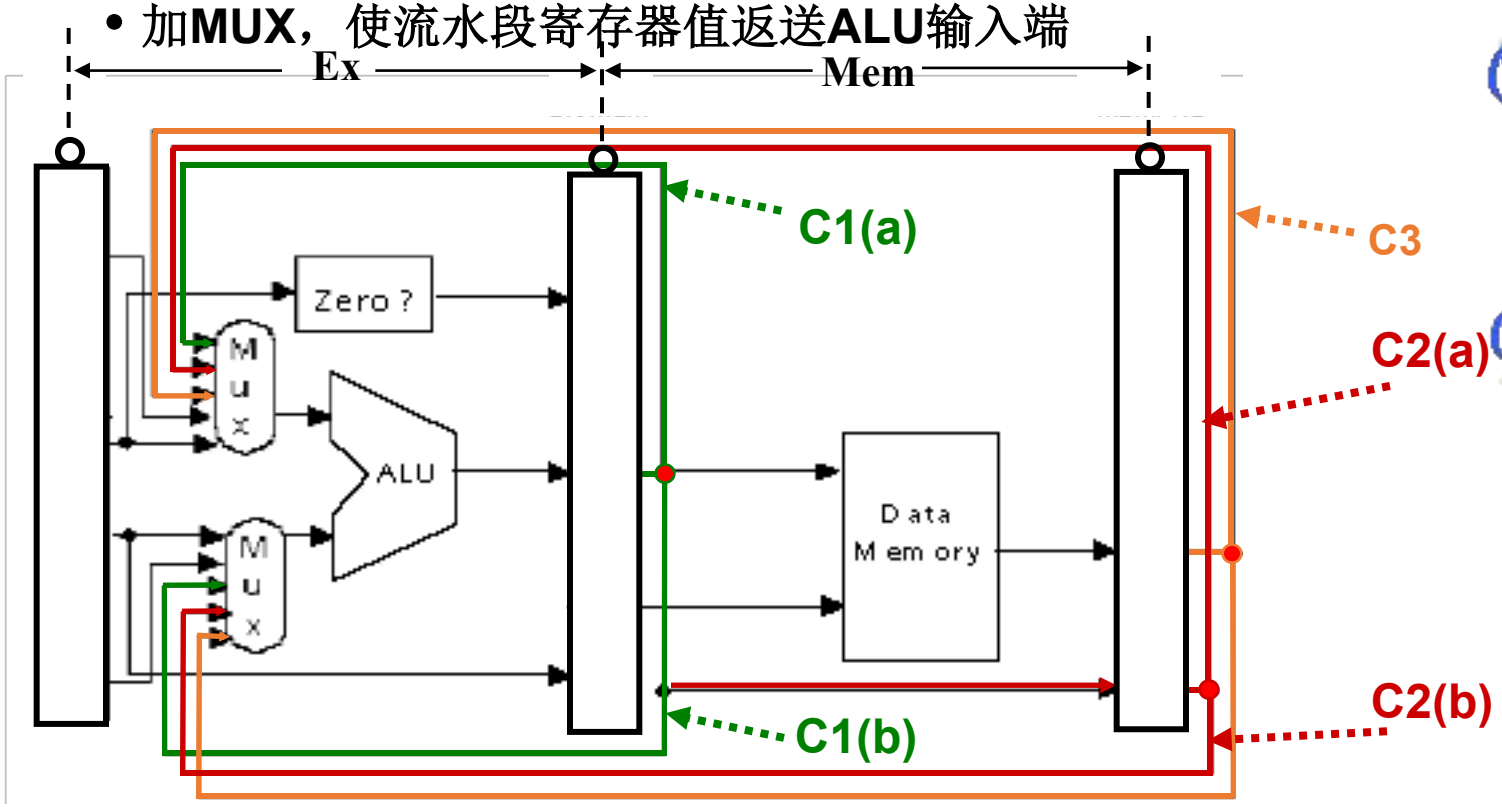
```
beq r3, r2, 100
sub r5, r3, r2
```

- 以下两种情况下，根据前面的转发条件转发会发生错误
  - 指令的结果不写入目的寄存器Rd时
    - 例如，Beq指令只对rs和rt相减，不写结果到目的寄存器
    - 即：EX / MEM 或 MEM / WB 流水段寄存器的RegWrite信号为0
  - Rd等于$0时
    - 例如，指令 sll $0, $1, 2 的转发结果为(R[$1]<<2) ，但实际上应该是0

- 因此，修改转发条件为：
  - C1(a): EX/MEM.RegWrite and EX/MEM. RegisterRd ≠ 0
    and EX/MEM. RegisterRd=ID/EX. RegisterRs
  - C1(b): EX/MEM.RegWrite and EX/MEM. RegisterRd ≠ 0
    and EX/MEM. RegisterRd=ID/EX. RegisterRt
  - C2(a): MEM/WB.RegWrite and MEM/WB. RegisterRd ≠ 0
    and MEM/WB. RegisterRd=ID/EX. RegisterRs
  - C2(b): MEM/WB.RegWrite and MEM/WB. RegisterRd ≠ 0
    and MEM/WB. RegisterRd=ID/EX. RegisterRt

转发路径和转发条件



C1反映本条指令和随后指令间的相关关系

C2反映本条指令和随后第二条指令间的相关关系

C1(a)和C1(b)可以合并为一个条件C1，并把转发线合一起后同时送A口和B口

即：C1=C1(a) or C1(b)，同样：C2=C2(a) or C2(b)，转发线合起来

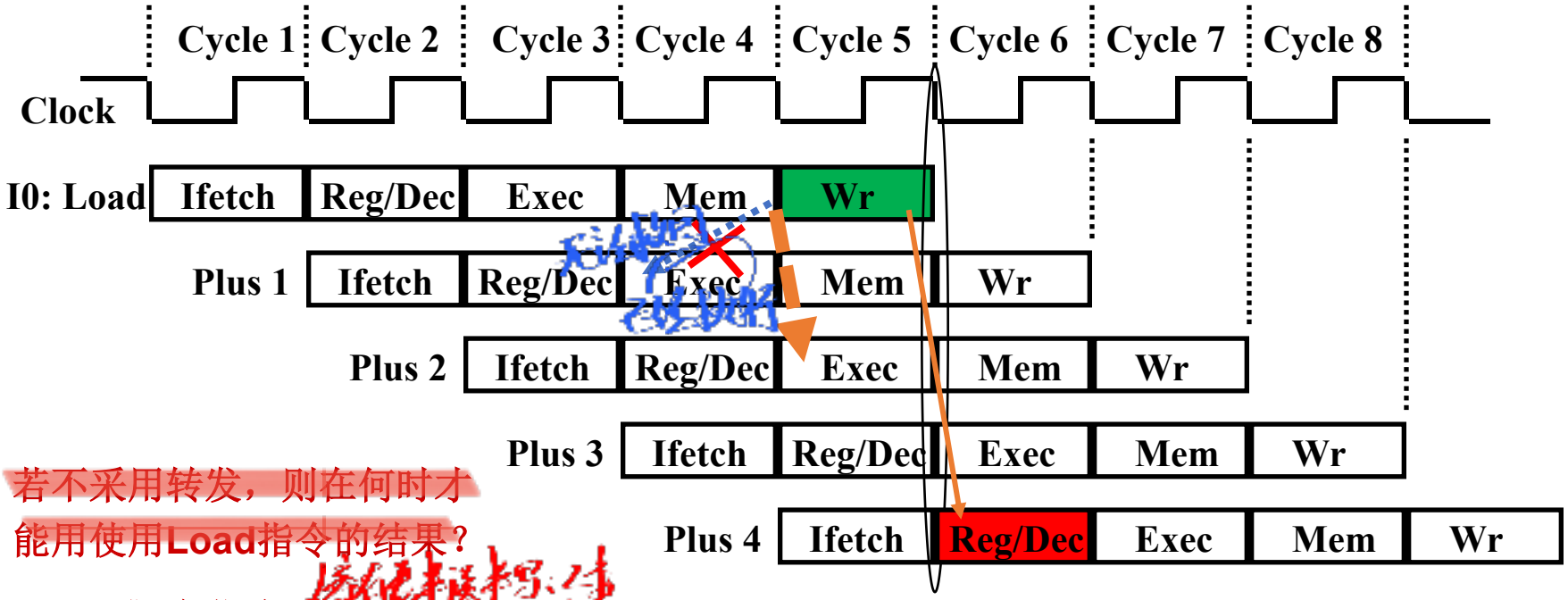
实际上红线和兰线可以合并，而且在原数据通路中是合并在一起的。记得吗？

由一个二路选择器（控制端为MemtoReg）合并输出到寄存器堆！

所以不需另外有一个检测条件C3!

C1和C2分别反映哪两条指令的关系呢？

复习：Load指令引起的延迟现象



若不采用转发，则在何时才能用使用Load指令的结果？

延迟2或3条指令！

- Load指令最早在哪个流水线寄存器中开始有后续指令需要的值？

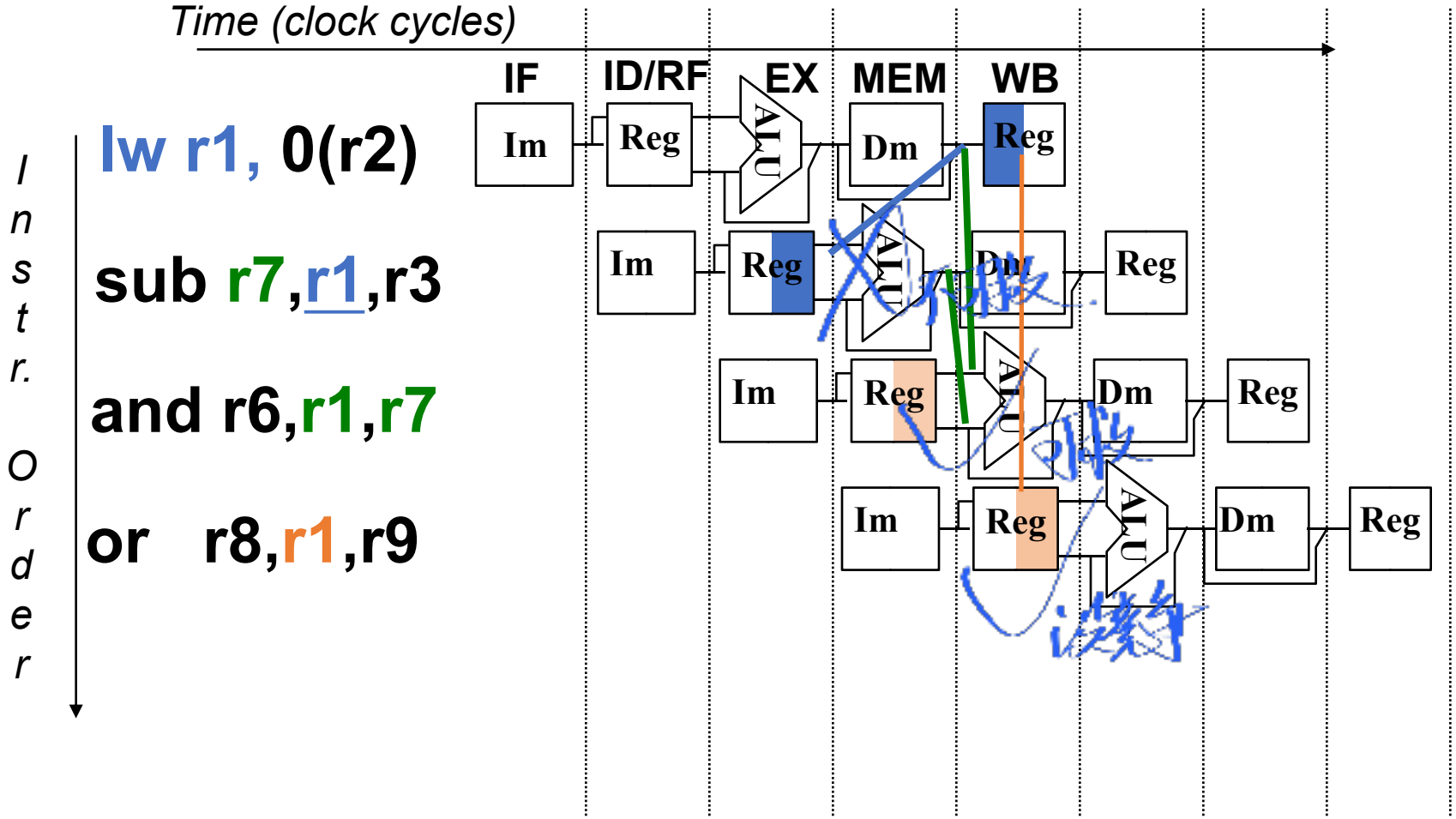
  实际上，在第四周期结束时，数据在流水段寄存器中已经有值。

  采用数据转发技术可以使load指令后面第二条指令得到所需的值

  但不能解决load指令和随后的第一条指令间的数据冒险，要延迟执行一条指令！

这种load指令和随后指令间的数据冒险，称为“装入- 使用数据冒险(load- use Data Hazard)”

“Forwarding”技术使Load-use冒险只需延迟一个周期



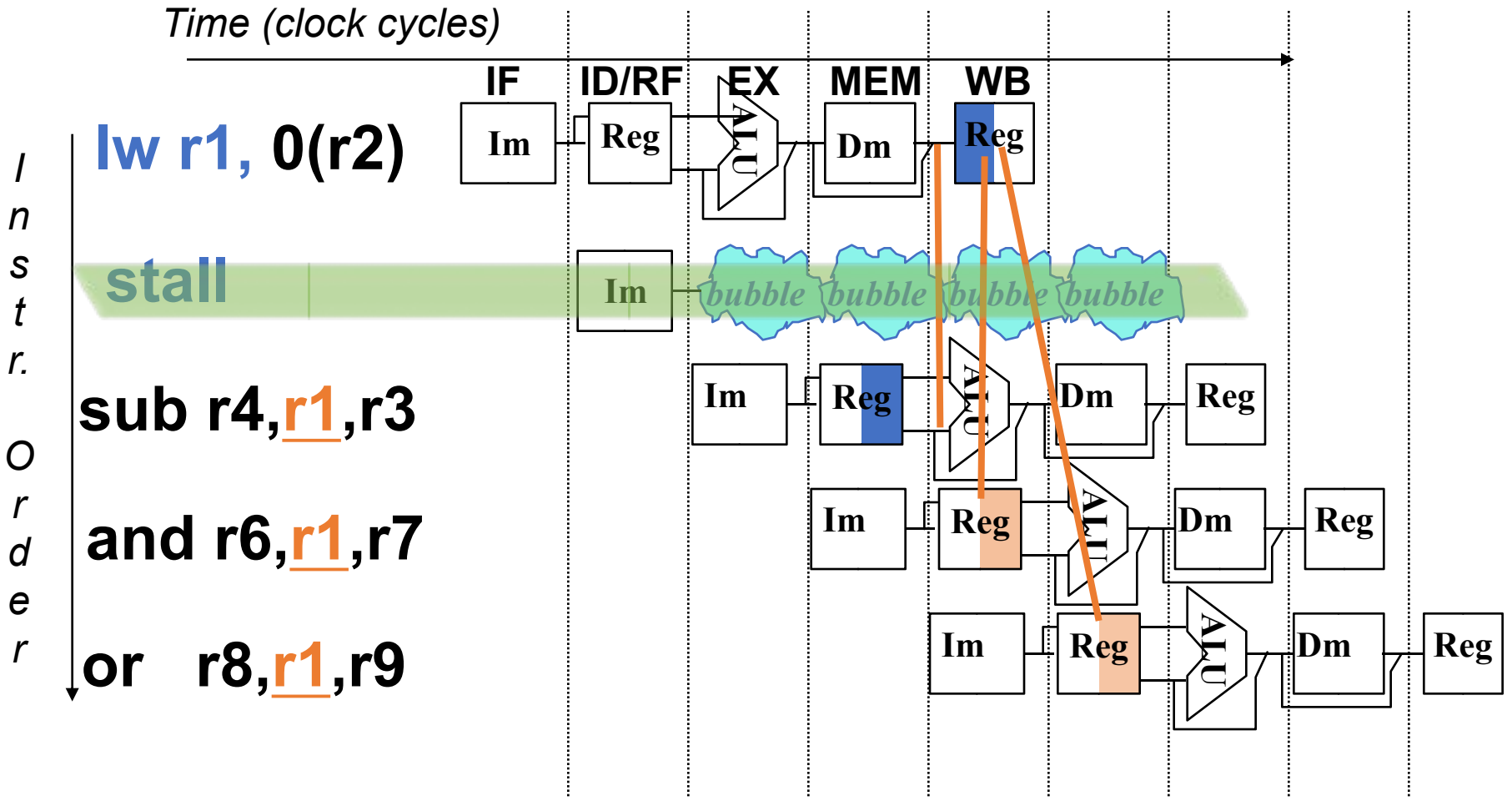
采用“转发”后仅第二条指令 **SUB r7,r1,r3** 不能按时执行！需要阻塞一个周期。

（其它后续指令顺延）

发生“装入- 使用数据冒险”时，需要对**load**后的指令阻塞一个时钟周期！

方案1：硬件阻止指令执行来解决load-use

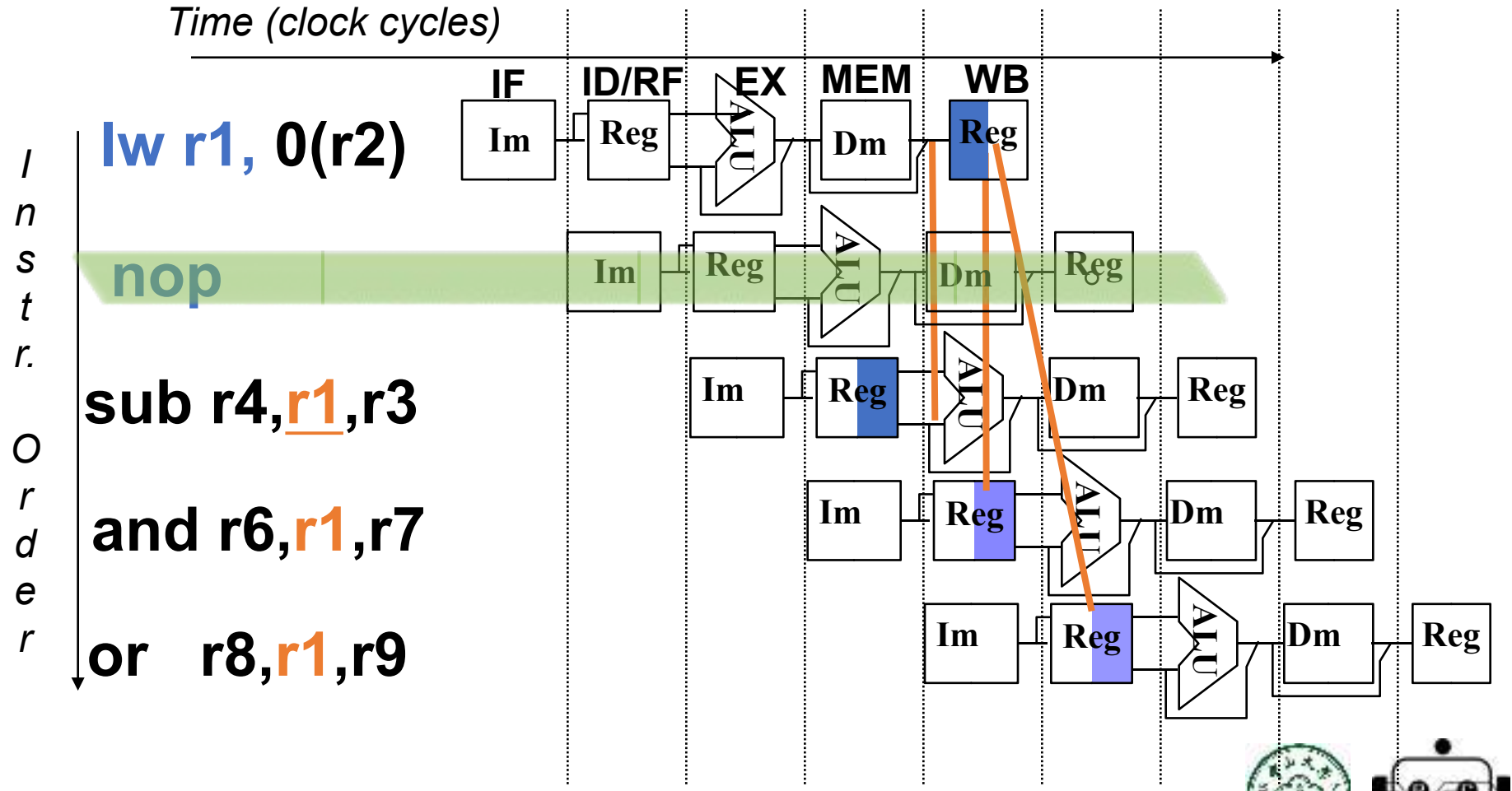
用硬件阻塞一个周期



方案2：软件上插入NOP指令来解决load-use

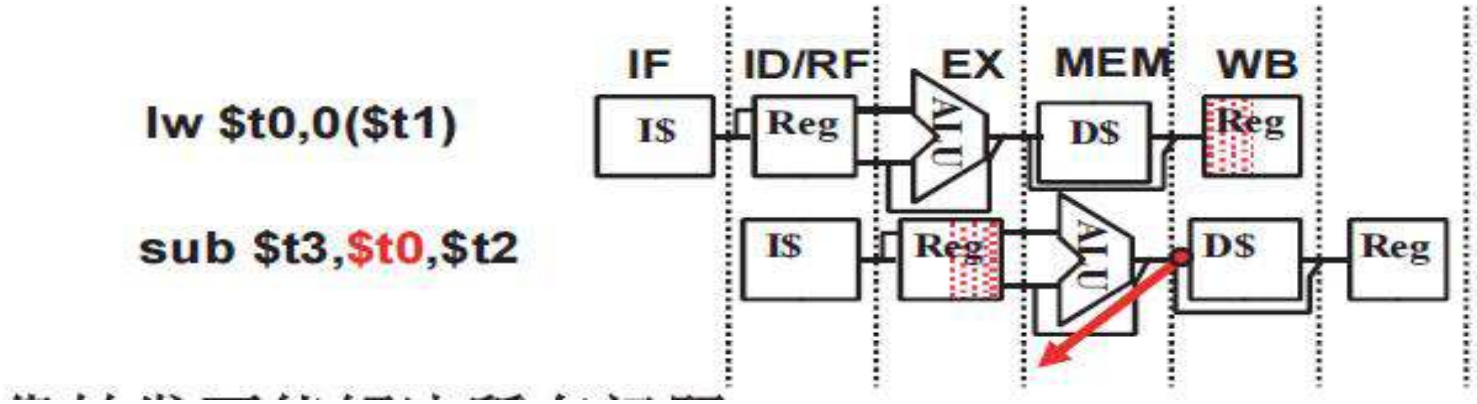
- 用软件插入一条NOP指令！（有些处理器不支持硬件阻塞处理）

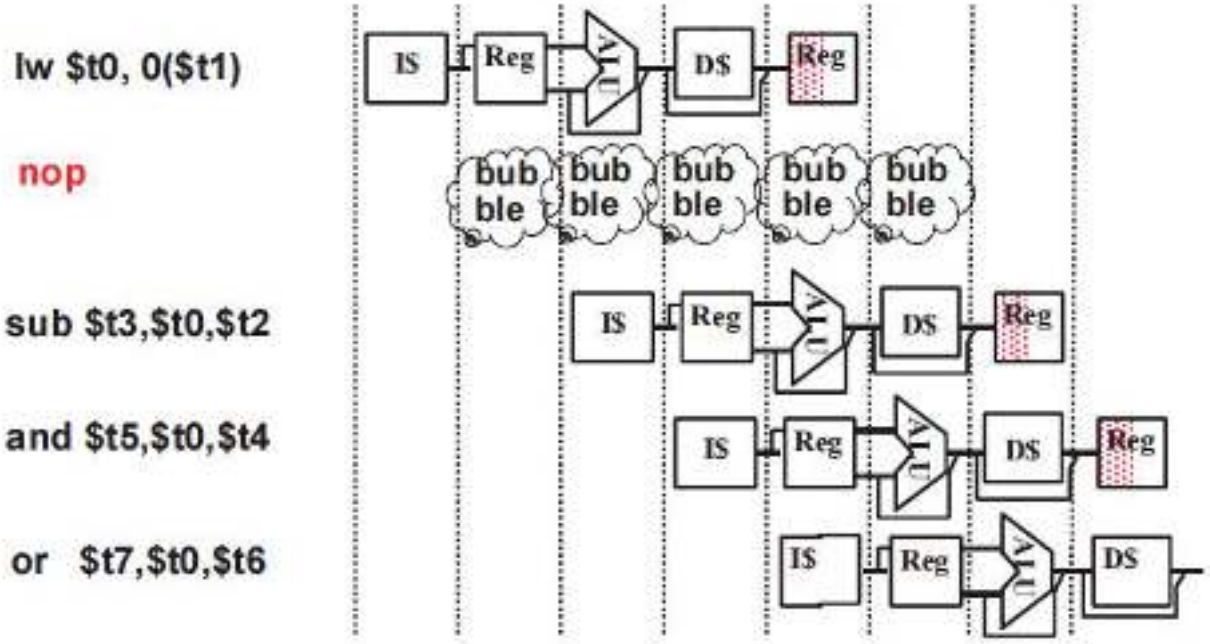
例如：MIPS 1 处理器没有硬件阻塞处理，而由编译器（或汇编程序员） 来处理。



数据冒险： loads的转发

- 仍需延迟一个周期

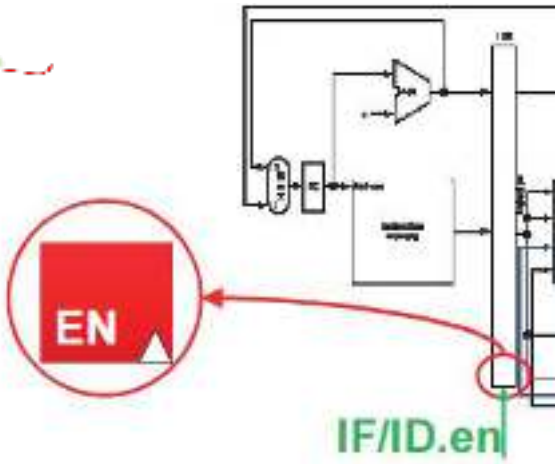




硬件上暂停流水线，等价于插入nop指令

如何插入NOP指令？

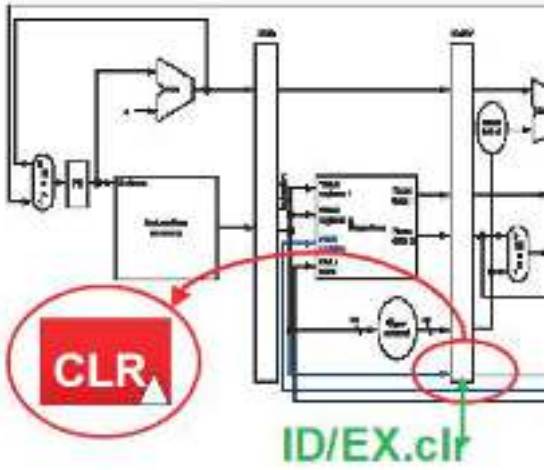
- 检测条件：IF/ID 的前序是lw指令，并且lw的rt寄存器与IF/ID的rs或rt相同
- 执行动作：
  - 1)冻结IF/ID:sub继续被保存
  - 2)清除ID/EX:指令全为0，等价于插入NOP
  - 3)禁止PC 防止PC继续计数，PC保持PC+4
- 数据通路:将IF/ID修改为使能型寄存器
- 控制系统：增加IF/ID.en控制信号
  - 当IF/ID.en为0时， IF/ID在下一个clock上升沿
  - 到来时保持不变



如何插入NOP指令？

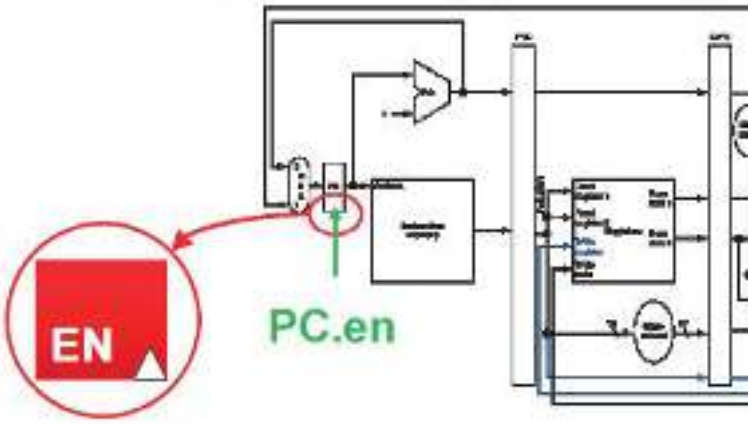
- 检测条件：IF/ID 的前序是lw指令，并且lw的rt寄存器与IF/ID的rs或rt相同
- 执行动作：
  - 1)冻结IF/ID:sub继续被保存
  - 2)清除ID/EX:指令全为0，等价于插入NOP
  - 3)禁止PC 防止PC继续计数，PC保持PC+4

- 数据通路:将ID/EX修改为复位型寄存器
- 控制系统：增加ID/EX.ctr控制信号
  - 当ID/EX.ctr为0时， ID/EX在下一个clock上升沿
  - 到来时被清除为0.



如何插入NOP指令？

- 检测条件： IF/ID 的前序是lw指令，并且lw的rt寄存器与IF/ID的rs或rt相同
- 执行动作：
  - 1)冻结IF/ID:sub继续被保存
  - 2)清除ID/EX:指令全为0，等价于插入NOP
  - 3)禁止PC 防止PC继续计数，PC保持PC+4



- 数据通路:将PC修改为使能型寄存器
- 控制系统： 增加PC. en控制信号
  - 当PC. en为0时， ID/EX在下一个clock上升沿
  - 到来时保持不变.

方案3：编译器进行指令顺序调整来解决load-use

以下源程序可生成两种不同的代码，优化的代码可避免Load阻塞

```
a = b + c;
d = e – f;
```

假定 a, b, c, d ,e, f 在内存

Slow code:

```
lw   $2, b
lw   $3, c
add  $1, $2, $3
sw   a, $1
lw   $5, e
lw   $6, f
sub  $4, $5, $6
sw   d, $4
```

调整后

Fast code:

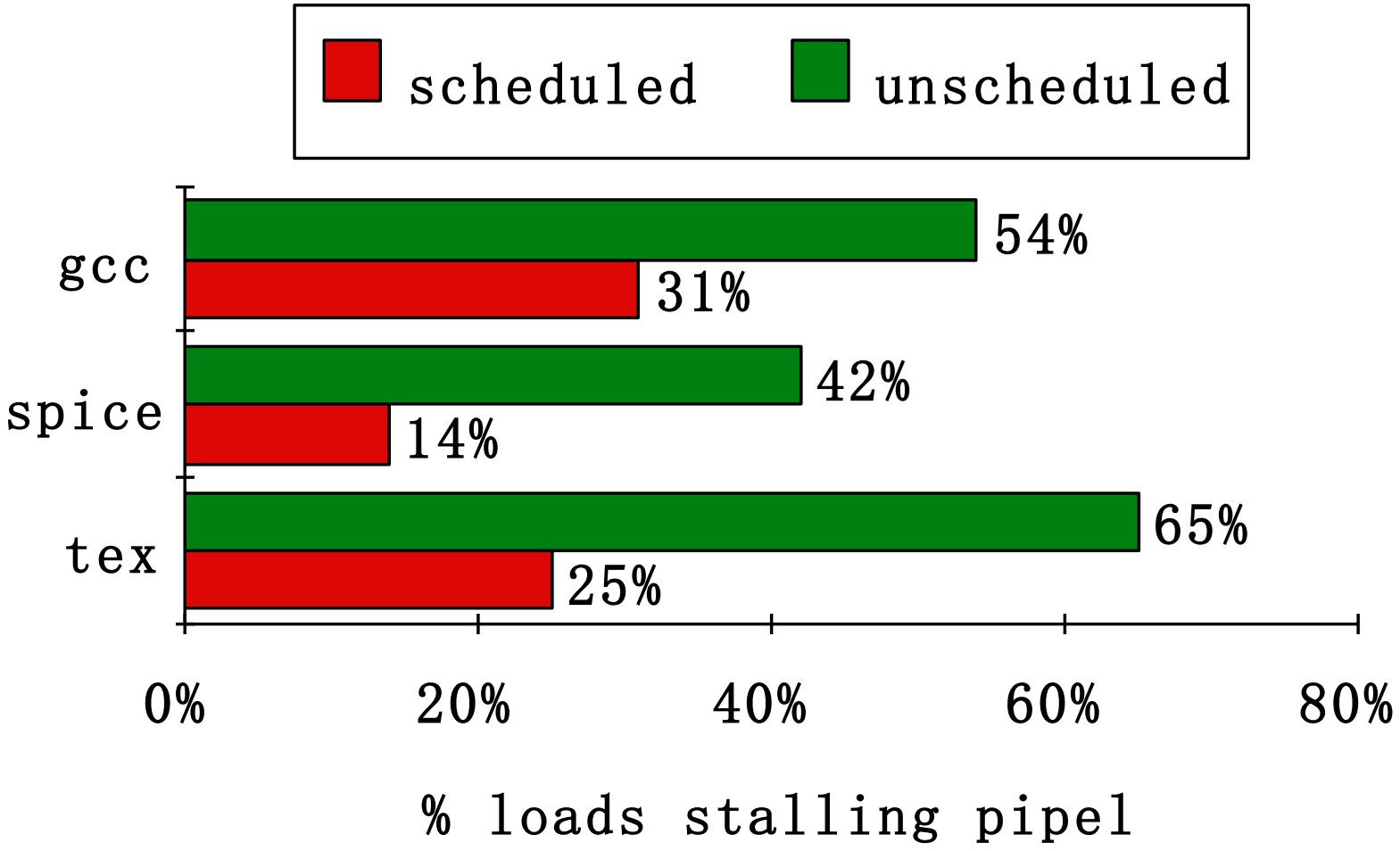
```
lw   $2, b
lw   $3, c
lw   $5, e
add  $1, $2, $3
lw   $6, f
sw   a, $1
sub  $4, $5, $6
sw   d, $4
```

真正执行时需阻塞2次

编译器的优化很重要！

真正执行时无需阻塞

编译器优化以避免阻塞的情况调查:



由此可见，优化调度后load阻塞现象大约降低了1/2~1/3

数据冒险的解决方法

- 方法1：硬件阻塞（stall）
- 方法2：软件插入“NOP”指令
- 方法3：编译优化：调整指令顺序，能解决所有数据冒险吗？
- 方法4：合理实现寄存器堆的读/写操作，能解决所有数据冒险吗？
  - 前半时钟周期写，后半时钟周期读，若同一个时钟内前面指令写入的数据正好是后面指令所读数据，则不会发生数据冒险
- 方法5：转发（Forwarding或Bypassing 旁路）技术，能解决所有数据冒险吗？
  - 若相关数据是ALU结果，则如何？

    可通过转发解决
  - 若相关数据是上条指令DM读出内容，则如何？

    不能通过转发解决，随后指令需被阻塞一个时钟 或 加NOP指令

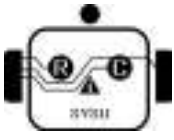
称为Load-use数据冒险！

实现“转发”和“阻塞”要修改数据通路：

（1）检测何时需要“转发”，并控制实现“转发”

（2）检测何时需要“阻塞”，并控制实现“阻塞”

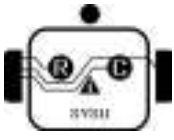




今天所有处理器都采用流水线技术

- 流水线提高吞吐率但是恶化延迟
- 流水线级数反映了潜在的吞吐率提升倍数
- Pipeline rate limited by slowest pipeline stage
  - Unbalanced lengths of pipe stages reduces speedup
  - Time to “fill” pipeline and time to “drain” it reduces speedup
- 需要能够检测和处理竞争问题
  - 结构竞争—硬件资源使用冲突
  - 数据竞争—数据存储器或者寄存器内容因指令关联而不可获得ReadAfterWrite，load-use
  - 控制竞争—因跳转分支指令使得PC无法及时确定

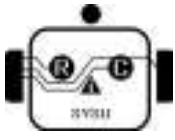




流水线CPU的实现

- 需要能够检测和处理竞争问题
  - **结构竞争**—硬件资源使用冲突
  - **数据竞争**—数据存储器或者寄存器内容因指令关联而不可获得RAW（写后读），load-use
  - **控制竞争**—因跳转分支指令使得PC无法及时确定
- 流水线的控制信号
  - 控制信号如何随流水线运行？



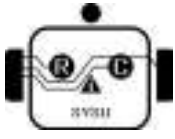


Designing a Pipelined Processor

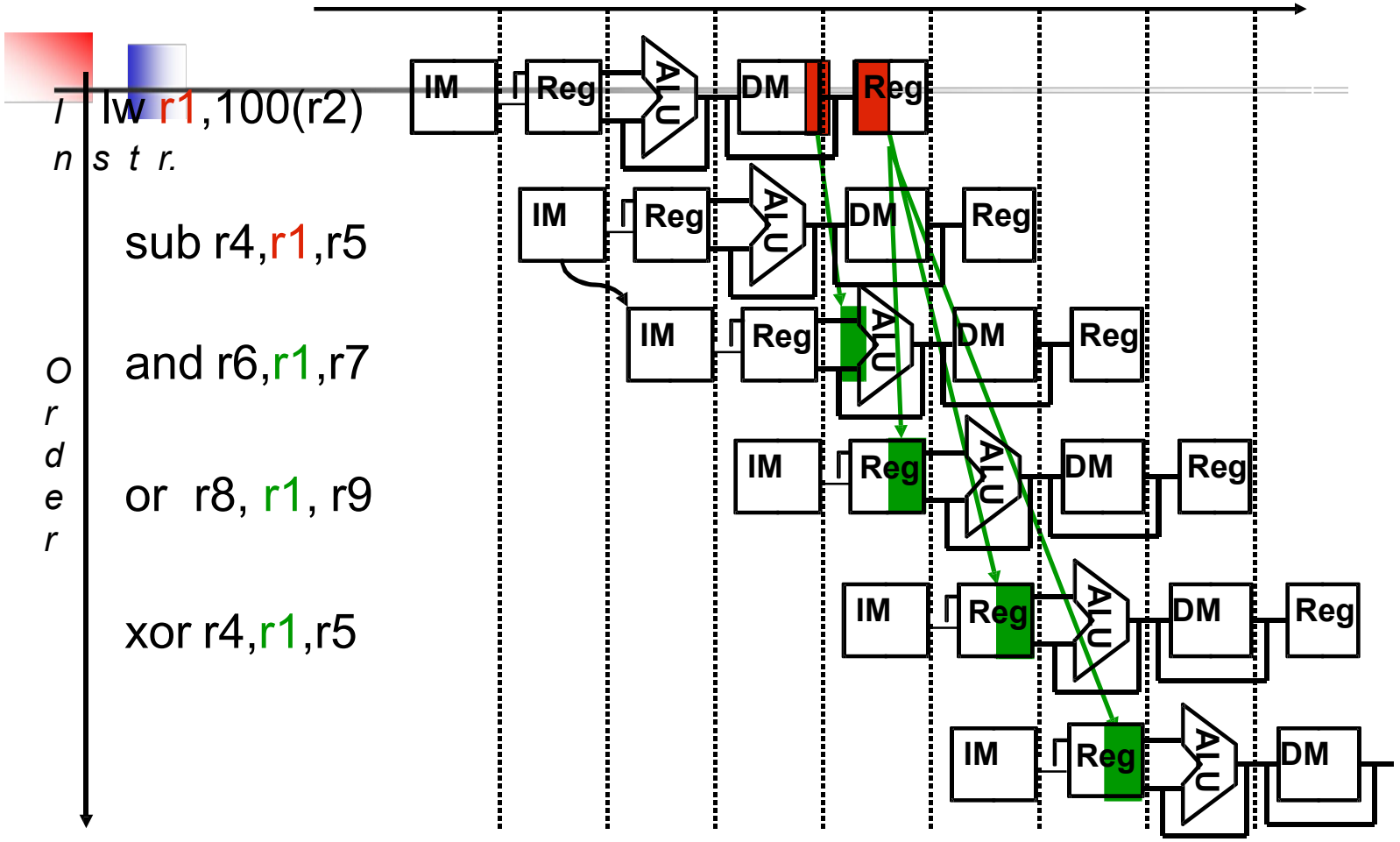
关键: datapath和控制信号的设计

- 将各个资源和状态(流水级)联系起来
  - 从设计上要避免structural hazards的出现: 保证资源是够用的
- 在各个级别之间增加寄存器
- 找到所有的数据竞争和控制竞争问题
  - 如果是寄存器关联引起的=> data hazard: forward or stall to resolve them
  - 如果是和PC有关的竞争=> control hazard: we’ll see next time
- 在合适的阶段使控制信号有效
- 合适的测试设计
  - If you don’t test it, it won’t work

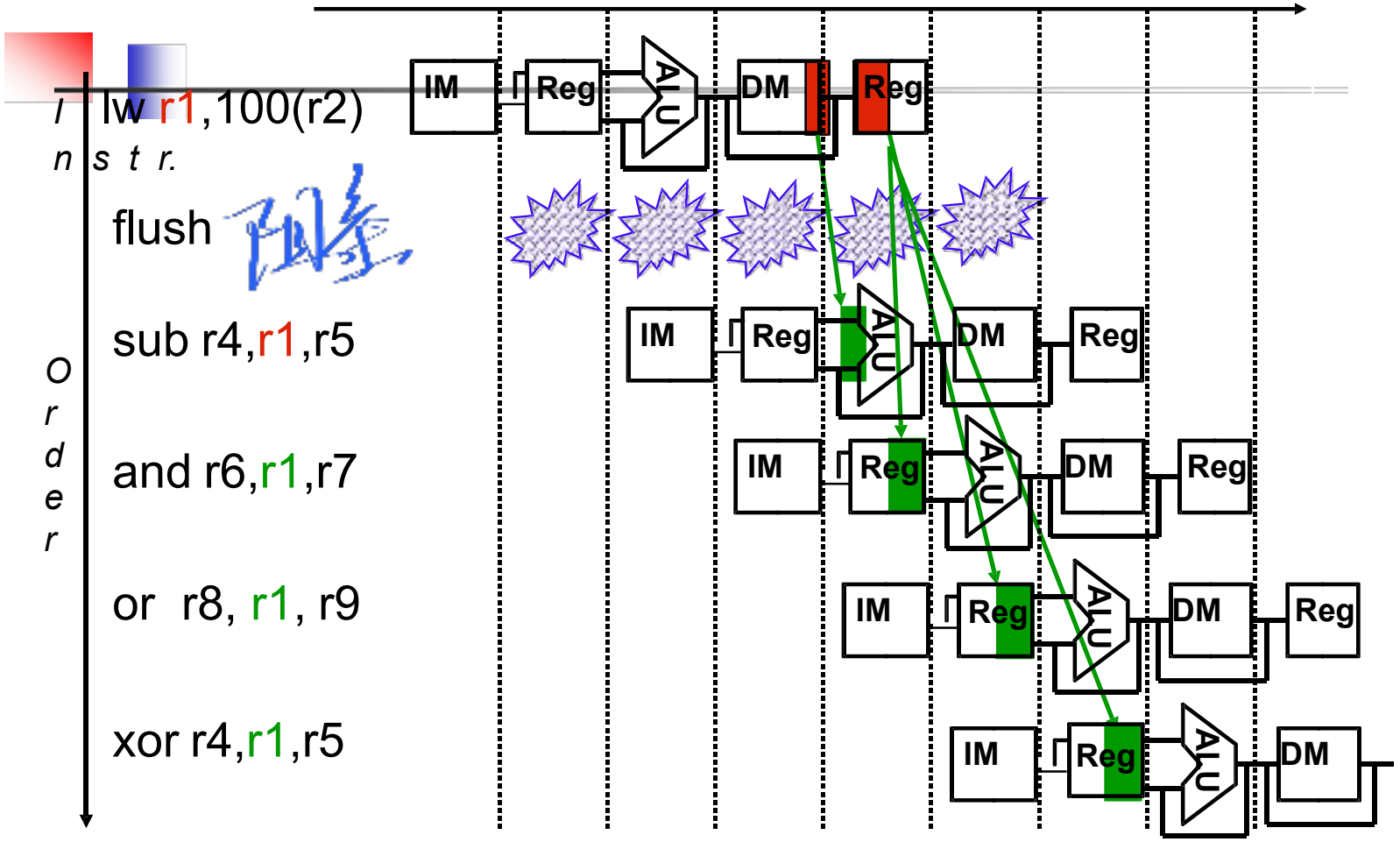




Load-use Data Hazards的datapath forwarding



Load-use Data Hazards的datapath forwarding



Load-use Hazard 检测

- 因为load-use hazard无法通过forward完全解决，即便加了forward电路仍然存在hazard, 必须加一个阻塞周期
- **ID Hazard Detection**

if (ID/EX.MemRead

and ((ID/EX.RegisterRt = IF/ID.RegisterRs) or (ID/EX.RegisterRt = IF/ID.RegisterRt))) stall the pipeline

- 首先检查是否是load指令; 然后检查EX stage load指令的destination是否与ID阶段的两个source registers 是一样的
- 经过这个检测导致的阻塞, forwarding 逻辑就可以处理剩下的竞争问题了

Load-use Hazard 检测

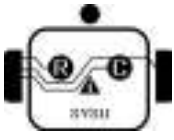
ID Hazard Detection

if (ID/EX.MemRead

and ((ID/EX.RegisterRt = IF/ID.RegisterRs) or (ID/EX.RegisterRt = IF/ID.RegisterRt))) stall the pipeline

- 首先检查是否是load指令; 然后检查EX stage load指令的destination是否与ID阶段的两个source registers 是一样的
- 经过这个检测导致的阻塞, Data forwarding 逻辑就可以处理剩下的竞争问题了

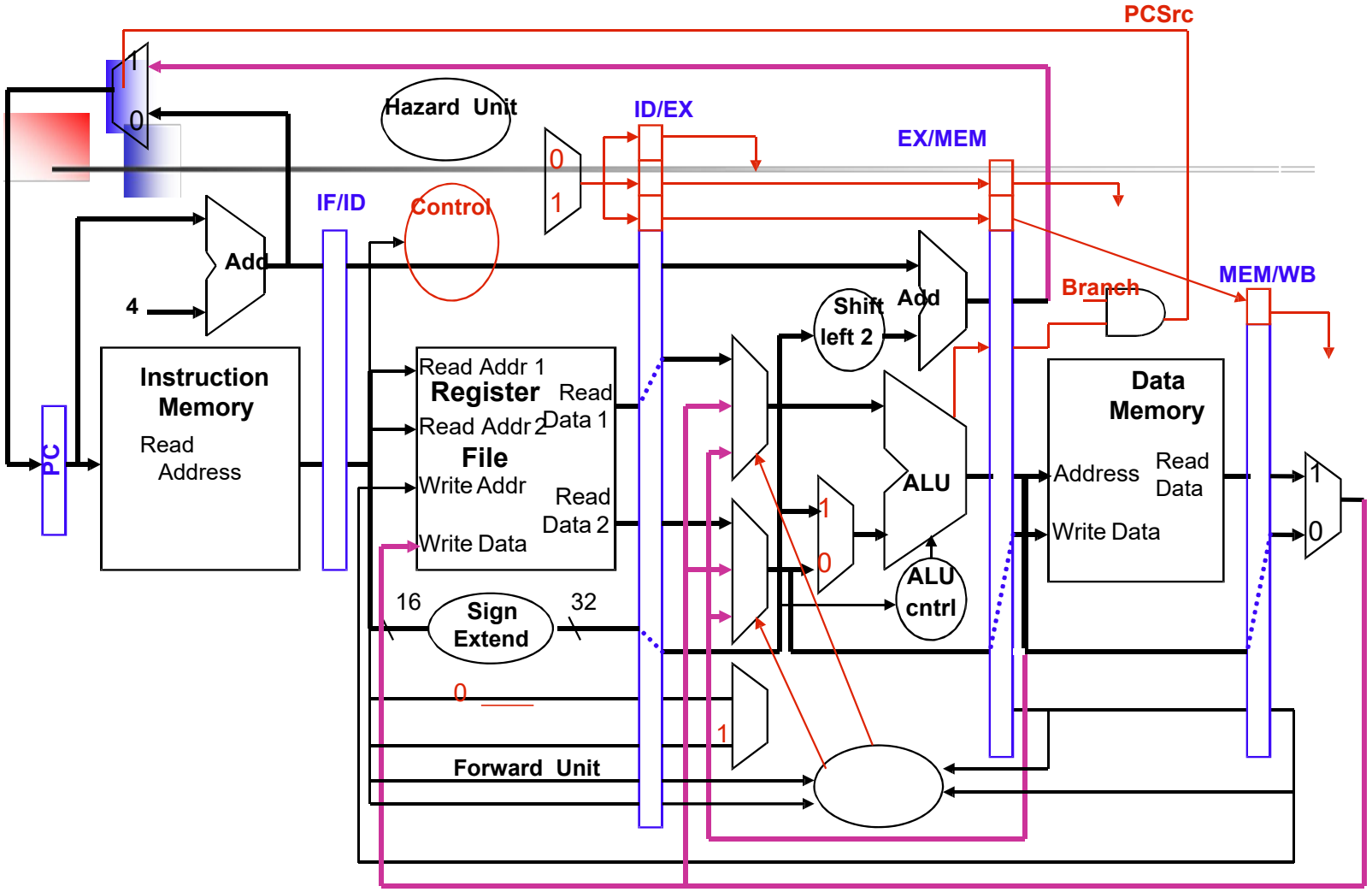




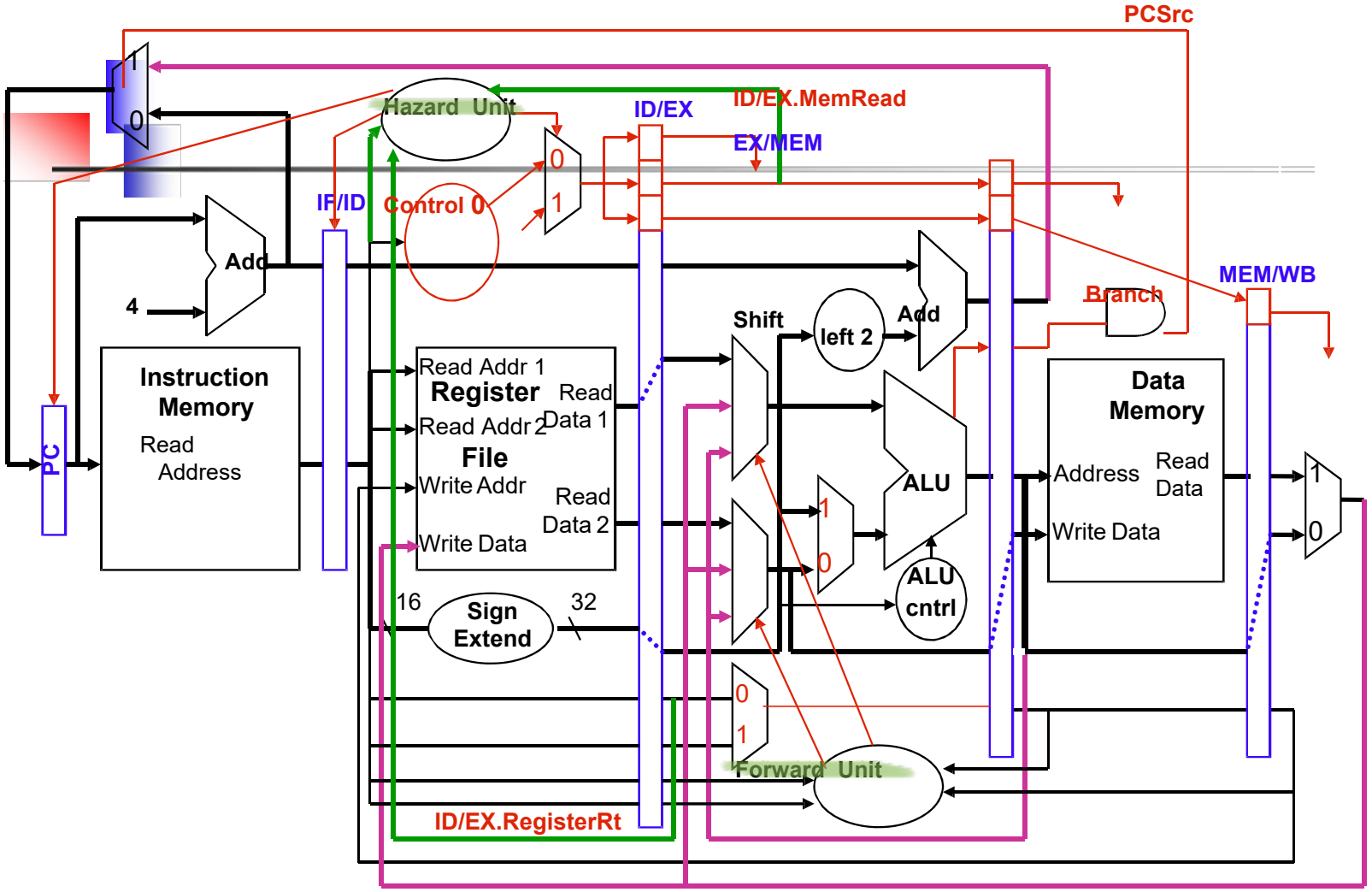
如何实现Stall 阻塞?

- 必须实现阻塞功能才能使forward发挥作用
- 阻止IF and ID 阶段的指令进入EX阶段执行，只要不让PC寄存器变化和IF/ID流水线状态寄存器保持不变就可以了
  - 只要让前面的hazard检测电路来控制这些寄存器的写入就可以了
- 从EX开始的后半段在阻塞时应处于清空状态:执行NOP
  - 将 ID/EX pipeline register中与EX,MEM,WB有关的控制信号清零,不工作状态
  - 由Hazard检测电路控制选择使用正常的控制信号还是直接为0.
  - 假定控制信号为0总是使电路不工作或者不影响电路状态的

Adding the Hazard Hardware



Adding the Hazard Hardware



找到所有的数据竞争和控制竞争问题

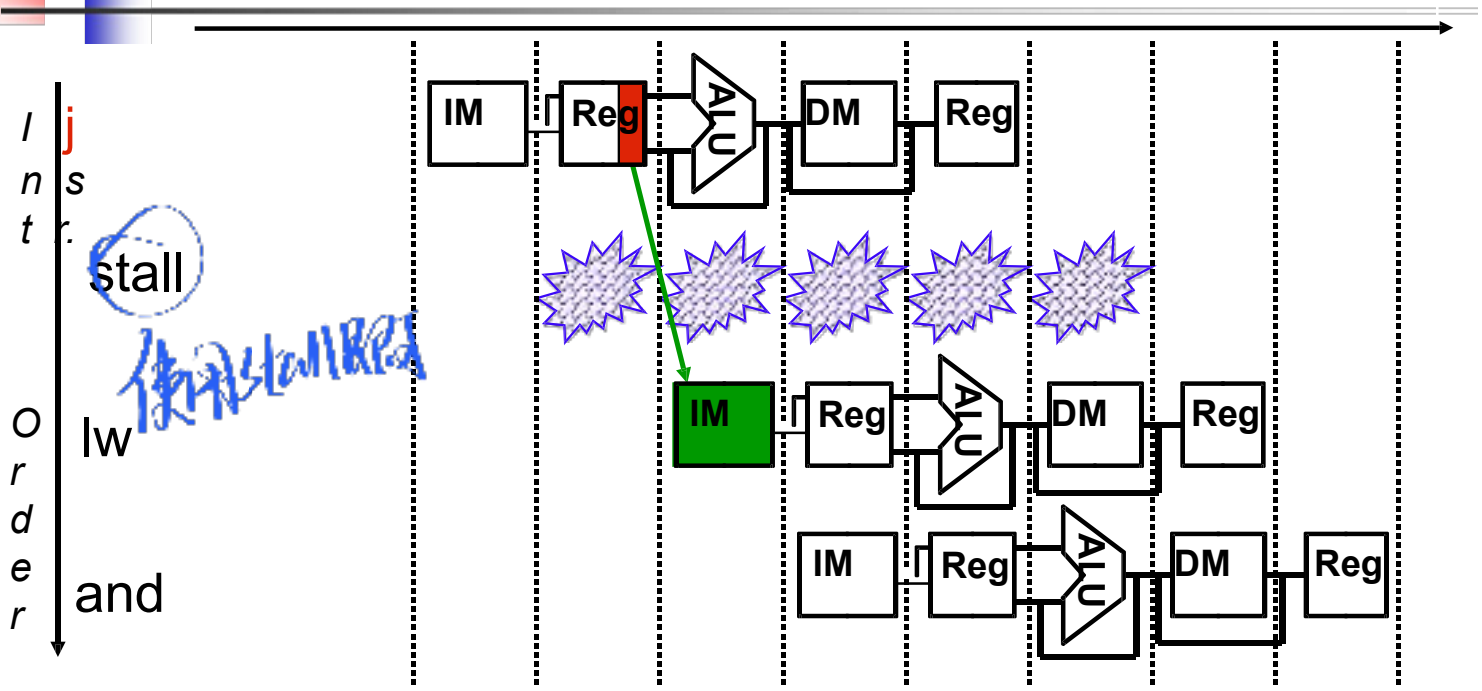
- 如果是寄存器关联引起的=> **data hazard**: forward or stall to resolve them
- 如果是和**PC**有关的竞争=> **control hazard**: we’ll see next

Control Hazards控制竞争

- 当指令流向不是预期的方式
  - 有条件的分支 (`beq`, `bne`)需阻塞3个周期
  - 无条件的跳转 (`j`)
- 可能的解决方案
  - Stall
  - 将判断提前
  - 预测
  - 延迟决定(即延迟分支)(requires compiler support)
- 控制竞争不像数据竞争那样频繁的发生，但是也没有像**forwarding**这样解决它的那么有效的方式

Jumps 引起一个阻塞

- Jumps 在ID阶段可以被判断出来这时只需要阻塞一个周期



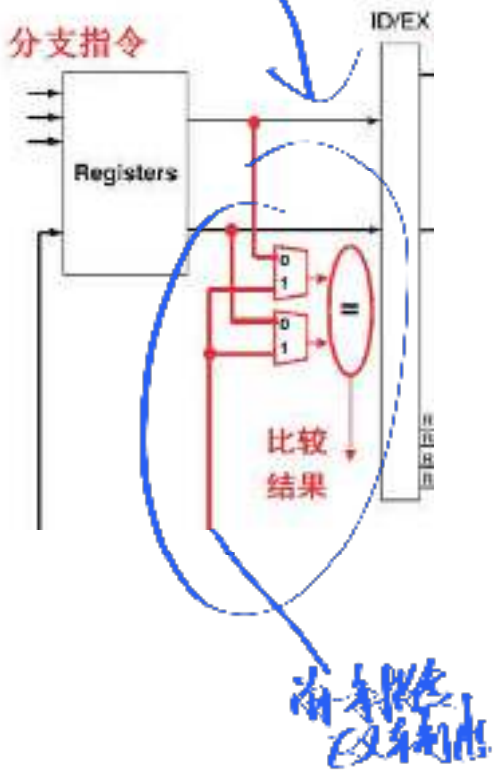
- 实际情况下,只有2% 指令是jump(SPEC int instruction mix)

在流水线中提前进行分支决定

- 将分支判断放在EX阶段中
  - 可以将STALL减少为2个
  - Adds an and gate and a 2x1 mux to the EX timing path
- 增加硬件以在ID阶段就进行比较运算和有效地址的运算，
  - 这样可以使STALL减少为1
  - 与RF的读操作同时
  - 需要一个比较器，多路选择器等电路
  - 需要在ID阶段引入forwarding hardware 电路

将分支判断放在ID阶段中

- 在ID阶段放置比较器，尽快得到指令结果
- 分支指令结果可以提前2个clock得到
  - 分支指令后继可能被废弃的指令减少为1条
  - 当需要转移时，清除IF/ID即可

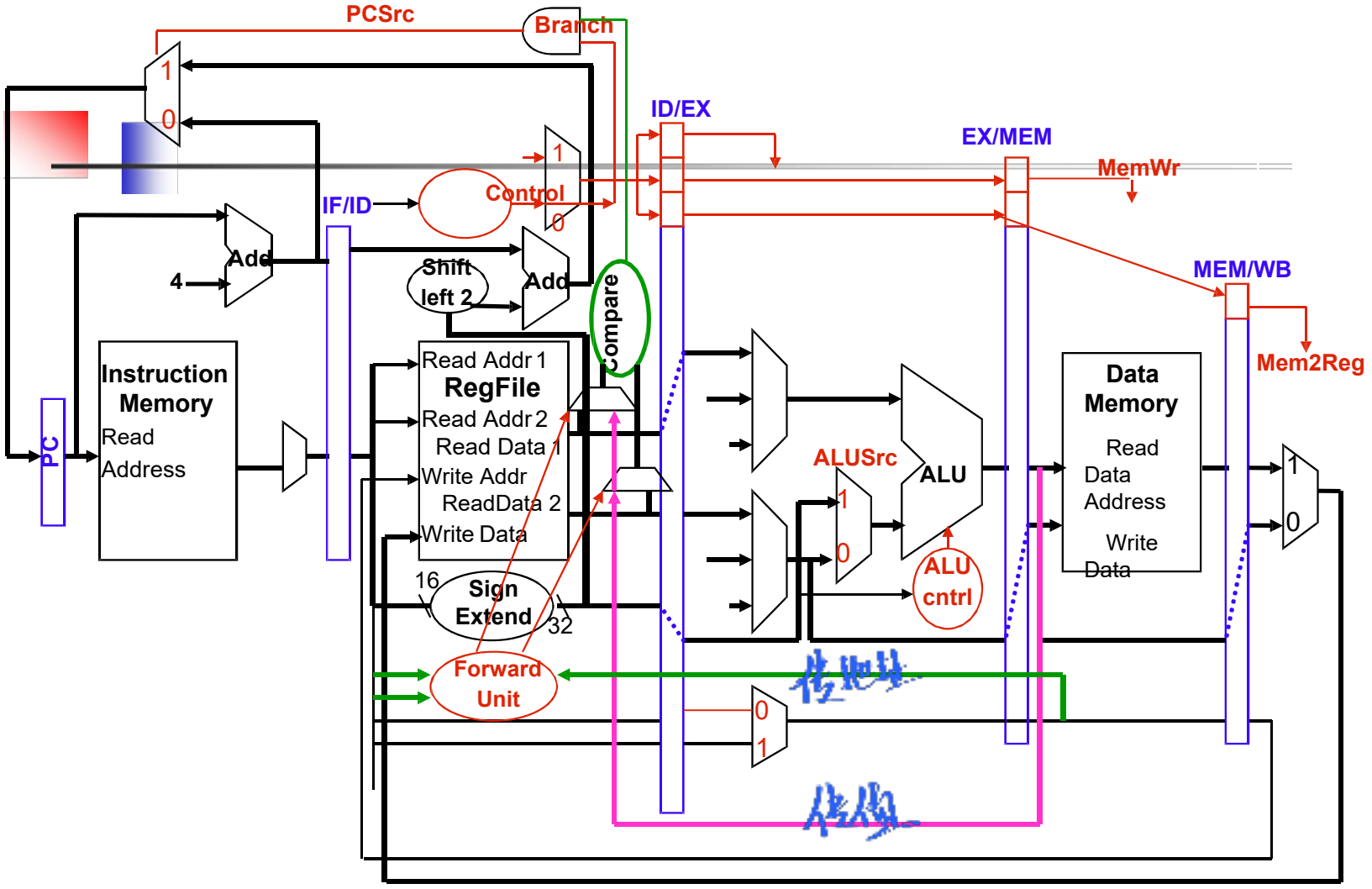


- 需要一个比较器，多路选择器等电路
- 需要在ID阶段引入forwarding hardware 电路

问题：如果比较的数据取决于前面指令的运算结果怎么办？数据相关

如果可以利用ALU运算结果可以转发

ID Stage 分支电路



在流水线中提前进行分支决定

从EX/MEM中旁路源操作数

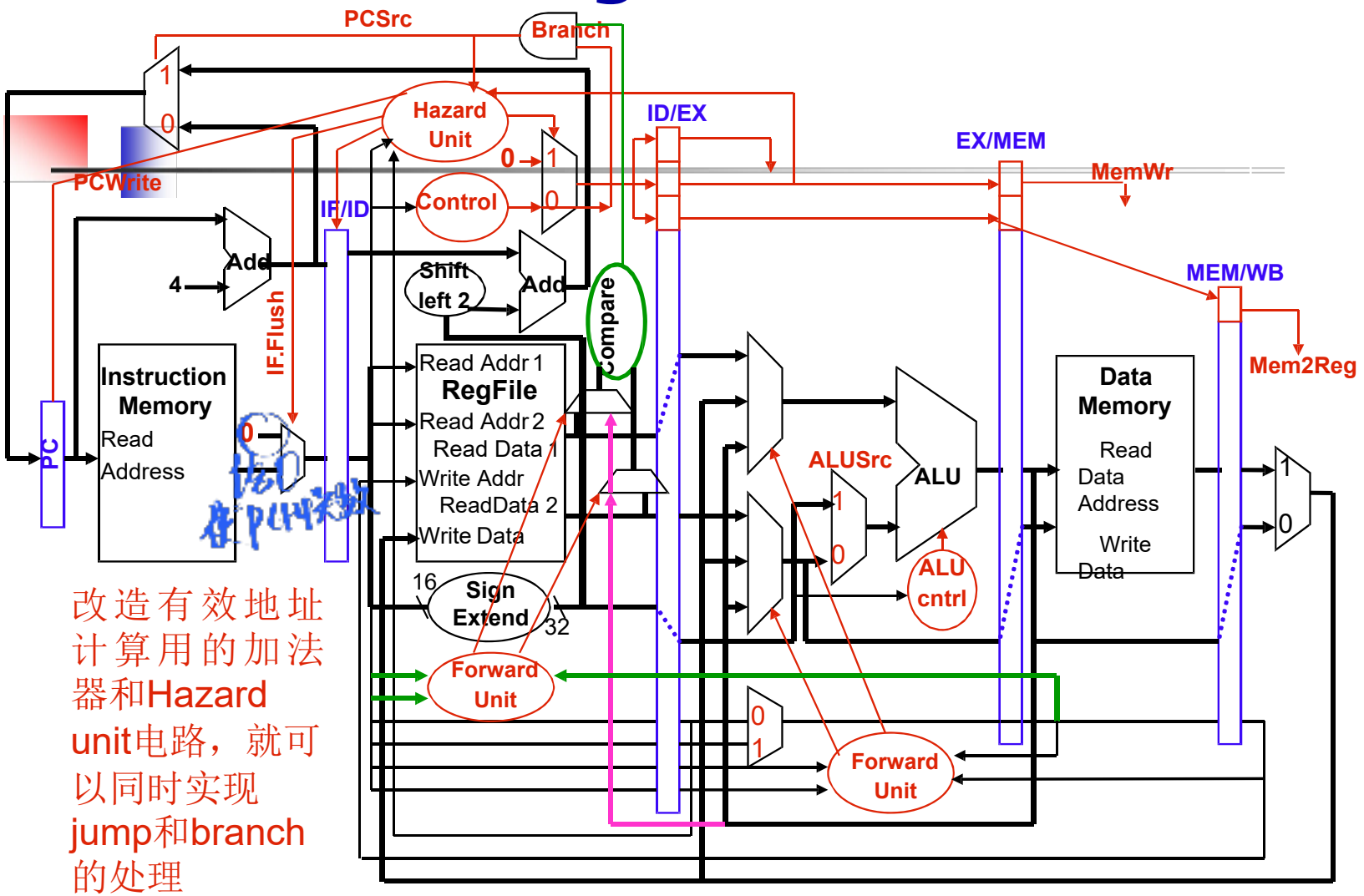
if (IDcontrol.Branch
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd = IF/ID.RegisterRs))
ForwardC = 1 if (IDcontrol.Branch
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd = IF/ID.RegisterRt))
ForwardD = 1

从前条指令的关联寄存器结果forward到ID阶段的比较器中

- MEM/WB “forwarding” 是考虑到正常的寄存器关联引起的竞争
- 如果分支指令中参与比较的寄存器是前一条指令的输出，则必须阻塞一个周期后再判断以等待前一条指令的结果

(因为比较运算在ID阶段与前一条指令的EX阶段是同时的)

ID Stage 分支电路



改造有效地址计算用的加法器和Hazard unit电路，就可以同时实现jump和branch的处理

分支预测

- 假定一个分支的结果，不等待实际的分支判断，直接根据假定的分支结果执行流程

（1）总假设分支不发生

（2）总假设分支会发生

（3）动态预测：

分支决策移到了IF段，用于转发和阻塞的多路复用器和控制也不得不复制到解码段

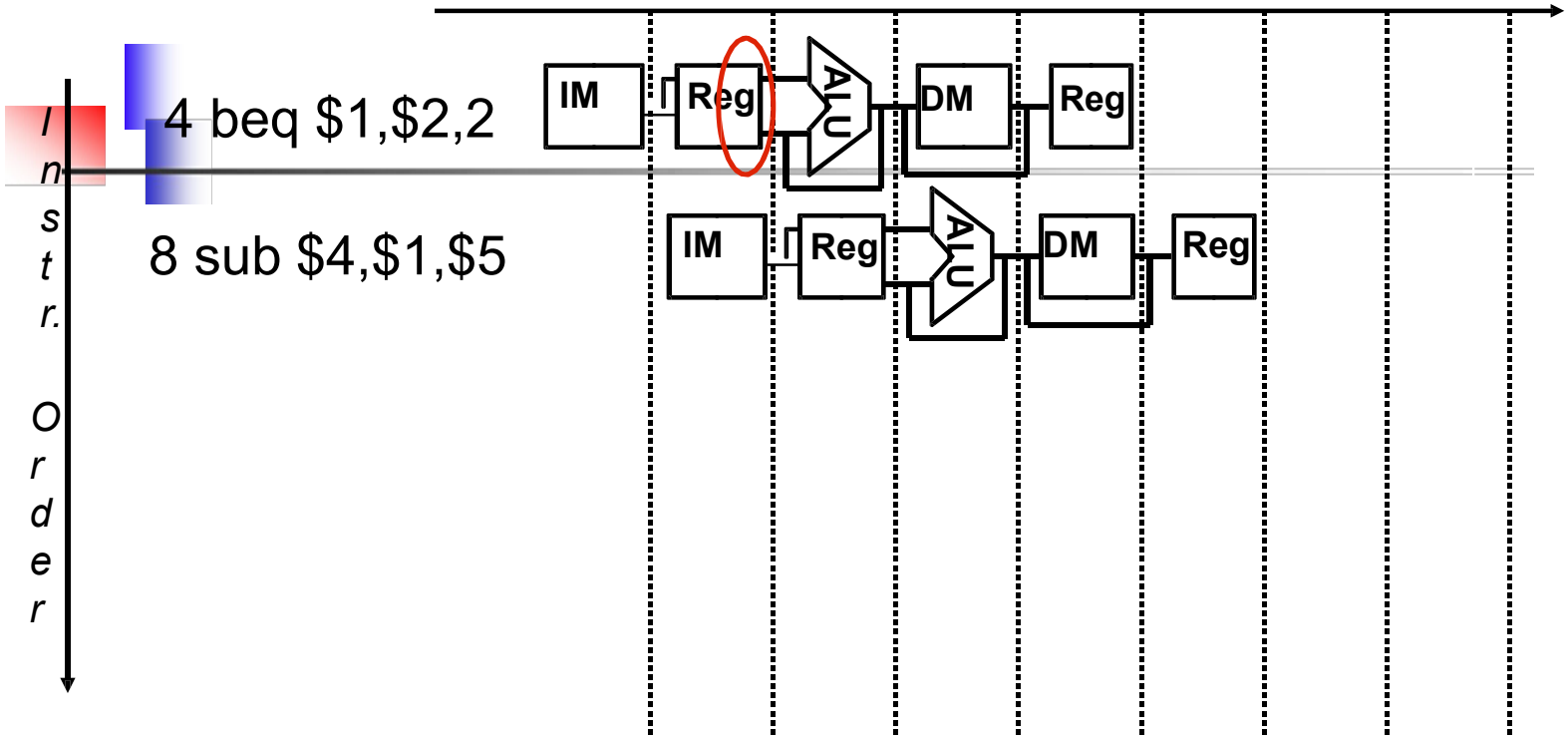
- 延迟分支

分支预测——不发生

（1）总假设分支不发生 – 遇到分支指令仍然按照正常的流程取指令,如果在流水线后期确实分支发生,则阻塞并取消已进入流水线指令

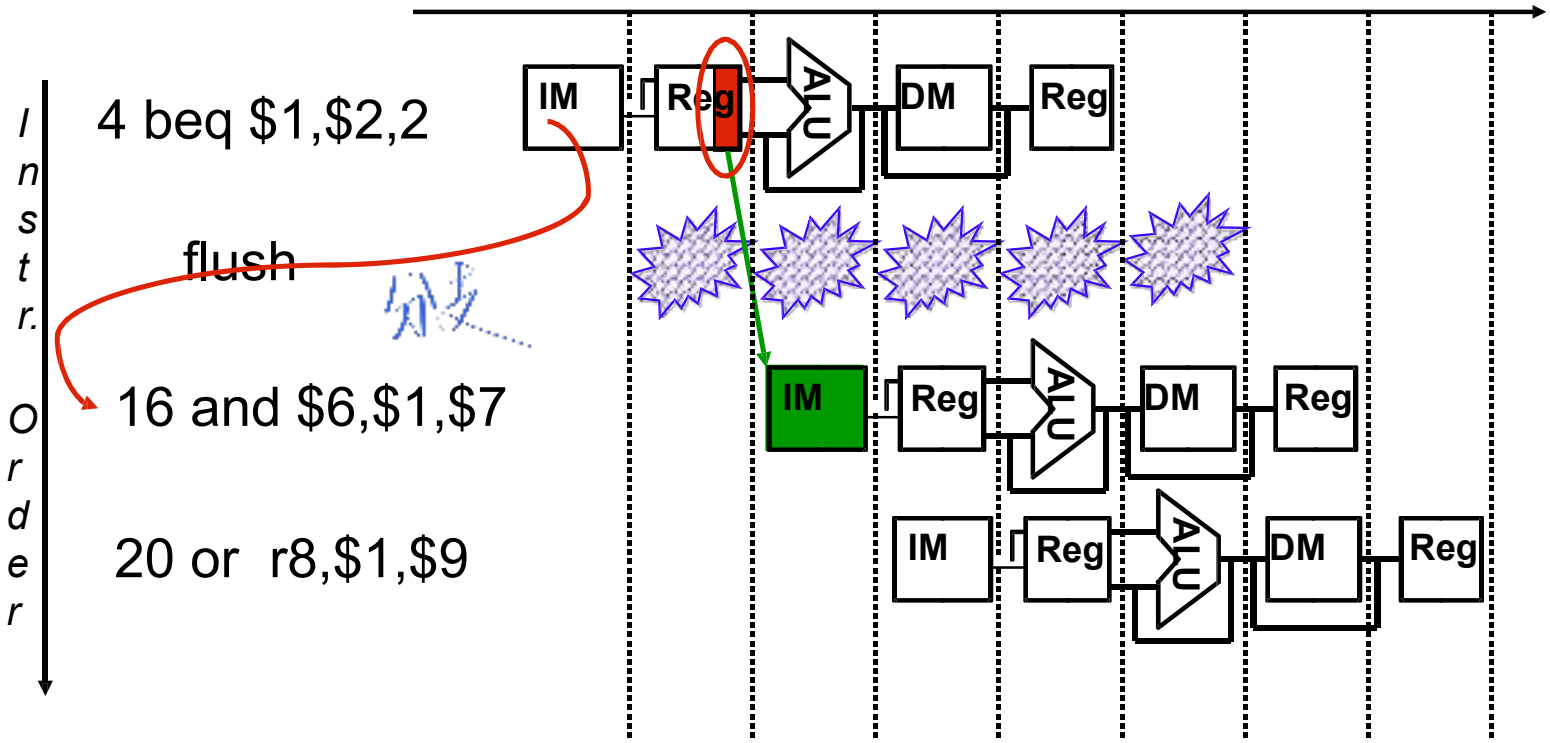
- 如果分支发生, 清除分支指令后的流水线内指令
  - IF, ID, and EX if branch logic in MEM – three stalls
  - IF if branch logic in ID – one stall
- 确保这些被清除的指令未改变机器状态– 由于机器状态是被(MemWrite or RegWrite) 控制信号在MEM和WB阶段改写的，因此这点可以保证
- 分支后继续流水线执行

预测错误时清除



- 为了实现IF阶段指令清零, 增加IF.Flush 控制线于IF/ID pipeline register,这个功能事实上在control hazard改造时已经加了.

预测错误时清除



- 为了实现IF阶段指令清零, 增加IF.Flush 控制线于IF/ID pipeline register,这个功能事实上在control hazard改造时已经加了.

分支预测——假设总发生

- 通过静态的假定一个分支判断结果的输出解决分支竞争

2 Predict taken – 总假定分支会发生

  - 发生分支引起一个周期的阻塞 (假定ID阶段的分支判断电路已经加上了)

- 这种策略损失效率

- 另外的解决方案就是再增加硬件进行动态预测

（3）Dynamic branch prediction – 利用运行时的信息进行动态预测

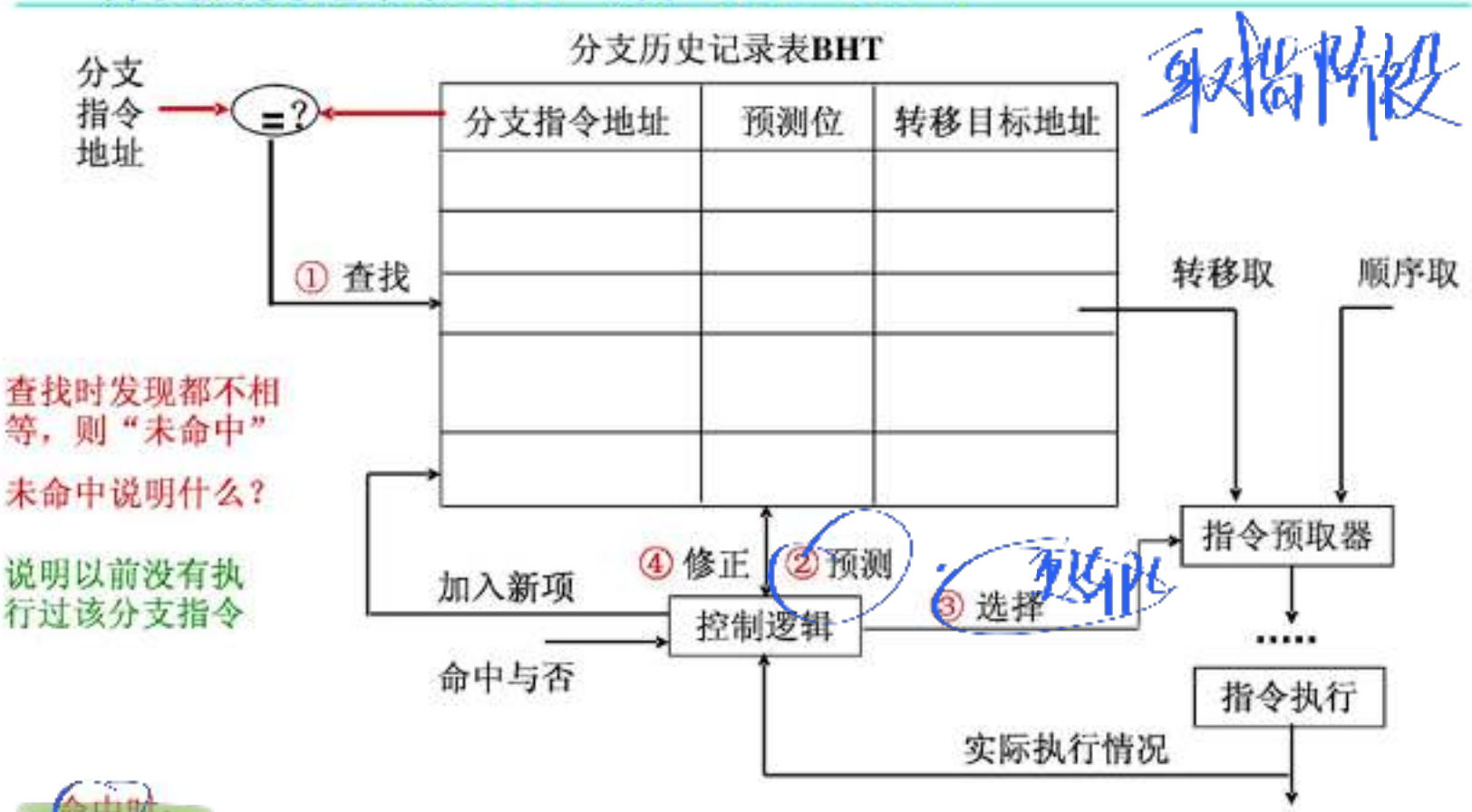
分支预测——动态预测

- 再增加硬件进行动态预测

3 Dynamic branch prediction – 利用运行时的信息进行动态预测

- 需要实现一个IF阶段分支预测缓存(即 branch history table (BHT)), 可以以*PC的低位地址*为索引,以1bit记录上次是否发生了转移
- *why?*

分支历史记录表BHT（或BTB、BPB）



查找时发现都不相等，则“未命中”

未命中说明什么？

说明以前没有执行过该分支指令

命中时：

根据预测位，选择“转移取”还是“顺序取”

未命中时：

加入新项，并填入指令地址和转移目标地址，初始化预测位

分支预测——动态预测

分支历史记录表**BHT(Branch History Table）**

- 每个表项有分支地址的地位索引，故在**IF**段可以取到预测位：
  - 低位地址相同的分支指令共享一个表项
    - 可能取得时其它指令的预测位，会不会有问题？
  - 仅用于预测，不影响执行结果

现在几乎所有的处理器都采用动态预测。

动态分支预测

- 一个bit可能会引起问题 (有可能这个分支指令和上一次的不是同一条指令仅仅是低位地址相同, 有可能这次就是和上次不一样) 不过这些只是一些性能牺牲
- 如果预测错误则清除相关指令,重新开始流水线,并将BHT中状态字更改
- **BHT预测是否跳转,**引入branch target buffer 将跳转地址甚至该处的指令缓存起来,当分支指令来时,如果要分支,则直接在IF的BTB中取出指令,即便跳转也可以不阻塞
- 请大家阅读教材，关于1bit和2bit预测状态机

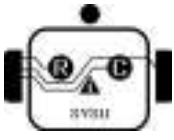


动态分支预测方法

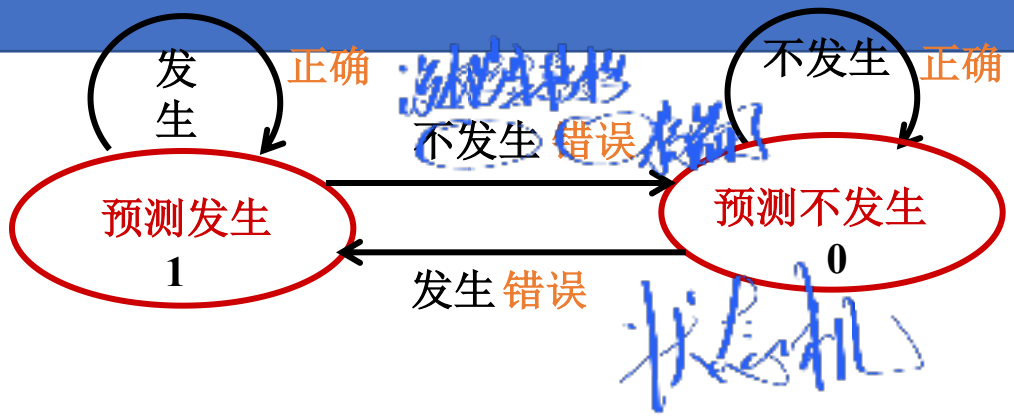
- 简单的静态分支预测方法的预测成功率不高，应考虑动态预测
- 动态预测基本思想：
  - 利用最近转移发生的情况，来预测下一次可能发生的转移
  - 预测后，在实际发生时验证并调整预测
  - 转移发生的历史情况记录在BHT中（有多个不同的名称）
    - 分支历史记录表BHT（Branch History Table）
    - 分支预测缓冲BPB（Branch Prediction Buffer）
    - 分支目标缓冲BTB（Branch Target Buffer）
  - 每个表项由分支指令地址低位作索引，故在IF阶段就可以取到预测位
    - 低位地址相同的分支指令共享一个表项，所以，可能取的是其他分支指令的预测位。会不会有问题？
    - 由于仅用于预测，所以不影响执行结果

现在几乎所有的处理器都采用动态预测（dynamic predictor）





一位预测状态图



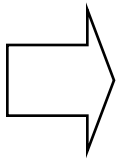

```
Loop:   g = g +A[i];
        i = i+ j;
        if (i != h) go to Loop:
```

- 指令预取时，按照预测读取相应分支的指令
  - ‘1’：选择“转移取”
  - ‘0’：选择“顺序取”
- 指令执行时，按实际执行结果修改预测位
  - 对照状态转换图来进行修改
  - 例如：对于一个循环分支
    - 若初始状态为0(再次循环时为0)，则第一次和最后一次都错
    - 若初始状态为1，则只有最后一次会错

```
Loop: add $7, $3, $3      ; i*2
      add $7, $7, $7      ; i*4
      add $7, $7, $5
      lw $6, 0($7)   ; $6=A[i]
      add $1, $1, $6      ; g= g+A[i]
      add $3, $3, $4
      bne $3, $2, Loop
      … …
```

举例：双重循环的一位动态预测

```
into sum (int N)
{
int i, j, sum=0;
for (i=0; i < N; i++)
    for (j=0; j <N;
  j++)
        sum=sum+1;
return sum;
}
```



```
… …
Loop-i:  beq $t1,$a0, exit-i    # 若( i=N)则跳出外循环
         add $t2, $zero, $zero  #j=0
Loop-j:  beq $t2, $a0, exit-j   # 若(j=N)则跳出内循环
         addi $t2, $t2, 1       # j=j+1
         addi $t0, $t0, 1       #sum=sum+1
         j  Loop-j
exit-j:  addi $t1, $t1, 1       # i=i +1
         j  Loop-i
exit-i:  … …
```

外循环中的分支指令共执行N+1次，
内循环中的分支指令共执行N×(N+1)次。

N=10, 分别90.9%和82.7%

N=100, 分别99%和98%

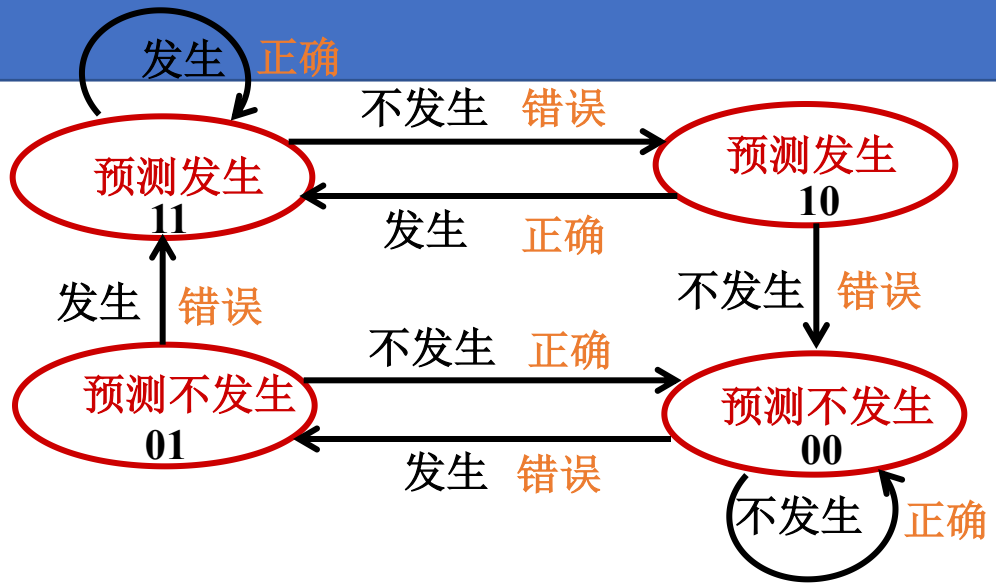
预测位初始为0，外循环只有最后1次预测错误；跳出内循环时预测 位变为1，再进入内循环时，第一次总是预测错误，并且任何一次循环的最后一次总是预测错误，因此，内循环有1+2×(N–1)次预测错误。

N越大准确率越高！





两位预测状态图

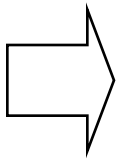


预测发生：选择“转移取”
预测不发生：选择“顺序取”

- 基本思想：只有两次预测错误才改变预测方向
  - 11状态时预测发生（强转移），实际不发生时，转到状态10（弱转移），下次仍预测为发生，如果再次预测错误（实际不发生），才使下次预测调整为不发生00
- 好处：连续两次发生不同的分支情况时，会预测正确

举例：双重循环的两位动态预测

```
into sum (int N)
{
int i, j, sum=0;
for (i=0; i < N; i++)
    for (j=0; j <N;
  j++)
        sum=sum+1;
return sum;
}
```



```
… …
Loop-i:  beq $t1,$a0, exit-i    # 若( i=N)则跳出外循环
         add $t2, $zero, $zero  #j=0
Loop-j:  beq $t2, $a0, exit-j   # 若(j=N)则跳出内循环
         addi $t2, $t2, 1       # j=j+1
         addi $t0, $t0, 1       #sum=sum+1
         j  Loop-j
exit-j:  addi $t1, $t1, 1       # i=i +1
         j  Loop-i
exit-i:  … …
```

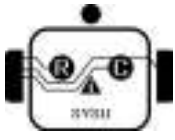
外循环中的分支指令共执行N+1次，
内循环中的分支指令共执行N×(N+1)次。

N=10, 分别90.9%和90.9%

N=100, 分别99%和99%

预测位初始为00，外循环只有最后一次预测错误；跳出内循环时预测位变为01，再进入内循环时，第一次预测正确，只有最后一次预测错误，因此，总共有N次预测错误。

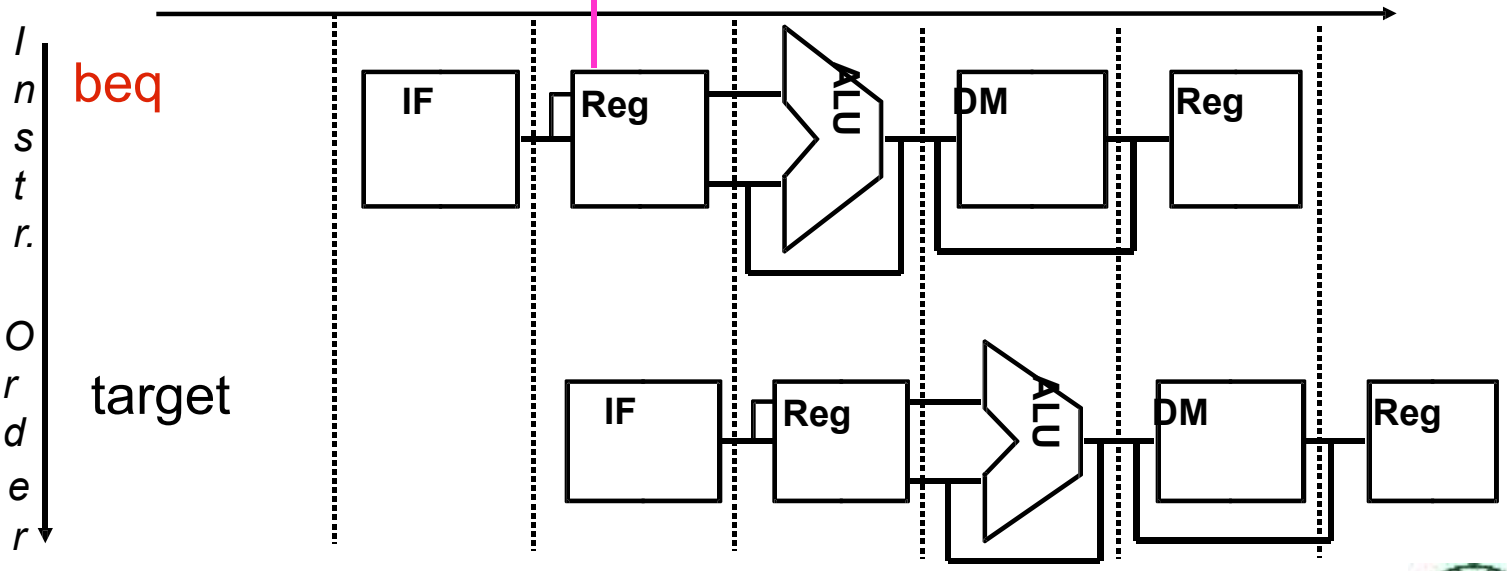




动态分支预测

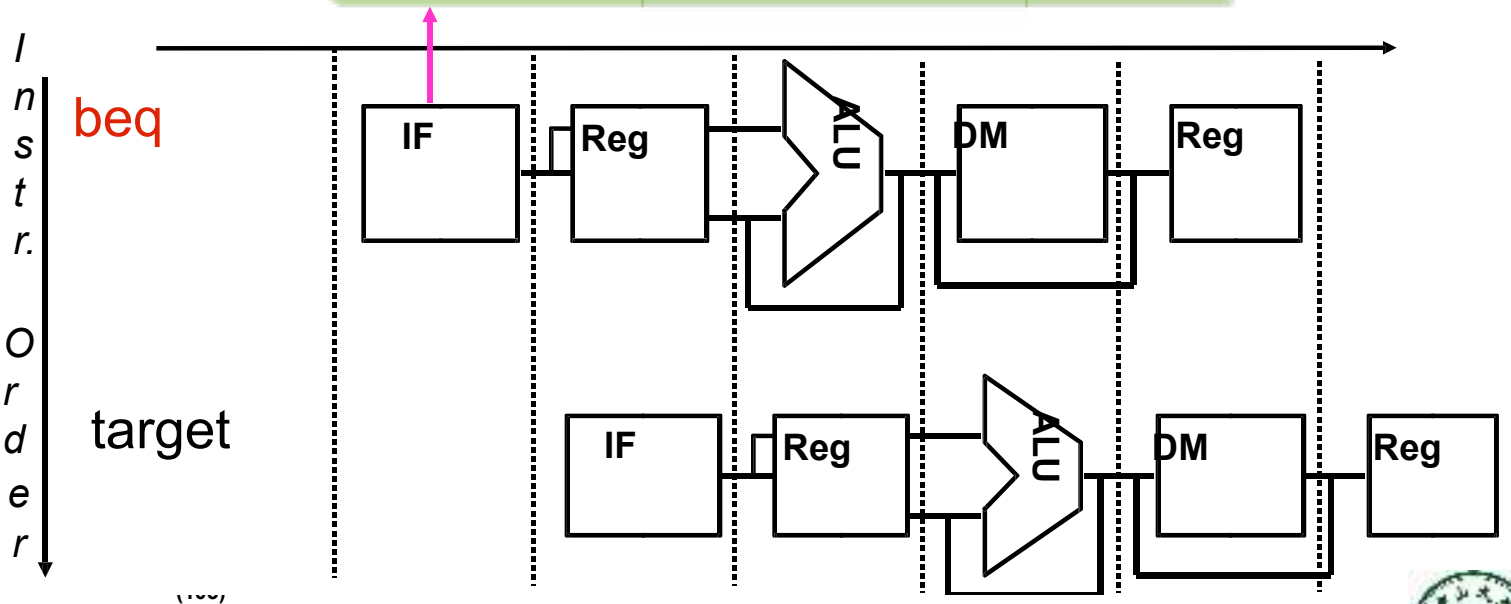
由第一个分支指令建立BHT和BTB：

- ID译码知道是beq，把这条指令的地址存下来，把这条指令是否跳转（1/0）存下来，把跳转目标地址存下来



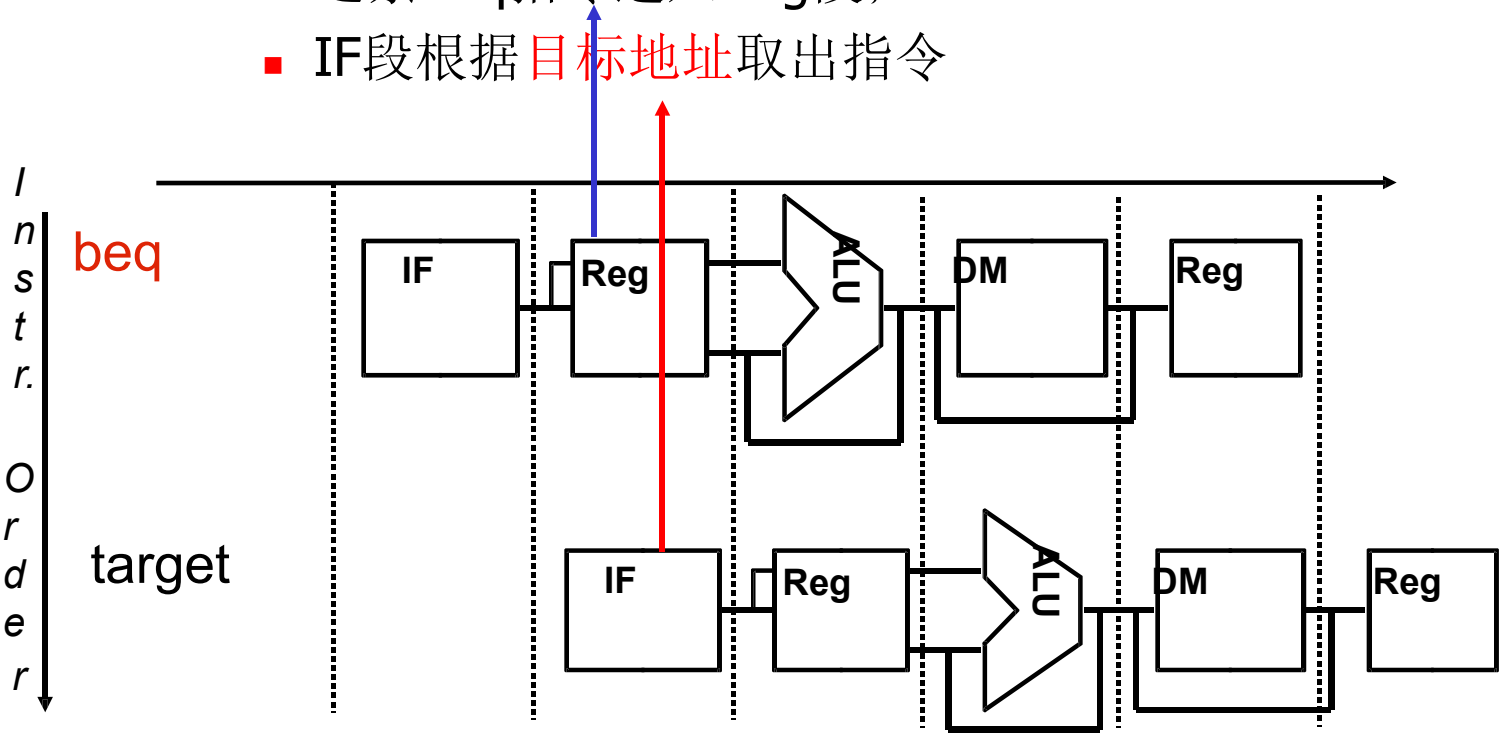
动态分支预测

- **Clock1: PCnew 在送IF指令存储器的同时**
  - 比较是否等于建立BHT和BTB的beq指令的地址，
  - 如果相等，看上一次这条指令是否跳转（1/0），
  - 如果跳转BHT的值为1，把跳转目标地址取出来，送给PC地址选择器作为下一条指令（Clock2 IF）的地址。



动态分支预测

- **Clock2:**
  - 这条beq指令进入Reg段；
  - IF段根据目标地址取出指令



延迟分支

- 首先还是要将分支的判断电路放在ID阶段
- 延迟分支技术使得分支指令的含义与原来不同,该机制规定分支指令后面紧跟的指令必须被执行，无论条件判断结果如何
  - MIPS支持该机制, 于是MIPS 编译器将一条不受分支条件影响的指令放在分支指令之后，这样可以隐藏该指令的执行时间
- 处理器流水线级数增加以后需要更多的延迟分支槽数目.
  - 延迟分支毕竟不如动态分支灵活目前已经不那么广泛使用,但是这是个简单的解决方案





分支延迟时间片的调度

- 属于静态调度技术，由编译程序重排指令顺序来实现
- 基本思想：
  - 把分支指令前面的与分支指令无关的指令调到分支指令后面执行，以填充延迟时间片（也称分支延迟槽Branch Delay slot），不够时用nop操作填充

举例：如何对以下程序段进行分支延迟调度？

（假定时间片C=2）

```
lw $1, 0($2)
lw $3, 0($2)
add  $6, $4, $2
beq $3, $5, 2
add $3, $3,$2
sw $1, 0($2)
……
```

调度后

```
lw $3, 0($2)
add  $6, $4, $2
beq $3, $5, 2
lw $1, 0($2)
nop
add $3, $3,$2
sw $1, 0($2)
```

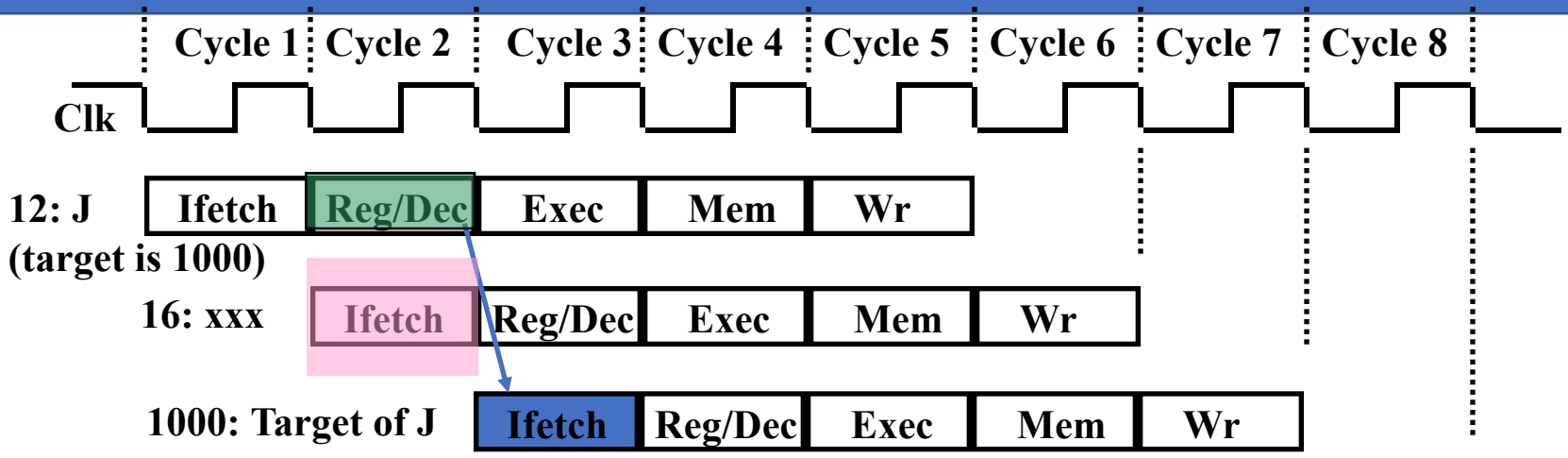
若C=1，则可以：

```
lw $3, 0($2)
add  $6, $4, $2
beq $3, $5, 2
lw $1, 0($2)
add $3, $3,$2
sw $1, 0($2)
……
```

调度后可能带来其他问题：产生新的load-use数据冒险

调度后，降低了分支延迟损失

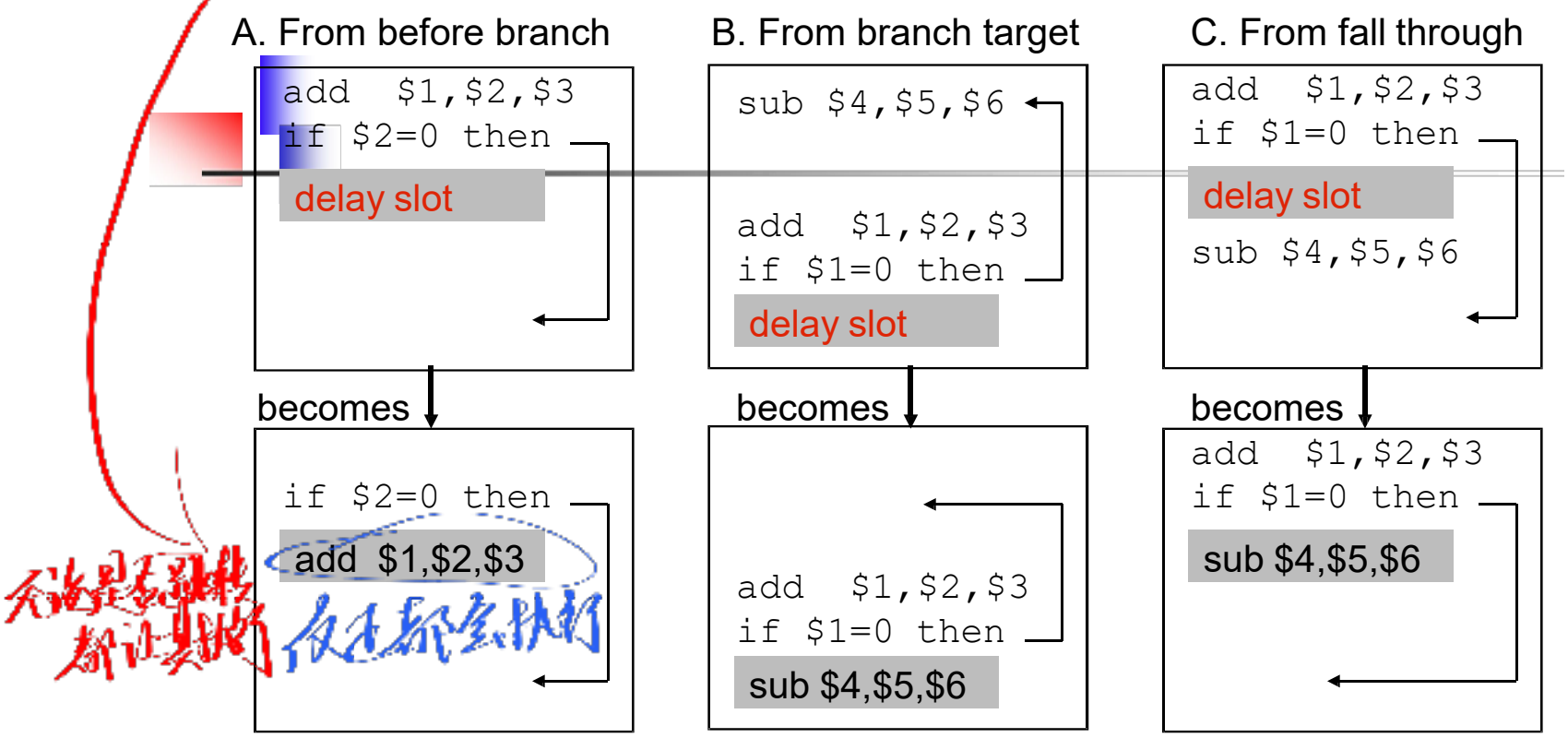
J指令的分支延迟（如果在ID段进行转移地址计算和回送）



- 假设J指令在第1周期取出，但:
  - 第2周期确定“转移目标地址”
  - 第3周期才取出正确指令执行

  结果：一定会有1条指令被错误的取出！要清除该条指令！
- 该情况下：延迟损失时间片C=1

延迟分支条件下编译器调度指令



- A 是最常使用的方案，提高效率，也保持了指令数不变
- B方式中,指令数可能是需要被复制过去，效率不高
- B和C还要保证如果不发生分支也可以这样执行

Control Hazard的解决方法

- 方法1：硬件上阻塞（stall）分支指令后三条指令的执行
  - 使后面三条指令清0或 其操作信号清0，以插入三条NOP指令
- 方法2：软件上插入三条“NOP”指令

（以上两种方法的效率太低，需结合分支预测进行）

- 方法3：分支预测（Predict）
  - 简单（静态）预测：
    - 总是预测条件不满足(not taken)或满足，

即：继续执行分支指令的后续指令

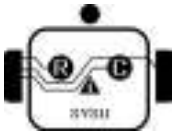
  - 动态预测：
    - 根据程序执行的历史情况，进行动态预测调整，能达90%的预测准确率

注：采用分支预测方式时，流水线控制必须确保错误预测指令的执行结果不能生效，而且要能从正确的分支地址处重新启动流水线工作

- 方法4：延迟分支（Delayed branch）（通过编译程序优化指令顺序！）
  - 把分支指令前面与分支指令无关的指令调到分支指令后面执行，也称延迟转移

另一种控制冒险：异常或中断控制冒险的处理



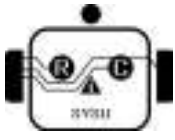


另一种控制冒险：异常和中断

- 异常和中断会改变程序的执行流程
- 某条指令发现异常时，后面多条指令已被取到流水线中正在执行
  - 例如ALU指令发现“溢出”时，已经到EX阶段结束了，此时，它后面已有两条指令进入流水线了
- 流水线数据通路如何处理异常？（举例说明）
  - 假设指令add r1, r2, r3产生了溢出

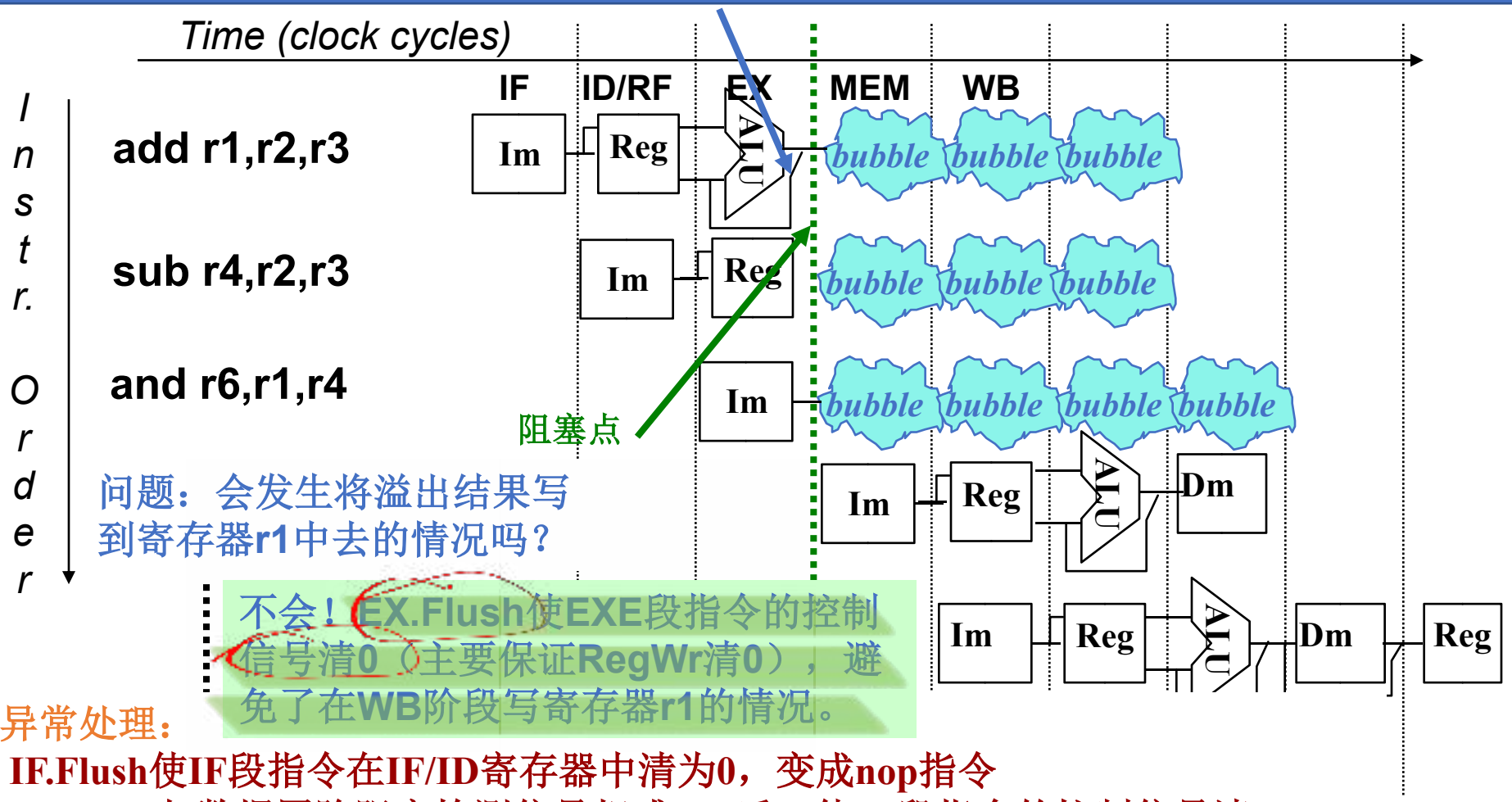
    （记住：MIPS异常处理程序的首地址为0x8000 0180）
  - 处理思路：
    - 清除add指令以及后面的所有已在流水线中的指令
    - 关中断（将中断允许触发器清0）
    - 保存PC或PC+4（断点） 到 EPC
    - 0x8000 0180送PC（从0x8000 0180处开始取指令）





异常的处理

- 异常（溢出）在第一条指令的EXE阶段被检出



异常处理：

IF.Flush使IF段指令在IF/ID寄存器中清为0，变成nop指令

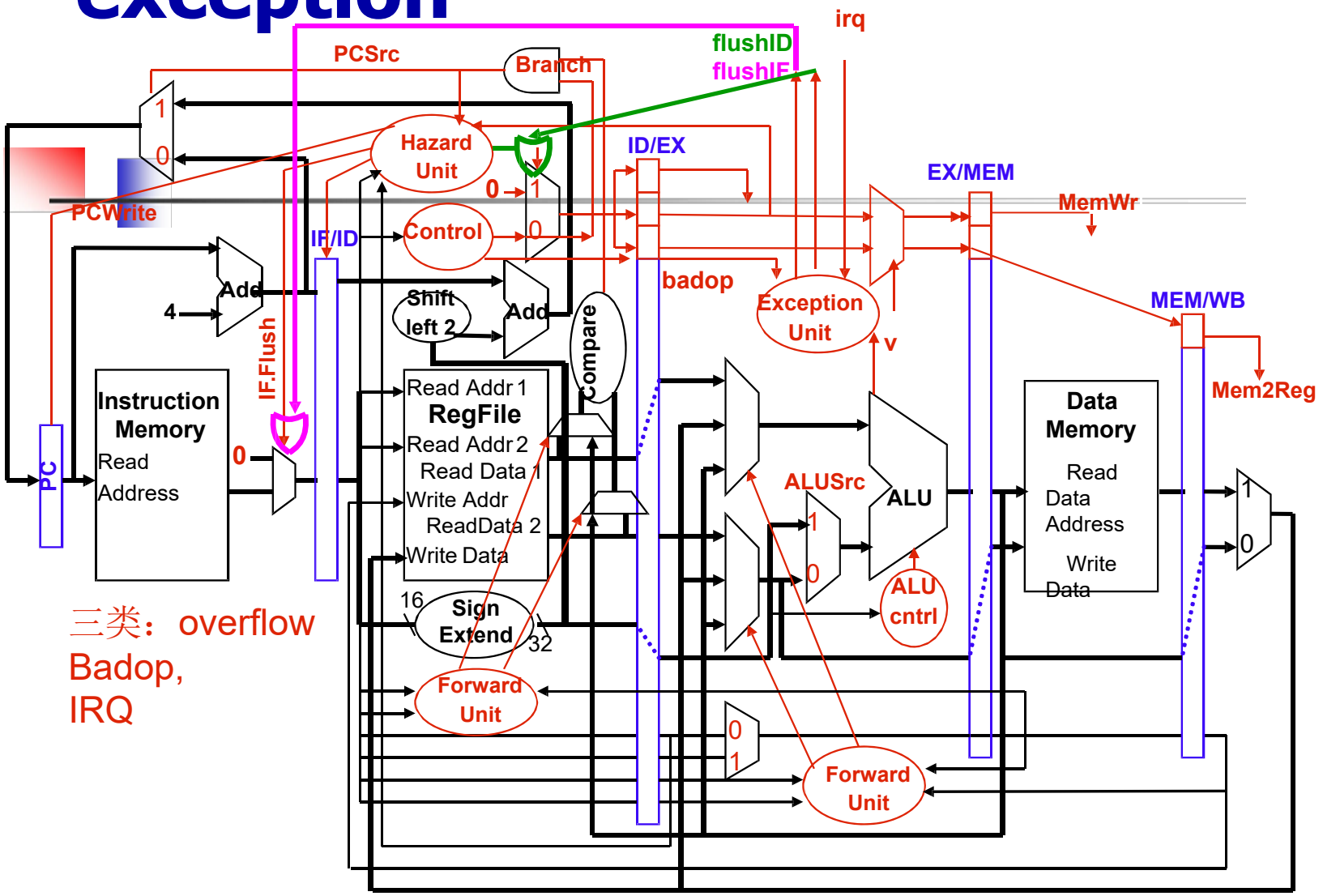
ID.Flush与数据冒险阻塞检测信号相或(or)后，使ID段指令的控制信号清0

EX.Flush使EX段指令的控制信号清0

关中断，并将断点（可能是PC、可能是PC+4）保存到EPC中

将0x8000 0180作为PC的一个输入，并控制PC输入端的多路选择器

exception



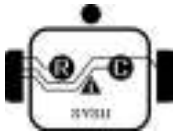
三类：overflow
Badop,
IRQ

(111)

小 结

- Forwarding是解决由于数据关联引起的竞争的有效方式
- 保证前面的指令是有写入操作的, destination 与后面指令source一样, 而且没有更新的数据可以用
- 所有可以forward的地方需要增加新的硬件，如果不能forward则等待
  - EX stage, MEM stage for store, ID stage for early branch
- Load指令需要阻塞
  - 分支也可能需要阻塞
- 控制竞争可以通过静态分支\动态分支\预测等方式处理
  - 预测方式下, 要解决预测错误时的处理





三种处理器实现方式的比较

解：CPU执行时间=指令条数×CPI×时钟周期长度

三种方式的指令条数都一样，所以只要比较CPI和时钟周期长度即可。

各指令类型要求的时间长度为：

Instruction class	Instruction memory	Register read	ALU operation	Data memory	Register write	Total
R-type	200	50	100	0	50	400 ps
Load word	200	50	100	200	50	600 ps
Store word	200	50	100	200		550 ps
Branch	200	50	100	0		350 ps
Jump	200					200 ps

对于单周期方式：

时钟周期将由最长指令来决定，应该是load指令，为600ps

所以，N条指令的执行时间为600N(ps)

对于多周期方式：

时钟周期将取功能部件最长所需时间，应该是存取操作，为200ps

根据各类指令的频度，计算平均时钟周期数为：

CPU时钟周期=5x25%+4x10%+4x52%+3x11%+3x2%=4.12

所以， N条指令的执行时间为4.12x200xN=824N(ps)

三种处理器实现方式的比较

对于流水线方式：

时钟周期和多周期相同，为200ps

Load指令：当发生Load-use依赖时，执行时间为2个时钟，否则1个时钟，故平均执行时间为1.5个时钟；

Store、ALU指令：1个时钟；

因为分支延迟槽是1（C=1），所以：

Branch指令：预测成功时，1个时钟，预测错误时，2个时钟，

所以：平均约为：.75x1+.25x2=1.25个；

Jump指令：2个时钟

平均CPI为：1.5x25%+1x10%+1x52%+1.25x11%+2x2%=1.17

所以，N条指令的执行时间为1.17x200xN=234N(ps)

高级流水线技术

- 高性能流水线 - 指令级并行(ILP)技术
  - 超流水线
  - 多发射流水线
    - 静态多发射（VLIW处理器+编译器静态调度）
    - 动态多发射（超标量处理器+动态流水线调度）
- 静态多发射（VLIW-超长指令字）
  - 编译器静态推测完成“指令打包”和“冒险处理”
- 动态多发射
  - 动态多发射流水线的执行模式
    - 按序发射、按序完成
    - 按序发射、无序完成
    - 无序发射、无序完成

提高性能措施—实现指令级并行

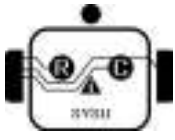
- 流水线实现了指令流内部的并行，这种并行称为指令级并行（ILP）
- 有两种指令级并行策略
  - 超流水线（Super- pipelining）
    - 级数更多的流水线　　CPI=?　　CPI=1
    - 理想情况下，流水线的加速比与流水段的数目成正比
  - 多发射流水线（Multiple issue pipelining ）
    - 多条指令(如整数浮点运算、装入/存储等）同时启动并独立运行
    - 前提：有多个执行部件。如：定浮点、乘/除、取数/存数部件等
    - 结果：能达到小于1的CPI，定义CPI的倒数为IPC
      （例如：四路多发射流水线的理想IPC为4）
    - 需要实现以下两个主要任务
      - **指令打包**
      - **冒险处理**

多发射流水线的两种实现方法

- 静态多发射：由编译器在编译时静态完成指令打包或冒险处理
- 动态多发射：由硬件在执行时动态完成指令打包或冒险处理

- 回顾：流水线中一定会发生冒险
  - 数据冒险：编译nop, 硬件阻塞, 编译调度，硬件动态调度
  - 控制冒险：编译nop, 硬件阻塞, 编译调度，硬件动态调度





静态多发射处理器

- 由编译器在编译时完成“指令打包”或“冒险处理”
  - 指令打包（将同时发射的多条指令合并到一个长指令中）
    - 将一个周期内发射的多个指令看成一条多个操作的长指令：“发射包”
    - 最初被称为“超长指令字”（VLIW-Very Long Instruction Word），采用这种技术的处理器被称为VLIW处理器
    - IA-64采用这种方法，Intel称其为EPIC（Explicitly Parallel Instruction Computer—显式并行指令计算机）
  - 冒险处理（主要是数据冒险和控制冒险）
    - 做法1：完全由编译器通过代码调度和插入nop指令来消除所有冒险，无需硬件实现冒险检测和流水线阻塞
    - 做法2：由编译器通过静态分支预测和代码调度来消除同时发射指令间内部依赖，由硬件检测数据冒险并进行流水线阻塞

注意：保证打包指令内部不会出现冒险！

静态多发射处理器实例

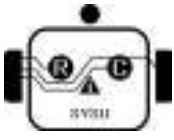
- 实例：MIPS ISA 指令集的静态多发射----2发射处理器

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

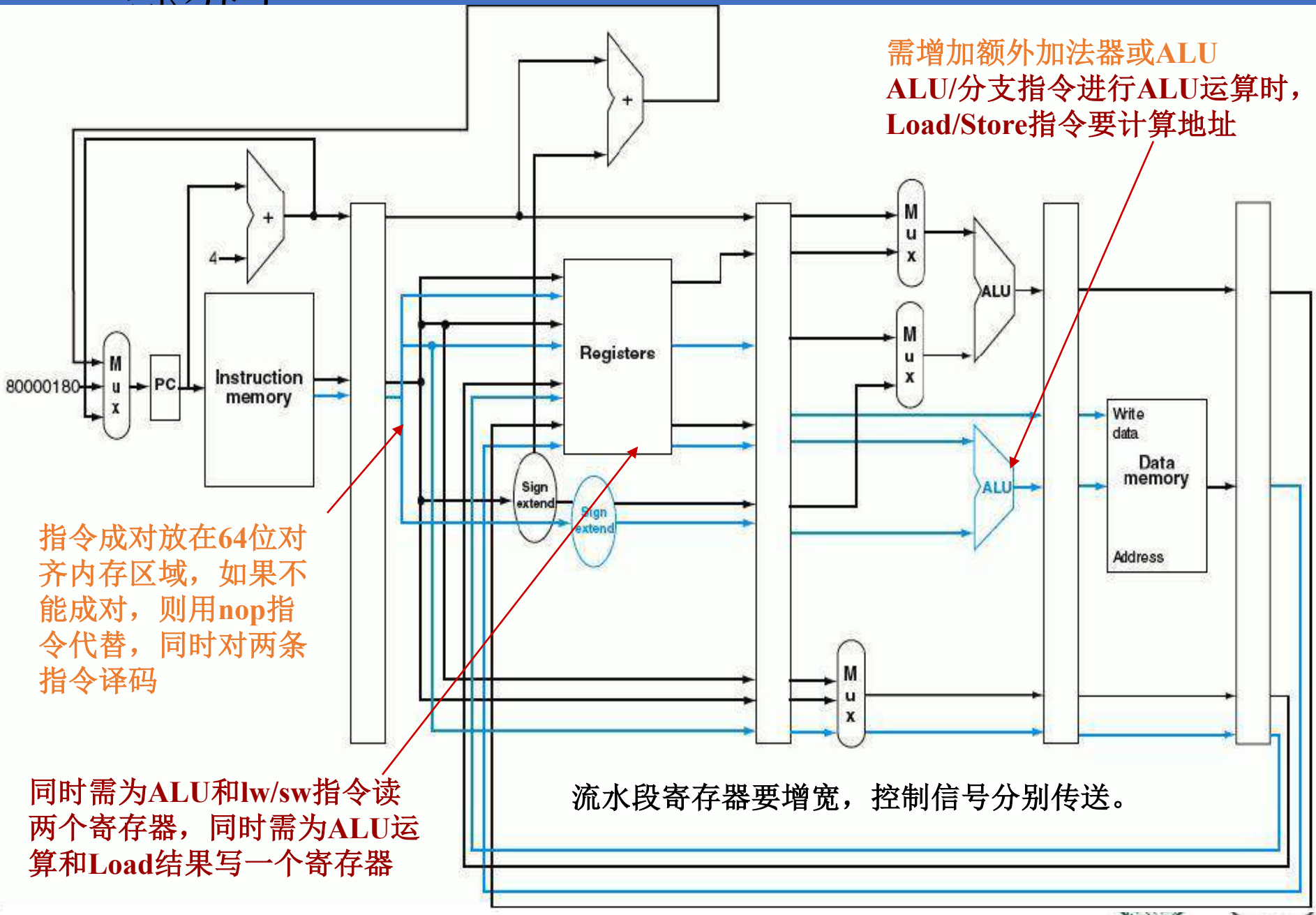
要使原来的MIPS处理器能够同时处理两条流水线，数据通路需要做哪些改进？

1. 同时要能取并译码两条指令，怎么办？
   - 将两条指令打包成64位长指令，前面为ALU/Branch，后面为lw/sw
   - 没有配对指令时，就用nop指令代替
   - 将64位长指令中的两个操作码同时送到控制器（指令译码器）进行译码
2. 两条指令同时要读两个寄存器或写寄存器（和lw配对时），怎么办？
   - 增加两个读口和一个写口
3. 两条指令同时要使用ALU进行运算，怎么办？
   - 增加一个ALU（包括2组输入总线和1组输出总线）





2发射流水线数据通路（蓝色是增加部分）



2发射流水线的特点

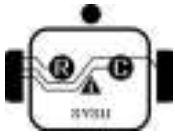
- 优点：潜在性能将提高大约2倍

  （实际上达不到！为什么？ ）

- 缺点：
  - 不可能完全成对发射
  - 为消除结构冒险，需增加额外部件
  - 新增的数据冒险和控制冒险导致性能损失

为了更有效地利用多发射处理器的并行性，必须有更强大的编译器，能够充分消除指令间的依赖关系，使指令序列达到最大的并行性！

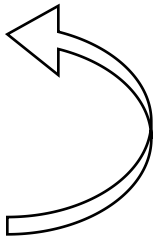




例：2发射MIPS指令调度

- 以下是一段循环代码段

```
Loop: lw    $t0,  0($s1)
      addu  $t0, $t0, $s2
      sw    $t0, 0($s1)
      addi  $s1, $s1, -4
      bne   $s1, $zero, Loop
```



循环内进行的是数组访问！

- 为了能在2发射MIPS流水线中有效执行，该怎样重新排列指令
  - 调度方案如下：没有指令配对时，用nop指令代替

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw $t0, 0($s1)	1
	addi $s1,$s1,-4		2
	addu $t0,$t0,$s2		3
	bne $s1,$zero,Loop	sw $t0, 4($s1)	4

一个循环内，五条指令在四个时钟内完成，实际CPI为0.8，即: IPC=1.25

在循环中访问数组的更好的调度技术是“循环展开”

The Scheduled Code (Unrolled)

	ALU or branch	Data transfer	CC
lp:	addi $s1,$s1,-16	lw $t0,0($s1)	1
		lw $t1,12($s1)	2
	addu $t0,$t0,$s2	lw $t2,8($s1)	3
	addu $t1,$t1,$s2	lw $t3,4($s1)	4
	addu $t2,$t2,$s2	sw $t0,16($s1)	5
	addu $t3,$t3,$s2	sw $t1,12($s1)	6
		sw $t2,8($s1)	7
	bne $s1,$0,lp	sw $t3,4($s1)	8

- Eight clock cycles to execute 14 instructions for a
  - CPI of 0.57 (versus the best case of 0.5)
  - IPC of 1.8 (versus the best case of 2.0)

用“循环展开”技术进行指令调度（自学）

- 基本思想：展开循环体，生成多个副本，在展开的指令中统筹调度
  - 为简化起见，假定循环执行次数是4的倍数
  - 指令最佳调度序列如下：

	ALU or branch instruction		Data transfer instruction		Clock cycle
Loop:	addi	$s1,$s1,-16	lw	$t0, 0($s1)	1
			lw	$t1,12($s1)	2
	addu	$t0,$t0,$s2	lw	$t2, 8($s1)	3
	addu	$t1,$t1,$s2	lw	$t3, 4($s1)	4
	addu	$t2,$t2,$s2	sw	$t0, 16($s1)	5
	addu	$t3,$t3,$s2	sw	$t1,12($s1)	6
			sw	$t2, 8($s1)	7
	bne	$s1,$zero,Loop	sw	$t3, 4($s1)	8

14条指令用了8个时钟，CPI达到8/14=0.57。

需要用到“重命名寄存器”技术，多用了三个临时寄存器**$t1,$t2,$t3**

并增加了代码大小（存储空间变大）

系统程序员必须非常了解机器结构！编译器的好坏直接影响机器的性能！

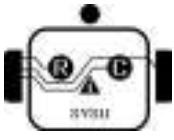
实例：Intel IA-64架构（3-发射流水线！）

- IA-64类似于64位MIPS架构，是Register-Register型的RISC风格指令集
- 但有独特性：要求编译器显式地给出指令级的并行性，Intel称其为EPIC

  Explicitly Parallel Instruction Computer—显式并行指令计算机
- 与MIPS-64架构的区别
  - 更多寄存器：128个整数、128个浮点数、8个专用分支、64个1位谓词
  - 支持寄存器窗口重叠技术
  - 同时发射的指令组织在指令包（bundle）中
  - 引入特殊的谓词化技术，以支持推测执行和消除分支，提高指令级并行度
- EPIC的实现技术
  - 指令组（Instruction Group）：相互间没有寄存器级数据依赖的指令序列
    - 指令组长度任意，用“停止标记”在指令组之间明显标识
    - 指令组内部的所有指令可并行执行，只要有足够硬件且无内存操作依赖
  - 指令包：同时发射的指令重新编码并形成指令包
    - 长度为128，由5位长的模板字段、三个41位长的指令组成
    - 模板字段对应于以下五类功能部件中的三条指令

      整数ALU、非整数ALU（移位和多媒体）、访存、浮点、分支
  - 谓词化：将指令的执行与谓词相关联，而不是与分支指令关联





动态多发射处理器

- 由硬件在执行时动态完成指令打包或冒险处理
- 通常被称为超标量处理器（Superscalar）
  - 在一个周期内执行一条以上指令
- 与VLIW处理器的不同点：
  - VLIW处理器：编译结果与机器结构密切相关，结构有差异的机器上要重新编译
  - 超标量处理器：编译器仅进行指令顺序调整，但不进行指令打包，由硬件根据机器的结构来决定一个周期发射哪几条指令。因此，编译后的代码能够被不同结构的机器正确执行

- 多数超标量处理器都结合动态流水线调度（Dynamic pipeline scheduling）技术

动态流水线调度的通用模型

- 动态流水线的一个重要的思想：通过指令相关性检测和动态分支预测等，在等待解决阻塞时，到后面找指令提前执行！

- 动态流水线的通用模型：
  - 一个指令预取和译码单元：发射
  - 多个并列执行的功能单元：乱序执行
  - 一个提交单元：提交

  ◦ **可分为三种执行模式：** （自学）
    - **按序发射按序完成（Pentium）**
    - **按序发射无序完成（Pentium II和Pentium III）**
    - **无序发射无序完成（Pentium 4）**

动态流水线调度的必要性

- 编译器可以依据数据依赖关系来调度代码，为什么还要超标量处理器来动态调度？
  - 不是所有阻塞都能事先确定，可在阻塞时提前执行无关指令
    - 例如，Cache缺失是不可预见的阻塞
  - 动态分支预测需要根据执行的真实情况进行预测
  - 采用动态调度使得硬件将处理器细节屏蔽起来

高性能微处理器并不能持续进行多条指令的发射：

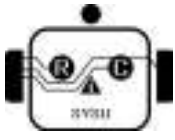
(1) 指令间的高度依赖关系。例如，使用指针的代码段，存在隐含依赖。

(2) 分支指令预测错误。

(3) 内存访问引起的阻塞（Cache缺失、缺页等）使得流水线难以满负荷运转。

skip



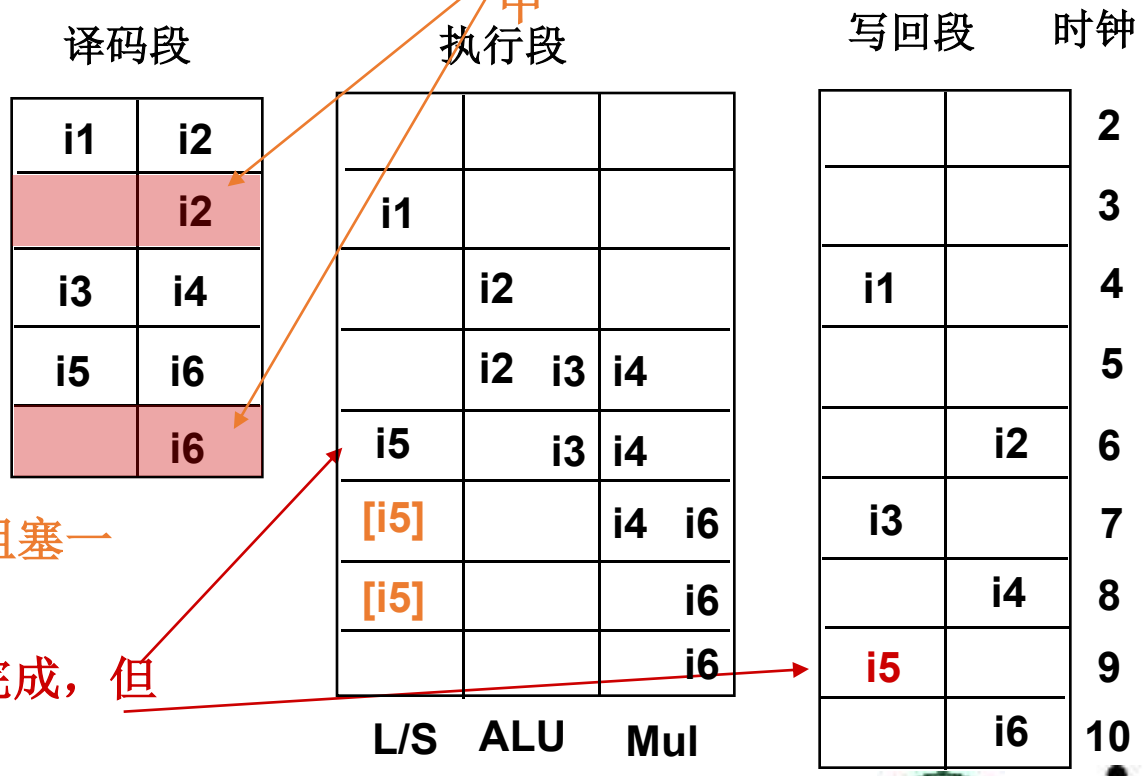


按序发射按序完成

- 举例：2发射超标量，分为取指（F）、译码（D）、执行（E）、写回（W）。F、D、W段在一个时钟周期内完成（可同时有两条指令在这三个阶段）；E段有三个执行部件：Load/Store部件完成数据Cache访问需要1个时钟，整数ALU完成简单ALU操作需2个时钟，整数乘法器完成乘法运算需要3个时钟。执行部件采用流水化方式。

按序发射按序完成的过程如下：

```
i1   lw  $1, A
i2   add $2, $2, $1
i3   add $3, $3, $4
i4   mul $4, $5, $4
i5   lw $6, B
i6   mul $6, $6, $7
```



i1、i2间 / i5、i6间RAW，需阻塞一个时钟周期，必须按序完成！

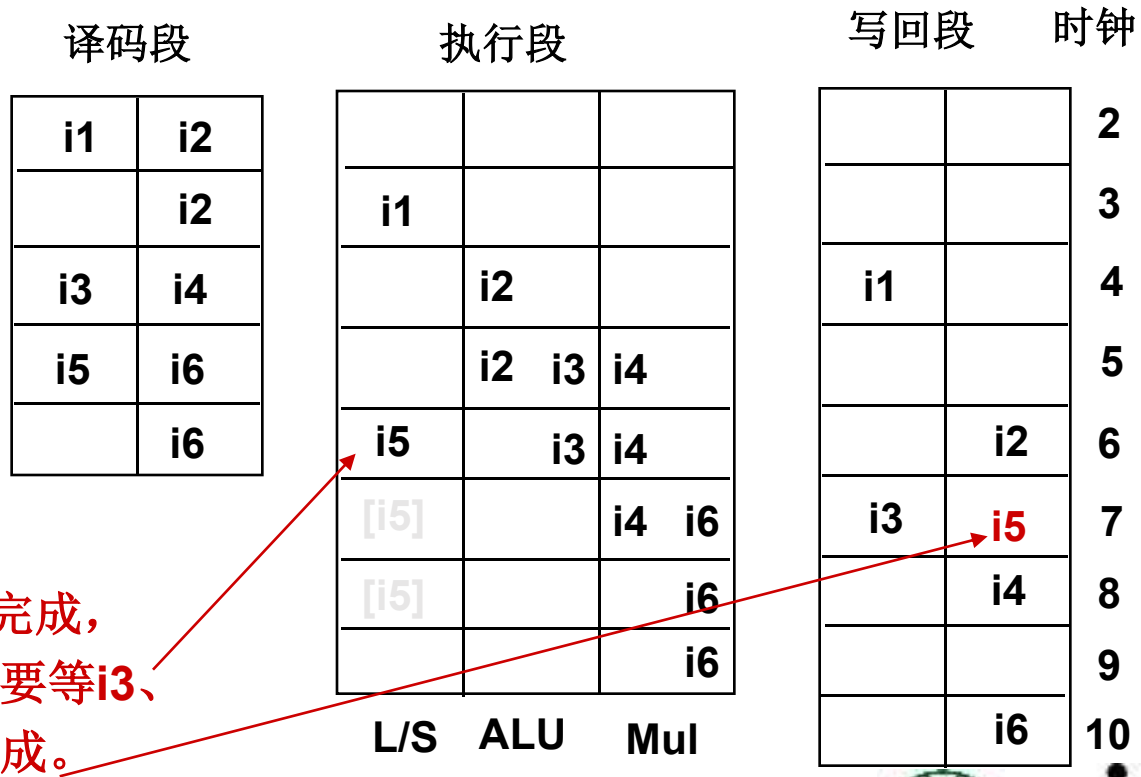
为按序完成，虽i5在时钟6已完成，但一直推迟到i4写回后才写回i5

按序发射无序完成

- 举例：2发射超标量，分为取指（F）、译码（D）、执行（E）、写回（W）。F、D、W段在一个时钟周期内完成（可同时有两条指令在这三个阶段）；E段有三个执行部件：Load/Store部件完成数据Cache访问需要1个时钟，整数ALU完成简单ALU操作只需2个时钟，整数乘法器完成乘法运算需要3个时钟。执行部件采用流水化方式。

按序发射无序完成的过程如下：

```
i1   lw  $1, A
i2   add $2, $2, $1
i3   add $3, $3, $4
i4   mul $4, $5, $4
i5   lw $6, B
i6   mul $6, $6, $7
```



无序完成时，因为i5在时钟6已完成，和i3、i4没有相关性，可以不需要等i3、i4写回后再写回，故可先于i4完成。

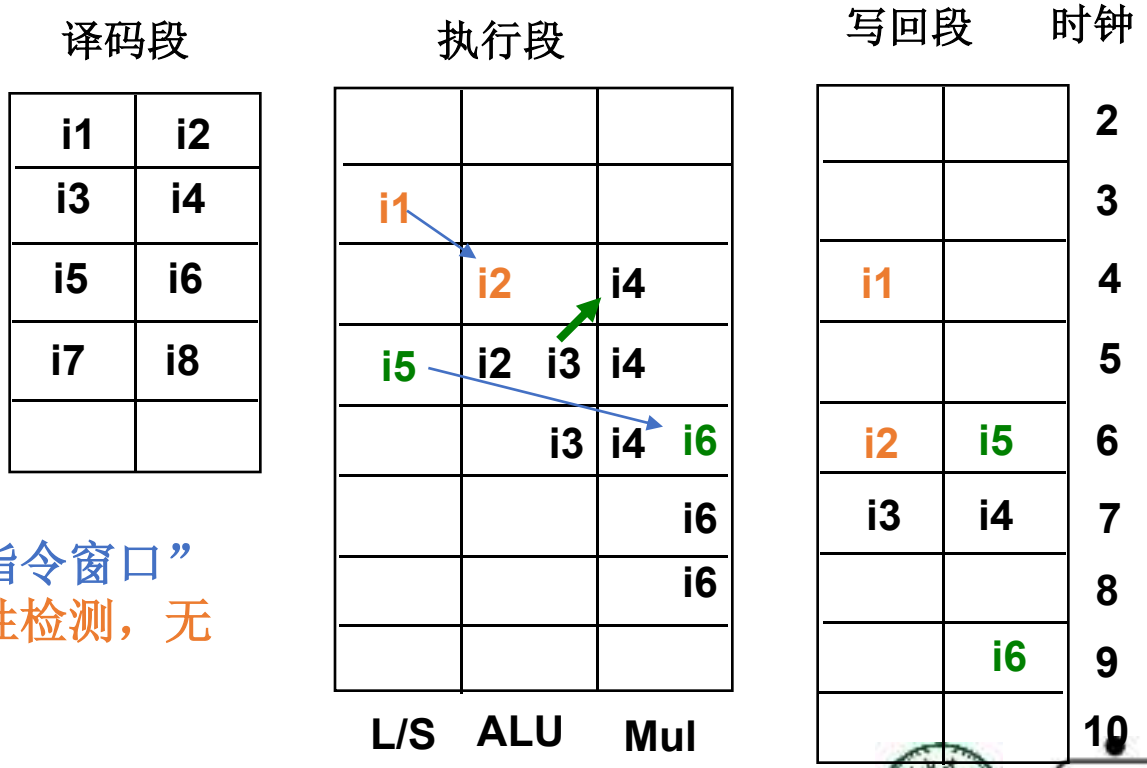
无序发射无序完成

- 举例：2发射超标量，分为取指（F）、译码（D）、执行（E）、写回（W）。F、D、W段在一个时钟周期内完成（可同时有两条指令在这三个阶段）；E段有三个执行部件：Load/Store部件完成数据Cache访问需要1个时钟，整数ALU完成简单ALU操作只需2个时钟，整数乘法器完成乘法运算需要3个时钟。执行部件采用流水化方式。

无序发射无序完成的过程如下：

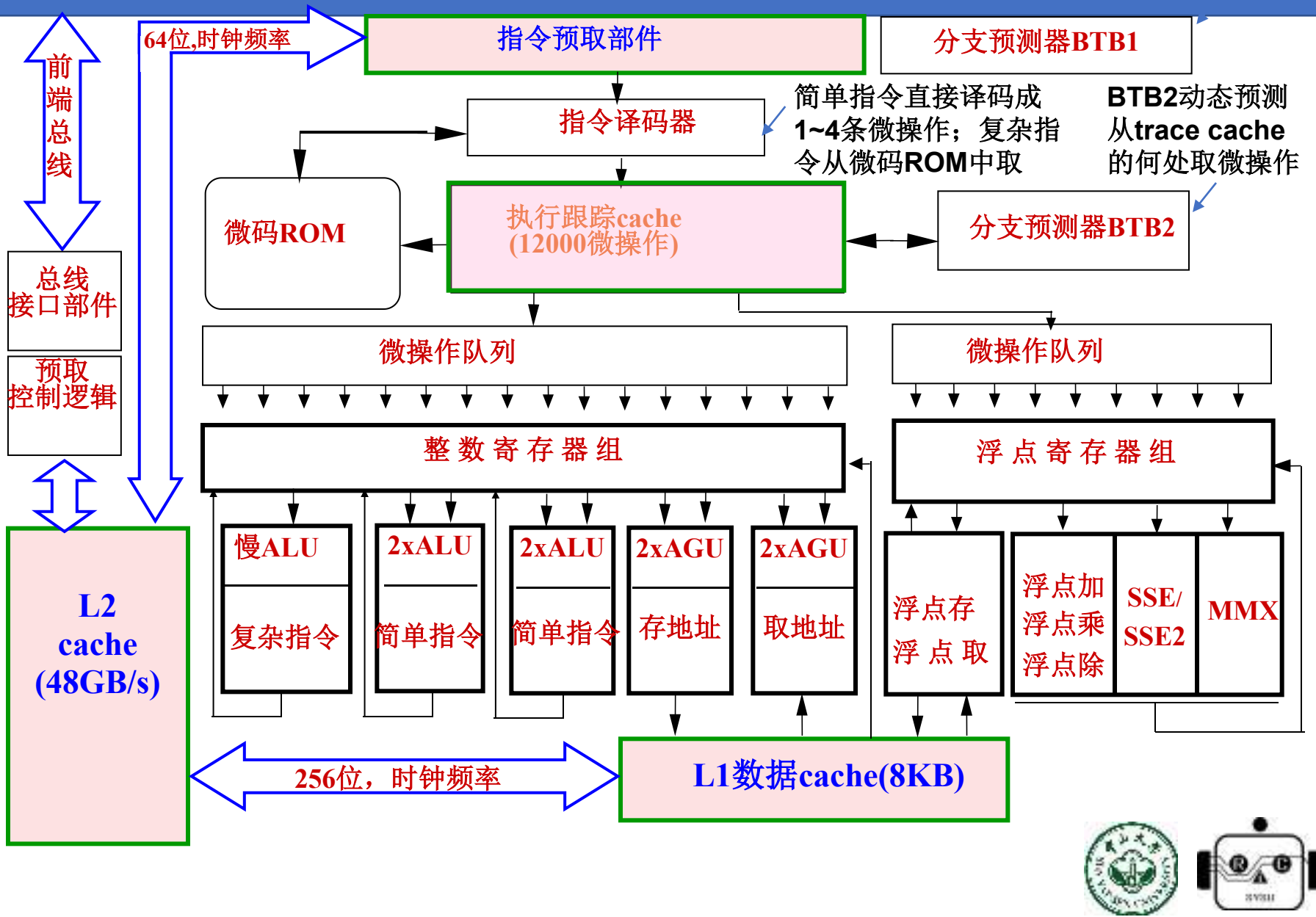
```
i1   lw  $1, A
i2   add $2, $2, $1
i3   add $3, $3, $4
i4   mul $4, $5, $4
i5   lw $6, B
i6   mul $6, $6, $7
```

例如：i4在i3前面发射！

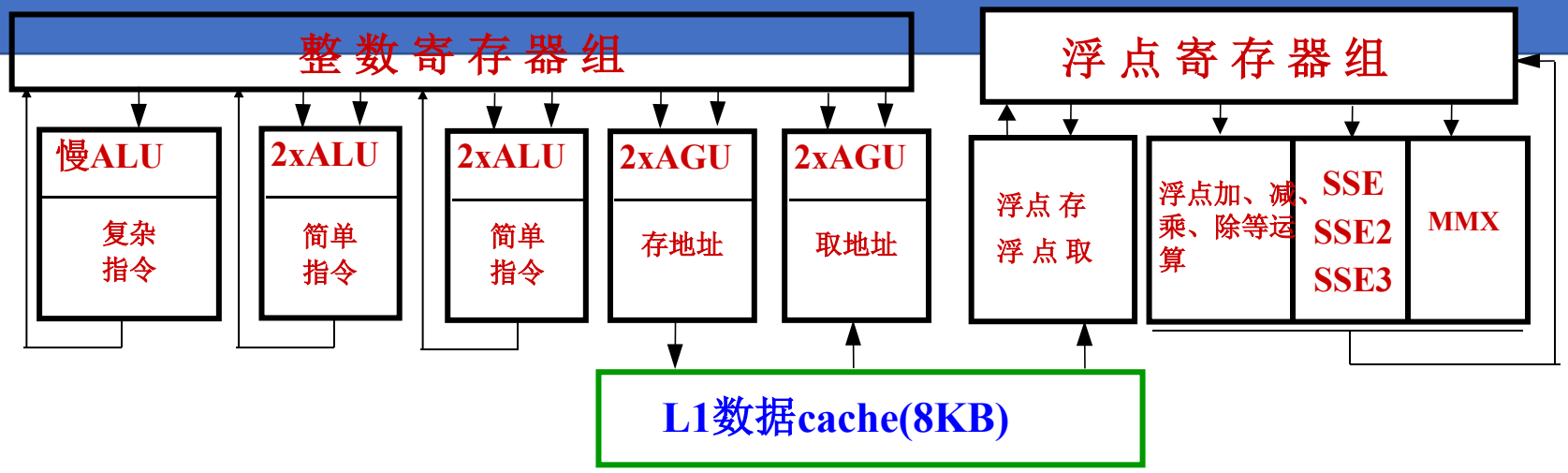


译码后的指令被存放在一个“指令窗口”的缓冲器中，发射前进行相关性检测，无关指令可先行发射和先行完成!

Pentium 4 处理器：20级超标量超流水线（自学）



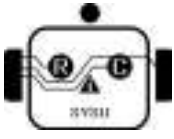
Pentium 4的超标量结构运算器



- 采用超标量（superscalar）结构，一共包含9个运算部件，可同时工作，所花时钟不同
  - 2个高速整数ALU(每个时钟周期进行2次操作)， 用于完成简单的整数运算(如加、减法)
  - 1个慢速整数ALU(需要多个时钟周期才能完成1次操作)，用于完成整数乘、除法运算
  - 2个地址生成部件（AGU），用于计算操作数的有效地址，所生成的地址分别用于从内存取操作数或向内存保存操作结果
  - 1个运算部件用于完成浮点操作数地址的计算
  - 1个运算部件用于完成浮点加法、乘法和除法运算
  - 1个运算部件用于执行流式的SIMD处理（SSE/SSE2/SSE3指令）
  - 1个运算部件用于完成多媒体信号处理（MMX指令）

在运算部件中执行的是微操作，而不是指令！运算器中的操作采用流水方式！





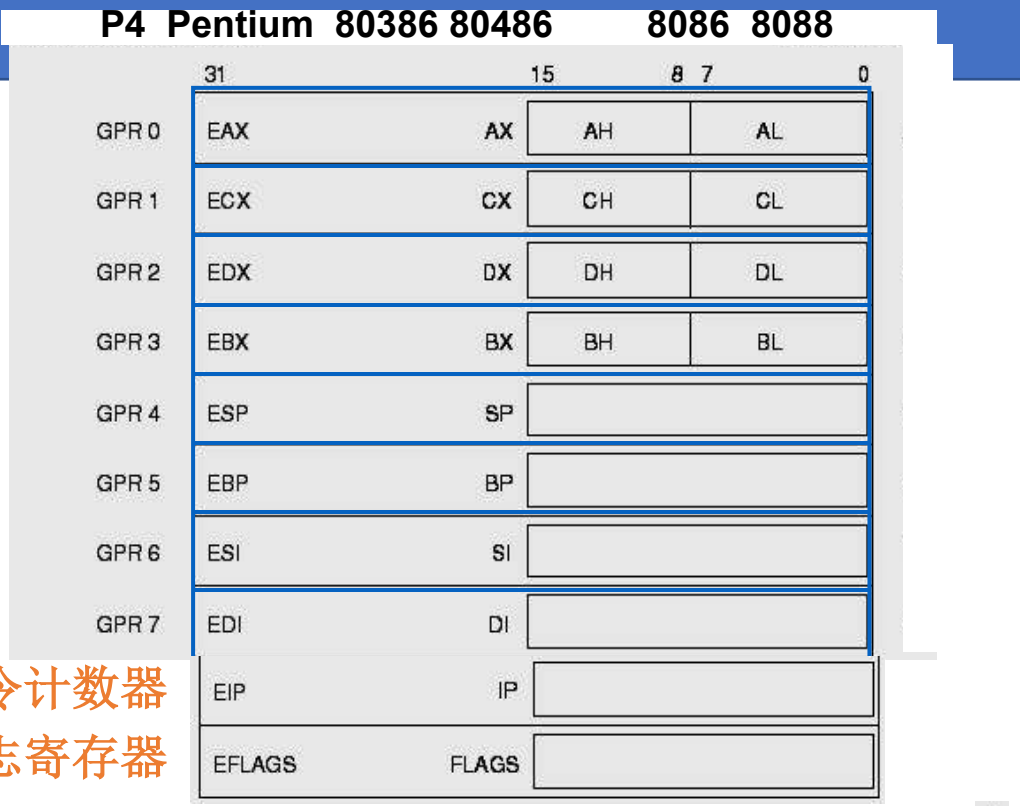
回顾：Pentium 4 的用户可见寄存器组

在Pentium4内部，整数和浮点数各有128个物理寄存器

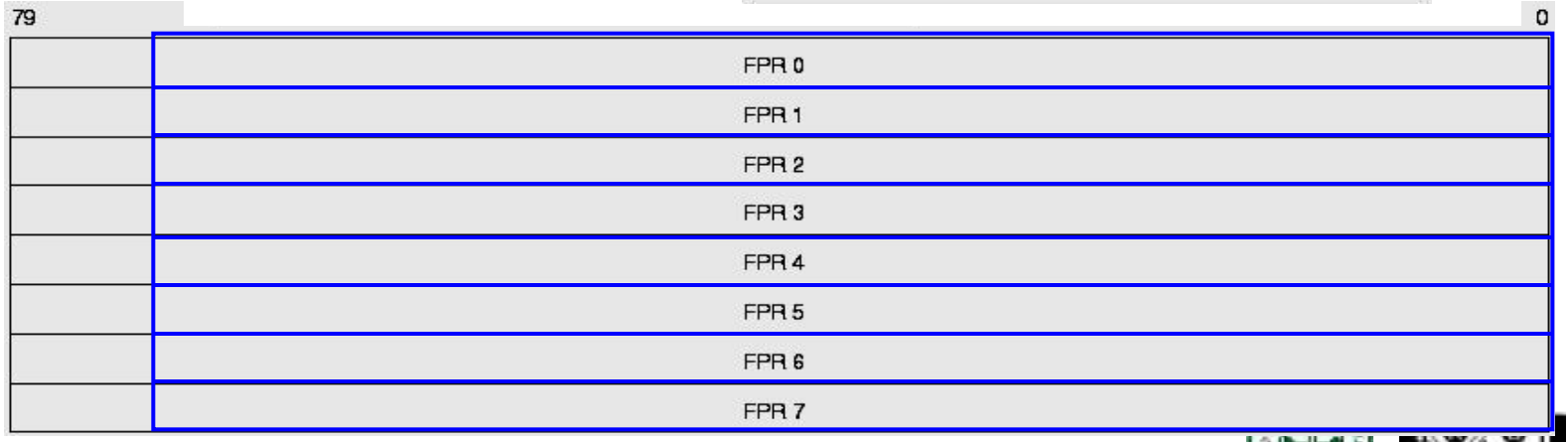
寄存器换名操作：将用户可见的逻辑寄存器换成内部的物理寄存器

寄存器换名时，要确定是真实依赖还是名字依赖（反依赖）

名字依赖时可用不同的物理寄存器替换相同的逻辑寄存器



浮点寄存器组

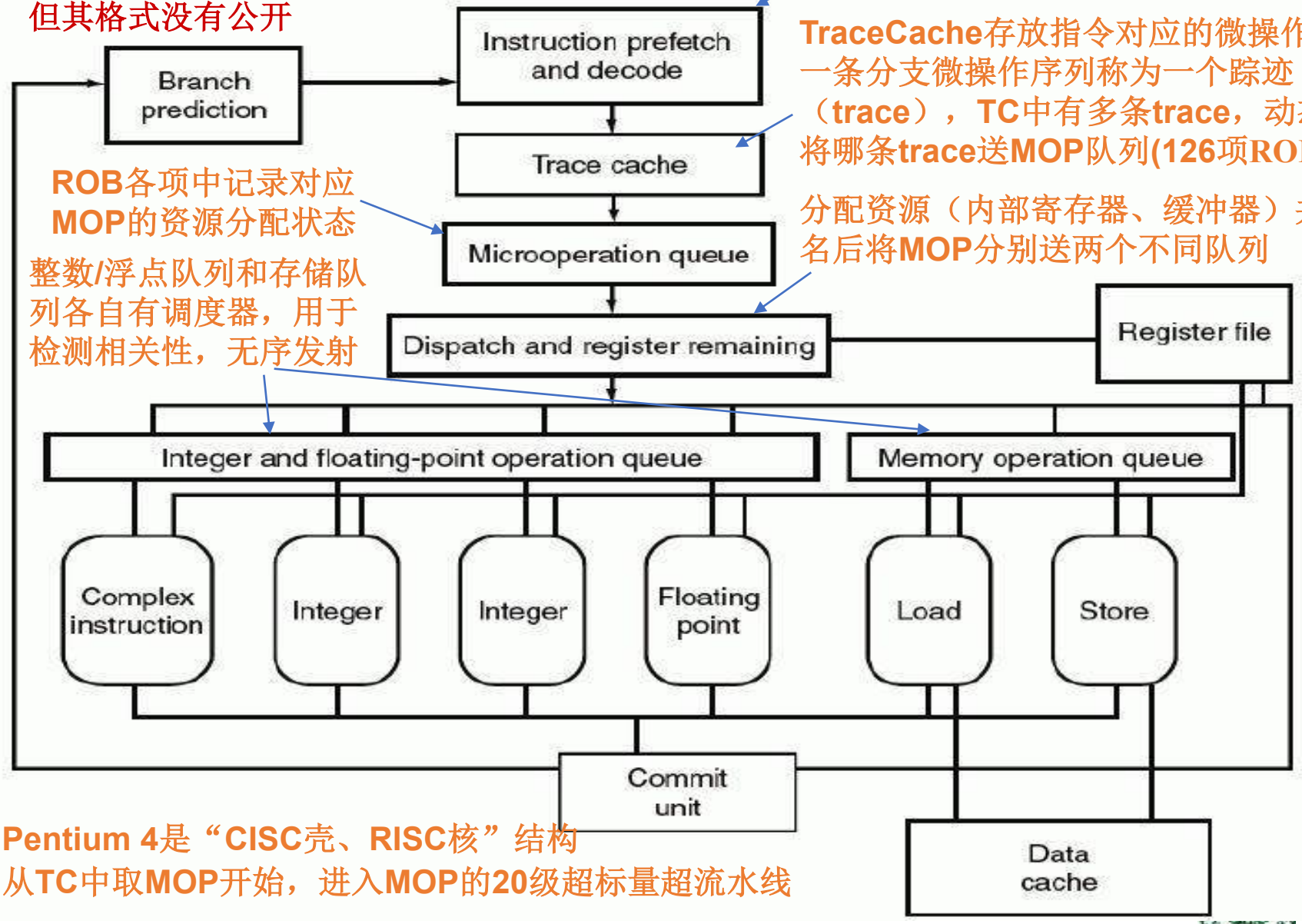


Pentium4 流水线结构部分

每个MOP相当于一条RISC指令

但其格式没有公开

指令译码结果是对应的若干条微操作MOP



TraceCache存放指令对应的微操作序列，一条分支微操作序列称为一个踪迹（trace），TC中有多条trace，动态预测将哪条trace送MOP队列(126项ROB)

ROB各项中记录对应MOP的资源分配状态

分配资源（内部寄存器、缓冲器）并重命名后将MOP分别送两个不同队列

整数/浮点队列和存储队列各自有调度器，用于检测相关性，无序发射

Pentium 4是“CISC壳、RISC核”结构

从TC中取MOP开始，进入MOP的20级超标量超流水线

Pentium 4的指令译码 - 对指令功能进行分解

- 指令译码逻辑：
  - 功能：将指令转换为一组基本操作，称为微操作MOP
  - 输入：程序中的指令
  - 输出：微操作（简单计算任务）

例1：addl %eax, %edx 的译码结果为什么？

一个“加法”操作（对应一个微操作MOP）

例2：addl %eax, 4 (%edx) 的译码结果呢？

四个简单操作（对应四个微操作MOP）：

“地址计算”：Reg[%edx]+4->addr

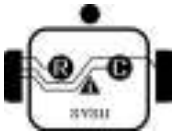
“装入”：Mem[addr]->Reg[Rtemp]

“加法”：Reg[Rtemp]+Reg[%eax]->Reg[Rtemp]

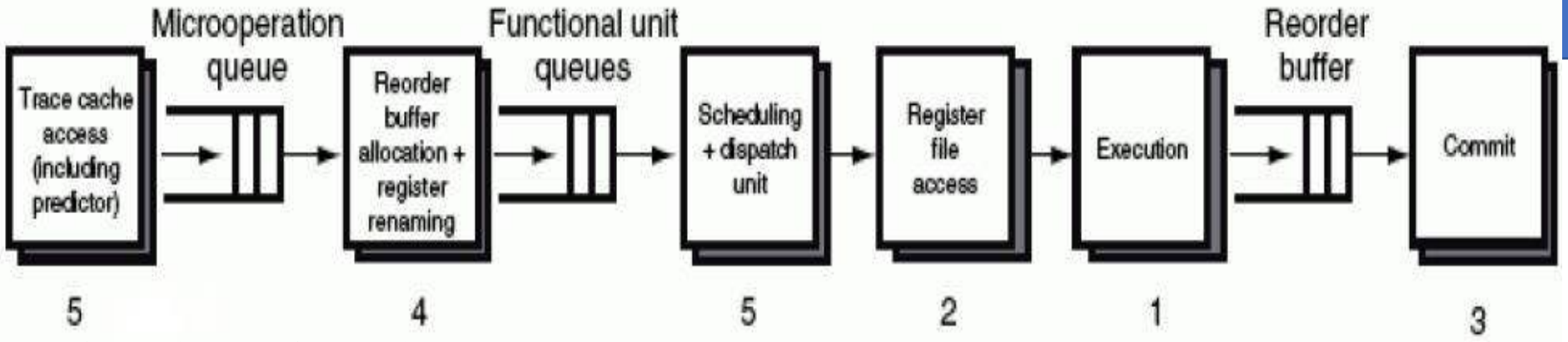
“存数”：Reg[Rtemp]->Mem[addr]

一个微操作相当于一条RISC指令，译码生成的微操作序列被存放到Trace Cache中



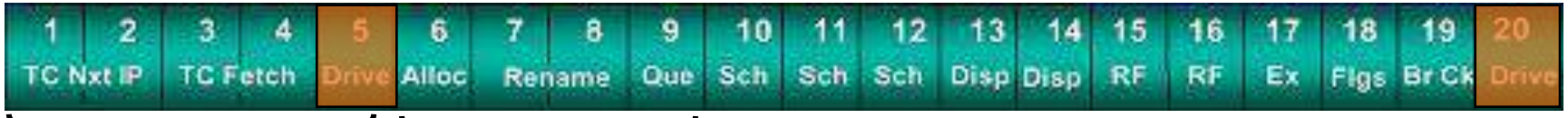


Pentium4 的20级超流水线(Hyper-pipeline)



整数运算微操作流水线为20级，浮点为29级（执行阶段的长度不同）

两个drive段用于芯片内传输信号的驱动，使其保证长距离传输



沿一个踪迹顺序取MOP，直到遇到一条转移MOP，通过BTB2预测下个踪迹开始点，继续取MOP送ROB/Alloc/Ren部件。预测目标处MOP不在时，要通知指令预取器，快从L2中取指令并译码。

一个周期3条MOP送ROB。ROB有126项，记录每个MOP及分配的资源和执行状态，根据资源分配情况进行寄存器重命名后，分别送两个MOP队列中进行排队。

每个队列按FIFO将MOP送到各自的调度器，在调度器中进行数据相关性检测，当所有源操作都就绪时，将MOP发射到对应的执行部件。是“无序”发射。

被发射的MOP开始读取物理寄存器中的源操作数，或从旁路由L1-D Cache读取。

在不同的执行部件中执行。每个部件执行时间长短不同

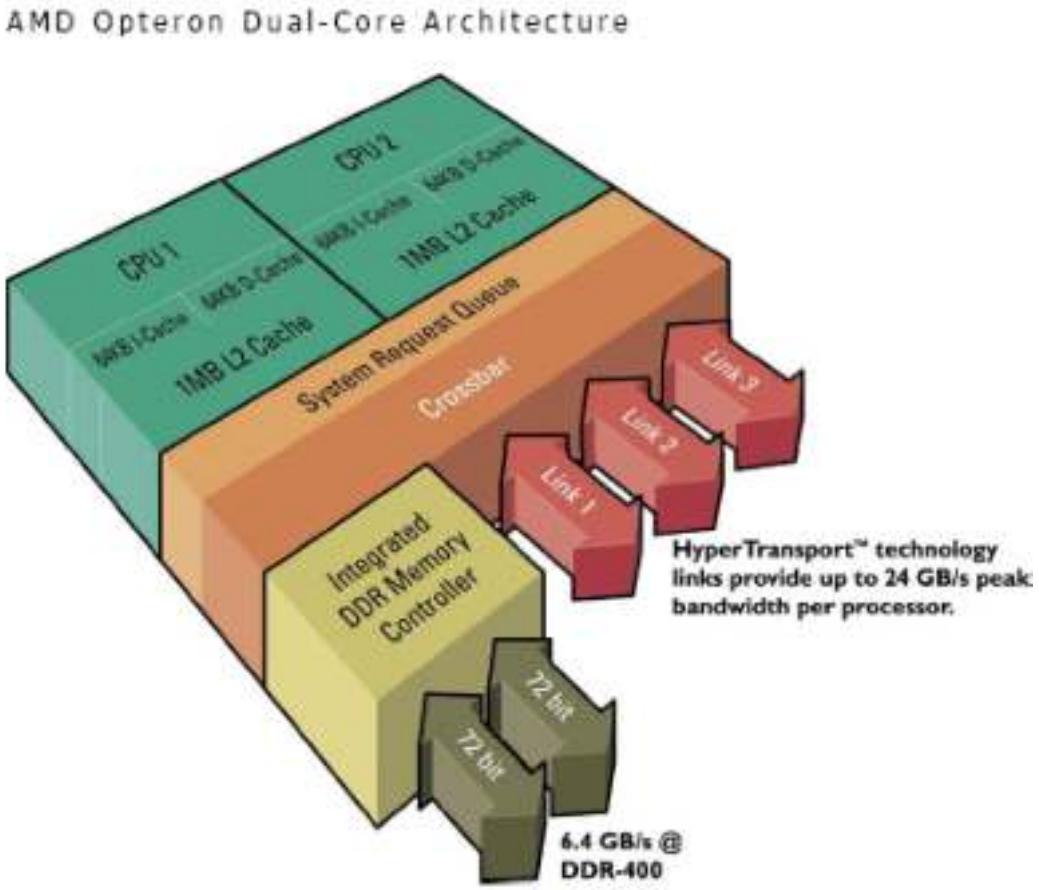
建立标志信息ZF/CF等，并将执行结果写入物理寄存器。对BTB2中预测是否正确进行确认及相应处理





What is next?

- Multi-Core
  - AMD X2 (2004)
  - Intel Montecito(late 2005)



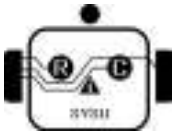
Multi-core mapps multi-thread applications to real parallelism

- **Already available in RISC platform;**
- **IBM Power5, Sum Ulta SPARC IV, HP PA-RISC 8800**

本讲小结

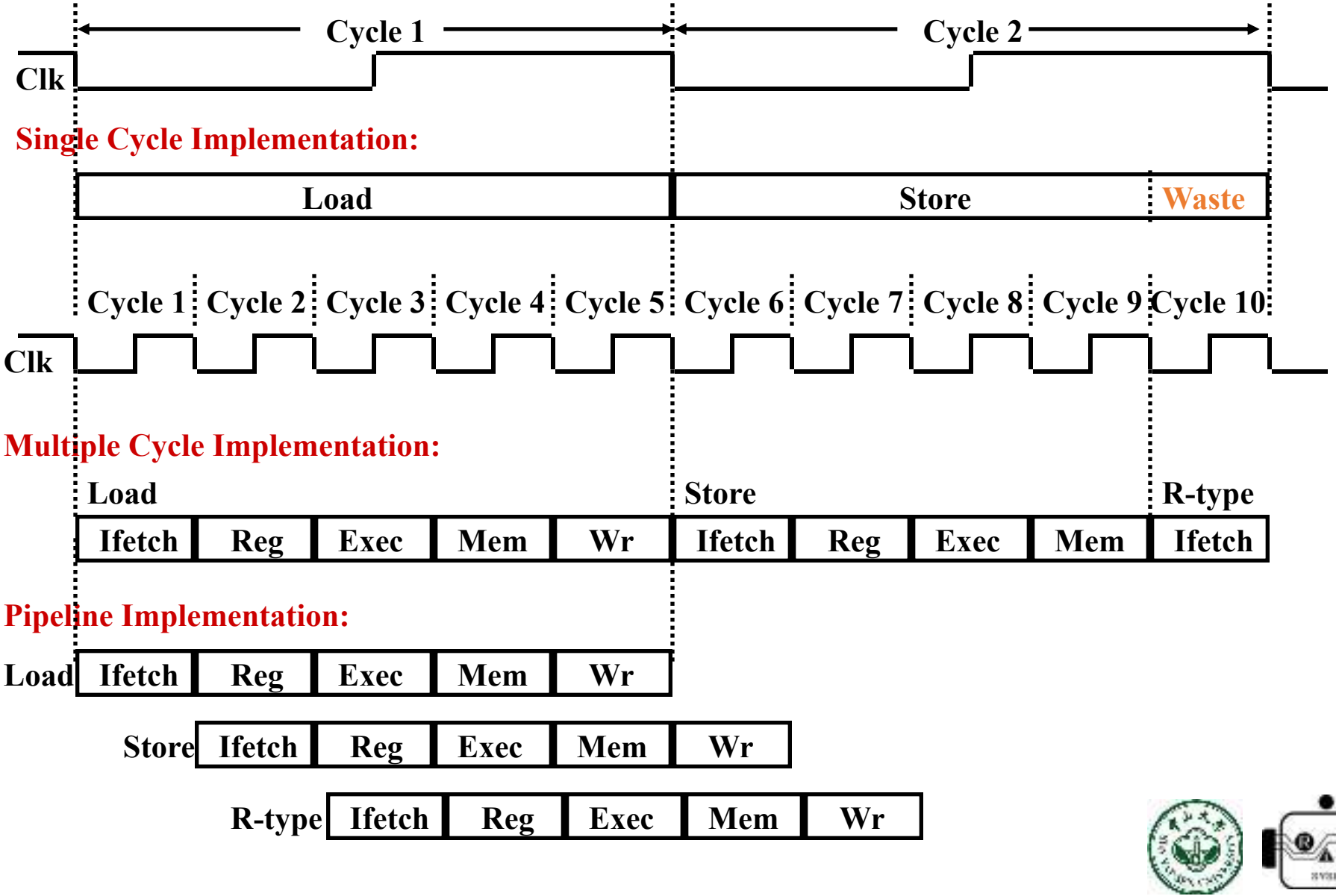
- 有以下两种指令级并行(ILP)技术（即：高性能流水线形式）
  - 超流水线：更多的流水线级数
  - 多发射流水线：同时发射多个指令，有多条流水线同时进行
    - 静态多发射（VLIW处理器+编译器静态调度）
    - 动态多发射（超标量处理器+动态流水线调度）





本章总结1

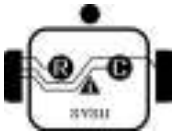
单周期, 多周期 和 流水线比较



本章总结2

- 指令流水线的设计
  - 将每条指令的执行规整化为若干个同样的流水阶段
  - 每个流水阶段的执行时间一样，都等于一个时钟
  - 理想情况下，每个时钟有一条指令进入流水线，也有一条指令执行结束
  - 每个流水段中的部件都是组合逻辑加寄存器，组合逻辑中产生的结果在时钟到来时被存储到寄存器（如：程序计数器、条件码寄存器、流水段寄存器）
  - 每两个相邻流水段之间的流水线寄存器，用以记录所有在后面阶段要用到的各种信息，有哪些呢？
    - 控制信号、指令的代码、参加运算的操作数、指令运算结果、指令异常信息、寄存器读口地址、寄存器写口地址、存储器地址、新的PC值等。
  - 指令译码得到的控制信号通过流水线寄存器传送到后面各个流水段中

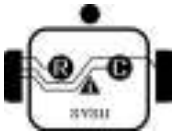




本章总结3

- 指令流水线的局限性
  - 并不是每个流水段都一样长
  - 随着流水线深度的增加，流水线寄存器的额外开销比例也增大
  - 指令在资源冲突、数据相关或控制相关时会发生流水线冒险
- 指令流水线的执行效率
  - 吞吐率：比非流水线方式下大大提高
  - 指令执行时间：相对于非流水线方式，一条指令的执行时间延长了
- 提高流水线指令效率的高级流水线技术
  - 超流水线：级数更多的流水线
  - 多发射流水线：同时发射多条指令的流水线
    - 静态多发射：VLIW结构、编译器静态推测
    - 动态多发射：超标量结构、硬件动态推测调度

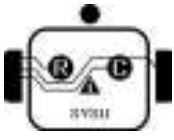




本章总结4

- 结构冒险（资源冲突）：多条指令同时使用同一个功能部件
  - 规定每个功能部件在一条指令中只能被用一次
  - 规定每个功能部件只能在某个特定的阶段被用
  - 指令存储器(Code Cache)和数据存储器(Data Cache)分开
- 数据冒险（数据相关）：前面指令的结果是后面指令的操作数
  - 软件阻塞：（如：编译器）在后面的数据相关指令前插入nop指令
  - 硬件阻塞：在后面数据相关指令的特定流水段插入“气泡”以“阻塞”指令继续执行，直到取得所需数据为止
  - “转发”（旁路）：把前面指令执行过程中得到的数据直接传送到后面指令。
  - 对于取数后直接使用的情况（如：Load指令取出的数据是随后的运算指令的操作数），则采用“阻塞加转发”的方式解决数据冒险





本章总结5

- 控制冒险（控制相关）：返回指令、分支指令等可能改变顺序增量的PC值，由于获取转移目标地址的时间较长，使得在目标地址产生前已经有指令被取到流水线中，如果已经取出执行的指令不是正确的指令，则发生控制冒险。
  - 软件阻塞：（如：编译器）在控制相关指令后面插入nop指令
  - 硬件阻塞：在控制相关指令后面的指令被取出前插入“气泡”，使流水线停顿若干时钟，直到控制相关指令得到正确的PC值为止
  - 采用“分支预测”技术。简单（静态）地预测每次分支结果都一样，或根据分支指令执行历史进行动态预测，动态预测能达到90%以上的成功率
  - 采用延迟分支技术。将前面一条与分支指令无关的指令放到分支指令后面执行，这样，流水线不会发生阻塞现象。这种对指令顺序进行调整的工作在程序编译阶段完成

联系方式

- Acknowledgements:
- This slides contains materials from following lectures:
  - Computer Architecture (ETH，NUDT，USTC)

- Research Area：
  - 计算机视觉与机器人应用计算加速，
  - 人工智能和深度学习芯片及智能计算机

- Contact：
  - 中山大学计算机学院
  - 管理学院D101（图书馆右侧）
  - 机器人与智能计算实验室
  - cheng83@mail.sysu.edu.cn